

=====

FILE: migrations\e63\_rheumatology\_immunology\_up.sql

=====

-- Migration for Rheumatology & Immunology Specialized Data  
-- Target: namaweb/migrations/e63\_rheumatology\_immunology\_up.sql

```
CREATE TABLE IF NOT EXISTS rheum_joint_logs (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,  
    tenant_id UUID NOT NULL,  
    joint_name TEXT NOT NULL, -- e.g., 'MCP 2-Right', 'PIP 3-Left'  
    tenderness_grade INTEGER CHECK (tenderness_grade BETWEEN 0 AND 3),  
    swelling_grade INTEGER CHECK (swelling_grade BETWEEN 0 AND 3),  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE IF NOT EXISTS rheum_activity_scores (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,  
    tenant_id UUID NOT NULL,  
    das28_score DECIMAL(4,2),  
    la28_score DECIMAL(4,2),  
    activity_level TEXT CHECK (activity_level IN ('Remission', 'Low', 'Moderate', 'High')),  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE IF NOT EXISTS rheum_biologic_tracking (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,  
    tenant_id UUID NOT NULL,  
    drug_name TEXT NOT NULL,  
    screening_status TEXT CHECK (screening_status IN ('Pending', 'Clear', 'Flagged')),  
    last_dose_date DATE,  
    next_dose_date DATE,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

-- Enable RLS

```
ALTER TABLE rheum_joint_logs ENABLE ROW LEVEL SECURITY;  
ALTER TABLE rheum_activity_scores ENABLE ROW LEVEL SECURITY;  
ALTER TABLE rheum_biologic_tracking ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY tenant_isolation_policy ON rheum_joint_logs  
    USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

```
CREATE POLICY tenant_isolation_policy ON rheum_activity_scores  
    USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

```
CREATE POLICY tenant_isolation_policy ON rheum_biologic_tracking  
    USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

=====

FILE: migrations\e64\_infectious\_diseases\_down.sql

=====

-- Down migration for Infectious Diseases & Infection Control  
-- Target: namaweb/migrations/e64\_infectious\_diseases\_down.sql

DROP TABLE IF EXISTS antimicrobial\_stewardship\_logs;  
DROP TABLE IF EXISTS infectious\_isolation\_logs;

=====

FILE: migrations\e64\_infectious\_diseases\_up.sql

=====

-- Migration for Infectious Diseases & Infection Control  
-- Target: namaweb/migrations/e64\_infectious\_diseases\_up.sql

```
CREATE TABLE IF NOT EXISTS infectious_isolation_logs (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,  
    tenant_id UUID NOT NULL,  
    isolation_type TEXT CHECK (isolation_type IN ('Airborne', 'Droplet', 'Contact', 'Protective')),  
    room_number TEXT,  
    start_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    end_date TIMESTAMP WITH TIME ZONE,  
    status TEXT CHECK (status IN ('Active', 'Resolved', 'Transferred')),  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE IF NOT EXISTS antimicrobial_stewardship_logs (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,  
    tenant_id UUID NOT NULL,  
    drug_name TEXT NOT NULL,  
    dose TEXT,  
    start_date DATE,  
    stop_date DATE,  
    indication TEXT,  
    stewardship_approval BOOLEAN DEFAULT FALSE,  
    review_date DATE,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

-- Enable RLS

ALTER TABLE infectious\_isolation\_logs ENABLE ROW LEVEL SECURITY;  
ALTER TABLE antimicrobial\_stewardship\_logs ENABLE ROW LEVEL SECURITY;

```
CREATE POLICY tenant_isolation_policy ON infectious_isolation_logs  
    USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

```
CREATE POLICY tenant_isolation_policy ON antimicrobial_stewardship_logs
    USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

```
=====
FILE: migrations\e65_dermatology_down.sql
=====
```

```
-- Down migration for Dermatology & Cosmetic Surgery Specialized Data
-- Target: namaweb/migrations/e65_dermatology_down.sql
```

```
DROP TABLE IF EXISTS dermatology_cosmetic_logs;
DROP TABLE IF EXISTS dermatology_lesion_logs;
```

```
=====
FILE: migrations\e65_dermatology_up.sql
=====
```

```
-- Migration for Dermatology & Cosmetic Surgery Specialized Data
-- Target: namaweb/migrations/e65_dermatology_up.sql
```

```
CREATE TABLE IF NOT EXISTS dermatology_lesion_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    tenant_id UUID NOT NULL,
    anatomical_site TEXT NOT NULL,
    morphology TEXT,
    size_mm DECIMAL(5,2),
    color TEXT,
    border_type TEXT,
    texture TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE IF NOT EXISTS dermatology_cosmetic_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    tenant_id UUID NOT NULL,
    procedure_type TEXT,
    product_used TEXT,
    dose_volume DECIMAL(6,2),
    site TEXT,
    session_number INTEGER,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Enable RLS
ALTER TABLE dermatology_lesion_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE dermatology_cosmetic_logs ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY tenant_isolation_policy ON dermatology_lesion_logs
    USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

```
CREATE POLICY tenant_isolation_policy ON dermatology_cosmetic_logs
    USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

```
=====
FILE: migrations\e65_oncology_extensions_down.sql
=====
```

```
-- Migration: e65_oncology_extensions_down.sql
-- Description: Reverses the oncology extensions migration.
```

```
BEGIN;
```

```
DROP TABLE IF EXISTS bmt_monitoring;
DROP TABLE IF EXISTS oncology_chemo_logs;
```

```
COMMIT;
```

```
=====
FILE: migrations\e65_oncology_extensions_up.sql
=====
```

```
-- Migration: e65_oncology_extensions_up.sql
-- Description: Adds specialized tables for Oncology and Hematology.
-- Compliance: PDPL / RLS Enabled
```

```
BEGIN;
```

```
-- 1. Chemotherapy Cycle Logs
```

```
CREATE TABLE IF NOT EXISTS oncology_chemo_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    cycle_number INTEGER,
    drug_name VARCHAR(255),
    dose_mg FLOAT,
    infusion_duration_min INTEGER,
    toxicity_grade INTEGER, -- Grade 1-4
    administered_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    notes TEXT,
    CONSTRAINT fk_tenant_onco FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
    CONSTRAINT fk_patient_onco FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);
```

```
-- 2. BMT (Bone Marrow Transplant) Monitoring
```

```
CREATE TABLE IF NOT EXISTS bmt_monitoring (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
```

```

patient_id UUID NOT NULL,
transplant_date DATE,
cd34_count_per_kg FLOAT,
engraftment_date DATE,
gvhd_grade INTEGER, -- Graft-vs-Host Disease
last_biopsy_result TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_tenant_bmt FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
CONSTRAINT fk_patient_bmt FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

```

```

-- Enable Row Level Security (RLS)

```

```

ALTER TABLE oncology_chemo_logs ENABLE ROW LEVEL SECURITY;

```

```

ALTER TABLE bmt_monitoring ENABLE ROW LEVEL SECURITY;

```

```

-- Create RLS Policies (Golden Access Rule: Tenant Isolation)

```

```

CREATE POLICY tenant_isolation_policy ON oncology_chemo_logs
    USING (tenant_id = current_setting('app.current_tenant')::UUID);

```

```

CREATE POLICY tenant_isolation_policy ON bmt_monitoring
    USING (tenant_id = current_setting('app.current_tenant')::UUID);

```

```

COMMIT;

```

```

=====

```

```

FILE: migrations\e66_general_surgery_down.sql

```

```

=====

```

```

-- Down migration for General Surgery Specialized Data

```

```

-- Target: namaweb/migrations/e66_general_surgery_down.sql

```

```

DROP TABLE IF EXISTS surgical_robotic_logs;

```

```

DROP TABLE IF EXISTS surgical_intra_op_logs;

```

```

DROP TABLE IF EXISTS surgical_safety_checklists;

```

```

=====

```

```

FILE: migrations\e66_general_surgery_up.sql

```

```

=====

```

```

-- Migration for General Surgery Specialized Data

```

```

-- Target: namaweb/migrations/e66_general_surgery_up.sql

```

```

CREATE TABLE IF NOT EXISTS surgical_safety_checklists (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    procedure_id UUID NOT NULL,
    tenant_id UUID NOT NULL,
    phase TEXT CHECK (phase IN ('Sign-in', 'Time-out', 'Sign-out')),
    checklist_completed BOOLEAN DEFAULT FALSE,
    verified_by UUID NOT NULL REFERENCES system_users(id),
    verification_timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

```

```

        created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
    );

CREATE TABLE IF NOT EXISTS surgical_intra_op_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    procedure_id UUID NOT NULL,
    tenant_id UUID NOT NULL,
    estimated_blood_loss DECIMAL(6,2),
    actual_blood_loss DECIMAL(6,2),
    anesthesia_start TIMESTAMP WITH TIME ZONE,
    anesthesia_end TIMESTAMP WITH TIME ZONE,
    tourniquet_time_min INTEGER,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS surgical_robotic_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    procedure_id UUID NOT NULL,
    tenant_id UUID NOT NULL,
    robot_model TEXT,
    console_time_min INTEGER,
    port_locations TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE surgical_safety_checklists ENABLE ROW LEVEL SECURITY;
ALTER TABLE surgical_intra_op_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE surgical_robotic_logs ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON surgical_safety_checklists
    USING (tenant_id = current_setting('app.current_tenant')::uuid);

CREATE POLICY tenant_isolation_policy ON surgical_intra_op_logs
    USING (tenant_id = current_setting('app.current_tenant')::uuid);

CREATE POLICY tenant_isolation_policy ON surgical_robotic_logs
    USING (tenant_id = current_setting('app.current_tenant')::uuid);

=====
FILE: migrations\e67_cardio_thoracic_vascular_down.sql
=====

-- Down migration for Cardiothoracic & Vascular Surgery Specialized Data
-- Target: namaweb/migrations/e67_cardio_thoracic_vascular_down.sql

DROP TABLE IF EXISTS vascular_graft_registry;
DROP TABLE IF EXISTS cardio_thoracic_metrics;

=====

```

FILE: migrations\e67\_cardio\_thoracic\_vascular\_up.sql

=====

-- Migration for Cardiothoracic & Vascular Surgery Specialized Data  
-- Target: namaweb/migrations/e67\_cardio\_thoracic\_vascular\_up.sql

```
CREATE TABLE IF NOT EXISTS cardio_thoracic_metrics (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    procedure_id UUID NOT NULL,  
    tenant_id UUID NOT NULL,  
    cpb_start TIMESTAMP WITH TIME ZONE,  
    cpb_end TIMESTAMP WITH TIME ZONE,  
    cross_clamp_start TIMESTAMP WITH TIME ZONE,  
    cross_clamp_end TIMESTAMP WITH TIME ZONE,  
    pump_flow_rate DECIMAL(6,2),  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE IF NOT EXISTS vascular_graft_registry (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    procedure_id UUID NOT NULL,  
    tenant_id UUID NOT NULL,  
    graft_type TEXT CHECK (graft_type IN ('Synthetic', 'Autologous')),  
    material TEXT,  
    diameter_mm DECIMAL(4,2),  
    location TEXT,  
    patency_check_result TEXT,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);  
  
-- Enable RLS  
ALTER TABLE cardio_thoracic_metrics ENABLE ROW LEVEL SECURITY;  
ALTER TABLE vascular_graft_registry ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY tenant_isolation_policy ON cardio_thoracic_metrics  
    USING (tenant_id = current_setting('app.current_tenant')::uuid);  
  
CREATE POLICY tenant_isolation_policy ON vascular_graft_registry  
    USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

=====

FILE: migrations\e67\_rheuma\_extensions\_down.sql

=====

-- Migration: e67\_rheuma\_extensions\_down.sql  
-- Description: Reverses the rheumatology extensions migration.

```
BEGIN;  
  
DROP TABLE IF EXISTS autoimmune_markers;  
DROP TABLE IF EXISTS rheuma_scores;
```

COMMIT;

=====

FILE: migrations\e67\_rheuma\_extensions\_up.sql

=====

-- Migration: e67\_rheuma\_extensions\_up.sql  
-- Description: Adds specialized tables for Rheumatology and Immunology.  
-- Compliance: PDPL / RLS Enabled

BEGIN;

-- 1. Rheumatology Scoring (DAS28, SLEDAI)

```
CREATE TABLE IF NOT EXISTS rheuma_scores (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    patient_id UUID NOT NULL,  
    record_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    das28_score FLOAT,  
    sleda_score FLOAT,  
    joint_count_tender INTEGER,  
    joint_count_swollen INTEGER,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_tenant_rheum FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,  
    CONSTRAINT fk_patient_rheum FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE  
);
```

-- 2. Autoimmune Markers (Serology)

```
CREATE TABLE IF NOT EXISTS autoimmune_markers (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    patient_id UUID NOT NULL,  
    sample_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    ana_titer VARCHAR(50),  
    rf_level_iu_ml FLOAT,  
    anti_ccp_level FLOAT,  
    anti_dsdna_level FLOAT,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_tenant_marker FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,  
    CONSTRAINT fk_patient_marker FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE  
);
```

-- Enable Row Level Security (RLS)

ALTER TABLE rheuma\_scores ENABLE ROW LEVEL SECURITY;

ALTER TABLE autoimmune\_markers ENABLE ROW LEVEL SECURITY;

-- Create RLS Policies (Golden Access Rule: Tenant Isolation)

```
CREATE POLICY tenant_isolation_policy ON rheuma_scores  
    USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

```
CREATE POLICY tenant_isolation_policy ON autoimmune_markers  
    USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

COMMIT;

=====

FILE: migrations\e68\_infectious\_extensions\_down.sql

=====

-- Migration: e68\_infectious\_extensions\_down.sql

-- Description: Reverses the infectious diseases extensions migration.

BEGIN;

DROP TABLE IF EXISTS asp\_reviews;

DROP TABLE IF EXISTS culture\_results;

COMMIT;

=====

FILE: migrations\e68\_infectious\_extensions\_up.sql

=====

-- Migration: e68\_infectious\_extensions\_up.sql

-- Description: Adds specialized tables for Infectious Diseases and Antimicrobial Stewardship.

-- Compliance: PDPL / RLS Enabled

BEGIN;

-- 1. Microbiology Culture Results

```
CREATE TABLE IF NOT EXISTS culture_results (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    patient_id UUID NOT NULL,  
    sample_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    sample_site VARCHAR(255),  
    pathogen_name VARCHAR(255),  
    sensitivity_profile JSONB, -- { "drug_a": "S", "drug_b": "R" }  
    is_critical BOOLEAN DEFAULT FALSE,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_tenant_inf FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,  
    CONSTRAINT fk_patient_inf FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE  
);
```

-- 2. Antimicrobial Stewardship (ASP) Reviews

```
CREATE TABLE IF NOT EXISTS asp_reviews (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    patient_id UUID NOT NULL,  
    review_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    original_drug VARCHAR(255),  
    suggested_drug VARCHAR(255),
```

```
justification TEXT,
status VARCHAR(50), -- e.g., Pending, Approved, Rejected
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_tenant_asp FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
CONSTRAINT fk_patient_asp FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);
```

```
-- Enable Row Level Security (RLS)
ALTER TABLE culture_results ENABLE ROW LEVEL SECURITY;
ALTER TABLE asp_reviews ENABLE ROW LEVEL SECURITY;

-- Create RLS Policies (Golden Access Rule: Tenant Isolation)
CREATE POLICY tenant_isolation_policy ON culture_results
    USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY tenant_isolation_policy ON asp_reviews
    USING (tenant_id = current_setting('app.current_tenant')::UUID);

COMMIT;
```

```
=====
FILE: migrations\e68_neurosurgery_spine_down.sql
=====
```

```
-- Migration: e68_neurosurgery_spine_down.sql
-- Purpose: Reverse changes from e68_neurosurgery_spine_up.sql
```

```
BEGIN;

DROP TABLE IF EXISTS spine_stability_metrics;
DROP TABLE IF EXISTS intracranial_pressure_logs;
DROP TABLE IF EXISTS neuro_surgical_logs;

COMMIT;
```

```
=====
FILE: migrations\e68_neurosurgery_spine_up.sql
=====
```

```
-- Migration: e68_neurosurgery_spine_up.sql
-- Purpose: Implement specialized tables for Neurosurgery and Spine Surgery
```

```
BEGIN;

-- 1. Neurosurgical Intervention Logs
CREATE TABLE IF NOT EXISTS neuro_surgical_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    surgeon_id UUID NOT NULL,
```

```

operation_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
procedure_type VARCHAR(100), -- e.g., Craniotomy, Laminectomy, Ventriculostomy
approach VARCHAR(100), -- e.g., Pterional, Midline, Posterior
side VARCHAR(10), -- Left, Right, Bilateral
duration_minutes INTEGER,
blood_loss_ml INTEGER,
intraop_findings TEXT,
complications TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. Intracranial Pressure (ICP) & Cerebral Perfusion Pressure (CPP) Logs
CREATE TABLE IF NOT EXISTS intracranial_pressure_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    log_time TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    icp_value FLOAT, -- mmHg
    map_value FLOAT, -- Mean Arterial Pressure (mmHg)
    cpp_value FLOAT, -- Cerebral Perfusion Pressure (MAP - ICP)
    gcs_score INTEGER, -- Glasgow Coma Scale
    pupil_status TEXT,
    nurse_id UUID,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Spine Stability & ASIA Metrics
CREATE TABLE IF NOT EXISTS spine_stability_metrics (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    assessment_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    spinal_level VARCHAR(20), -- e.g., C5-C6, T12-L1
    asia_impairment_grade VARCHAR(10), -- A, B, C, D, E
    motor_score INTEGER,
    sensory_score INTEGER,
    stability_grade VARCHAR(50), -- Stable, Unstable
    fusion_recommended BOOLEAN DEFAULT FALSE,
    surgeon_id UUID,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE neuro_surgical_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE intracranial_pressure_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE spine_stability_metrics ENABLE ROW LEVEL SECURITY;

-- RLS Policies
CREATE POLICY neuro_surgical_logs_tenant_policy ON neuro_surgical_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY intracranial_pressure_logs_tenant_policy ON intracranial_pressure_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

```

```
CREATE POLICY spine_stability_metrics_tenant_policy ON spine_stability_metrics
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

COMMIT;
```

```
=====
FILE: migrations\e69_derm_extensions_down.sql
=====
```

```
-- Migration: e69_derm_extensions_down.sql
-- Description: Reverses the dermatology extensions migration.
```

```
BEGIN;
```

```
DROP TABLE IF EXISTS derm_cosmetic_logs;
DROP TABLE IF EXISTS derm_lesion_records;
```

```
COMMIT;
```

```
=====
FILE: migrations\e69_derm_extensions_up.sql
=====
```

```
-- Migration: e69_derm_extensions_up.sql
-- Description: Adds specialized tables for Dermatology and Cosmetic Procedures.
-- Compliance: PDPL / RLS Enabled
```

```
BEGIN;
```

```
-- 1. Dermatology Lesion Records
```

```
CREATE TABLE IF NOT EXISTS derm_lesion_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    lesion_location VARCHAR(255),
    description TEXT,
    biopsy_status VARCHAR(50), -- e.g., Pending, Benign, Malignant
    image_path TEXT, -- Path to phi_vault/
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tenant_derm FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
    CONSTRAINT fk_patient_derm FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);
```

```
-- 2. Cosmetic & Laser Logs
```

```
CREATE TABLE IF NOT EXISTS derm_cosmetic_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    procedure_type VARCHAR(100),
```

```
laser_settings JSONB, -- { "wavelength": "532nm", "energy": "10mJ" }
session_number INTEGER,
outcome_notes TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_tenant_cosm FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
CONSTRAINT fk_patient_cosm FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);
```

```
-- Enable Row Level Security (RLS)
```

```
ALTER TABLE derm_lesion_records ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE derm_cosmetic_logs ENABLE ROW LEVEL SECURITY;
```

```
-- Create RLS Policies (Golden Access Rule: Tenant Isolation)
```

```
CREATE POLICY tenant_isolation_policy ON derm_lesion_records
```

```
    USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

```
CREATE POLICY tenant_isolation_policy ON derm_cosmetic_logs
```

```
    USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e69_orthopedics_trauma_down.sql
```

```
=====
```

```
-- Migration: e69_orthopedics_trauma_down.sql
```

```
-- Purpose: Reverse changes from e69_orthopedics_trauma_up.sql
```

```
BEGIN;
```

```
DROP TABLE IF EXISTS fracture_management_logs;
```

```
DROP TABLE IF EXISTS joint_replacement_registry;
```

```
DROP TABLE IF EXISTS ortho_surgical_logs;
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e69_orthopedics_trauma_up.sql
```

```
=====
```

```
-- Migration: e69_orthopedics_trauma_up.sql
```

```
-- Purpose: Implement specialized tables for Orthopedics & Traumatology
```

```
BEGIN;
```

```
-- 1. Orthopedic Surgical Logs
```

```
CREATE TABLE IF NOT EXISTS ortho_surgical_logs (
```

```
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    tenant_id UUID NOT NULL,
```

```
    patient_id UUID NOT NULL,
```

```

surgeon_id UUID NOT NULL,
operation_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
procedure_type VARCHAR(100), -- e.g., Total Hip Arthroplasty, ACL Reconstruction
approach VARCHAR(100), -- e.g., Anterior, Posterior, Parapatellar
duration_minutes INTEGER,
blood_loss_ml INTEGER,
intraop_findings TEXT,
complications TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

-- 2. Joint Replacement Registry (Implant Tracking)

```

CREATE TABLE IF NOT EXISTS joint_replacement_registry (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    procedure_id UUID REFERENCES ortho_surgical_logs(id),
    joint_replaced VARCHAR(50), -- Hip, Knee, Shoulder, Ankle
    implant_brand VARCHAR(100),
    implant_model VARCHAR(100),
    implant_serial_number VARCHAR(100),
    implant_size VARCHAR(20),
    alignment_angle FLOAT, -- Degrees
    stability_grade VARCHAR(50),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

-- 3. Fracture Management & Reduction Logs

```

CREATE TABLE IF NOT EXISTS fracture_management_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    fracture_site VARCHAR(100), -- e.g., Distal Radius, Femoral Neck
    ao_ota_classification VARCHAR(50), -- e.g., 32-A1
    reduction_type VARCHAR(50), -- Open, Closed, Percutaneous
    reduction_status VARCHAR(50), -- Anatomical, Acceptable, Poor
    fixation_method VARCHAR(100), -- e.g., Intramedullary Nail, Plate & Screws
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

-- Enable RLS

```

ALTER TABLE ortho_surgical_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE joint_replacement_registry ENABLE ROW LEVEL SECURITY;
ALTER TABLE fracture_management_logs ENABLE ROW LEVEL SECURITY;

```

-- RLS Policies

```

CREATE POLICY ortho_surgical_logs_tenant_policy ON ortho_surgical_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY joint_replacement_registry_tenant_policy ON joint_replacement_registry
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY fracture_management_logs_tenant_policy ON fracture_management_logs

```

```

        USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

COMMIT;

=====
FILE: migrations\e6_01_mar_administrations_down.sql
=====

-- e6_01_mar_administrations_down.sql (rollback of e6_01_mar_administrations_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ???? ?????? ????? (DROP TABLE ???? ??????? ????????)? ?? ????? ??????? idempotent (IF EXISTS).
-- LOWER-3: DROP TABLE FIRST so a failed/locked DROP POLICY can never block the table teardown;
-- the trailing DROP POLICY/INDEX are guarded best-effort cleanup for a partial-state rollback.
BEGIN;

-- Primary teardown: DROP TABLE cascades its own dependent policies + indexes.
DROP TABLE IF EXISTS mar_administrations;

-- Best-effort residue cleanup (only fire if the table somehow survived; all IF EXISTS so a no-op is safe).
DROP POLICY IF EXISTS rls_mar_administrations_tenant_isolation ON mar_administrations;
DROP INDEX IF EXISTS idx_mar_administrations_prescription_ref;
DROP INDEX IF EXISTS idx_mar_administrations_patient_id;
DROP INDEX IF EXISTS idx_mar_administrations_tenant_id;

COMMIT;

=====
FILE: migrations\e6_01_mar_administrations_up.sql
=====

-- =====
-- e6_01_mar_administrations_up.sql
-- E6 NURSING / MAR ? Medication Administration Record (barcode 5-rights, server-enforced).
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ??? ????? ??????? (MAR) ??????? ??????? ??????? (pharmacy_prescriptions_queue) ????? ??????.
-- ?? ?? = ??? ????? ??? ?? ????? ?? ????? ??????? (??????/??????/??????/??????/?????) ?? ??? ??????.
-- prescription_ref ??? ?????? ??????? ?????? (??? pharmacy_dispose.prescription_id ?? E5)? NULL ?????
-- ??????? ??? ????????. ?????? status: given/refused/held. ?????? witness ??????? ?????? ???????.
-- ??? ????? ??? 150 ?????? FORCE RLS: tenant_id NOT NULL REFERENCES tenants(id) + ENABLE+FORCE RLS
-- + ?????? ??? tenant_id + ?????? tenant_id.
--
-- idempotent: CREATE TABLE IF NOT EXISTS + DROP/ADD CONSTRAINT IF EXISTS ??? status CHECK
-- + DROP POLICY IF EXISTS + CREATE INDEX IF NOT EXISTS. ????? ?? BEGIN/COMMIT.
-- =====
BEGIN;

CREATE TABLE IF NOT EXISTS mar_administrations (
    id SERIAL PRIMARY KEY,
    tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,

```

```

        facility_id INTEGER,
        patient_id INTEGER NOT NULL REFERENCES patients(id),          -- right-patient FK (server-resolve
d, never trusted)
        prescription_ref INTEGER REFERENCES pharmacy_prescriptions_queue(id), -- nullable: source dose row in pha
rmacy_prescriptions_queue
        medication TEXT NOT NULL,
        dose TEXT,
        route TEXT,
        scheduled_at TIMESTAMP,          -- the planned/due time the nurse acted against
        administered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- server clock (never client clock)
        administered_by INTEGER,          -- system_users.id of the nurse
        administered_by_name TEXT DEFAULT '',
        witness_by INTEGER,          -- second-nurse witness (high-alert drugs)
        witness_by_name TEXT DEFAULT '',
        status TEXT NOT NULL DEFAULT 'given',
        override_reason TEXT DEFAULT '',          -- dose/route/time-window override justification
        cds_warnings TEXT DEFAULT '',          -- bonus CDS allergy/interaction findings at admini
ster time
        notes TEXT DEFAULT '',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        CONSTRAINT chk_mar_status CHECK (status IN ('given', 'refused', 'held'))
);

-- idempotent: ensure the status CHECK exists even when the table pre-dates this migration
ALTER TABLE mar_administrations DROP CONSTRAINT IF EXISTS chk_mar_status;
ALTER TABLE mar_administrations ADD CONSTRAINT chk_mar_status CHECK (status IN ('given', 'refused', 'held'));

CREATE INDEX IF NOT EXISTS idx_mar_administrations_tenant_id ON mar_administrations (tenant_id);
CREATE INDEX IF NOT EXISTS idx_mar_administrations_patient_id ON mar_administrations (patient_id);
CREATE INDEX IF NOT EXISTS idx_mar_administrations_prescription_ref ON mar_administrations (prescription_ref);

ALTER TABLE mar_administrations ENABLE ROW LEVEL SECURITY;
ALTER TABLE mar_administrations FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_mar_administrations_tenant_isolation ON mar_administrations;
CREATE POLICY rls_mar_administrations_tenant_isolation ON mar_administrations
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

=====
FILE: migrations\e6_01_mar_administrations_validate.sql
=====

-- e6_01_mar_administrations_validate.sql (run AFTER e6_01_mar_administrations_up.sql; read-only)
-- PASS = mar_administrations exists with tenant_id FK + FORCE RLS + isolation policy + indexes + status CHECK
.

SELECT
    (SELECT count(*) FROM information_schema.tables WHERE table_name='mar_administrations') AS mar_exists,
    -- expect 1

```

```

(SELECT count(*) FROM information_schema.columns
 WHERE table_name='mar_administrations'
    AND column_name IN ('tenant_id','patient_id','prescription_ref','medication','dose','route','scheduled_
at','administered_at','administered_by','witness_by','status','override_reason')) AS mar_cols, -- expect 12
(SELECT relforcerowsecurity FROM pg_class WHERE relname='mar_administrations') AS mar_force_rls,
-- expect t
(SELECT count(*) FROM pg_policies WHERE tablename='mar_administrations' AND policyname='rls_mar_administrati
ons_tenant_isolation') AS mar_policy, -- expect 1
(SELECT count(*) FROM pg_constraint WHERE conname='chk_mar_status') AS mar_status_check,
-- expect 1
(SELECT count(*) FROM pg_indexes WHERE tablename='mar_administrations' AND indexname='idx_mar_administration
s_tenant_id') AS mar_idx; -- expect 1

```

-- ----- tenant\_id is NOT NULL -----

```

SELECT count(*)::int AS tenant_id_not_null FROM information_schema.columns
 WHERE table_name='mar_administrations' AND column_name='tenant_id' AND is_nullable='NO';
-- expect 1

```

-- ----- FK to tenants(id) -----

```

SELECT count(*)::int AS fk_to_tenants FROM pg_constraint
 WHERE conrelid='mar_administrations'::regclass AND confrelid='tenants'::regclass AND contype='f';
-- expect 1

```

-- ----- FK patient\_id -> patients(id) (right-patient FK) -----

```

SELECT count(*)::int AS fk_to_patients FROM pg_constraint
 WHERE conrelid='mar_administrations'::regclass AND confrelid='patients'::regclass AND contype='f';
-- expect 1

```

-- ----- FK prescription\_ref -> pharmacy\_prescriptions\_queue(id) (nullable) -----

```

SELECT count(*)::int AS fk_to_rx_queue FROM pg_constraint
 WHERE conrelid='mar_administrations'::regclass AND confrelid='pharmacy_prescriptions_queue'::regclass AND co
ntype='f'; -- expect 1

```

=====

FILE: migrations\e6\_02\_nursing\_io\_records\_down.sql

=====

```

-- e6_02_nursing_io_records_down.sql (rollback of e6_02_nursing_io_records_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ???? ?????? ?????? (DROP TABLE ???? ???????? ???????? ????????)? ?? ?????? ???????? idempotent (IF EXISTS).
-- LOWER-3: DROP TABLE FIRST so a failed/locked DROP POLICY can never block the table teardown.

```

BEGIN;

DROP TABLE IF EXISTS nursing\_io\_records;

DROP POLICY IF EXISTS rls\_nursing\_io\_records\_tenant\_isolation ON nursing\_io\_records;

DROP INDEX IF EXISTS idx\_nursing\_io\_records\_patient\_id;

DROP INDEX IF EXISTS idx\_nursing\_io\_records\_tenant\_id;

COMMIT;

```

=====
FILE: migrations\e6_02_nursing_io_records_up.sql
=====

-- =====
-- e6_02_nursing_io_records_up.sql
-- E6 NURSING / MAR ? Intake/Output (I/O) records.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ????? ?????????? ?????????? (I/O) ?????? ?????? (?????/???/???) ?????? ???. direction: intake/output
.
--   ??? ??? 150 ????? FORCE RLS: tenant_id NOT NULL REFERENCES tenants(id) + ENABLE+FORCE RLS
--   + ????? ??? tenant_id + ????? tenant_id.
--
-- idempotent: CREATE TABLE IF NOT EXISTS + DROP/ADD CONSTRAINT IF EXISTS ??? direction CHECK
--   + DROP POLICY IF EXISTS + CREATE INDEX IF NOT EXISTS. ????? ?? BEGIN/COMMIT.
-- =====
BEGIN;

CREATE TABLE IF NOT EXISTS nursing_io_records (
    id SERIAL PRIMARY KEY,
    tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    facility_id INTEGER,
    patient_id INTEGER NOT NULL REFERENCES patients(id), -- right-patient FK
    direction TEXT NOT NULL DEFAULT 'intake',           -- intake | output
    category TEXT DEFAULT '',                           -- e.g. Oral, IV, Urine, Drain, Emesis
    amount_ml INTEGER NOT NULL DEFAULT 0,
    recorded_by INTEGER,
    recorded_by_name TEXT DEFAULT '',
    shift TEXT DEFAULT 'Morning',
    notes TEXT DEFAULT '',
    recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_io_direction CHECK (direction IN ('intake', 'output'))
);

ALTER TABLE nursing_io_records DROP CONSTRAINT IF EXISTS chk_io_direction;
ALTER TABLE nursing_io_records ADD CONSTRAINT chk_io_direction CHECK (direction IN ('intake', 'output'));

CREATE INDEX IF NOT EXISTS idx_nursing_io_records_tenant_id ON nursing_io_records (tenant_id);
CREATE INDEX IF NOT EXISTS idx_nursing_io_records_patient_id ON nursing_io_records (patient_id);

ALTER TABLE nursing_io_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE nursing_io_records FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_nursing_io_records_tenant_isolation ON nursing_io_records;
CREATE POLICY rls_nursing_io_records_tenant_isolation ON nursing_io_records
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

=====

FILE: migrations\e6\_02\_nursing\_io\_records\_validate.sql

=====

```
-- e6_02_nursing_io_records_validate.sql (run AFTER e6_02_nursing_io_records_up.sql; read-only)
-- PASS = nursing_io_records exists with tenant_id FK + FORCE RLS + isolation policy + indexes + direction CHE
CK.
```

SELECT

```
(SELECT count(*) FROM information_schema.tables WHERE table_name='nursing_io_records') AS io_exists,
    -- expect 1
(SELECT count(*) FROM information_schema.columns
    WHERE table_name='nursing_io_records'
    AND column_name IN ('tenant_id','patient_id','direction','category','amount_ml','recorded_by','shift'))
AS io_cols, -- expect 7
(SELECT relforcerowsecurity FROM pg_class WHERE relname='nursing_io_records') AS io_force_rls,
    -- expect t
(SELECT count(*) FROM pg_policies WHERE tablename='nursing_io_records' AND policyname='rls_nursing_io_record
s_tenant_isolation') AS io_policy, -- expect 1
(SELECT count(*) FROM pg_constraint WHERE conname='chk_io_direction') AS io_direction_check,
    -- expect 1
(SELECT count(*) FROM pg_indexes WHERE tablename='nursing_io_records' AND indexname='idx_nursing_io_records_
tenant_id') AS io_idx; -- expect 1
```

```
-- ----- tenant_id is NOT NULL -----
```

```
SELECT count(*)::int AS tenant_id_not_null FROM information_schema.columns
    WHERE table_name='nursing_io_records' AND column_name='tenant_id' AND is_nullable='NO';
    -- expect 1
```

```
-- ----- FK to tenants(id) -----
```

```
SELECT count(*)::int AS fk_to_tenants FROM pg_constraint
    WHERE conrelid='nursing_io_records'::regclass AND confrelid='tenants'::regclass AND contype='f';
    -- expect 1
```

```
-- ----- FK patient_id -> patients(id) -----
```

```
SELECT count(*)::int AS fk_to_patients FROM pg_constraint
    WHERE conrelid='nursing_io_records'::regclass AND confrelid='patients'::regclass AND contype='f';
    -- expect 1
```

=====

FILE: migrations\e6\_03\_nursing\_scores\_down.sql

=====

```
-- e6_03_nursing_scores_down.sql (rollback of e6_03_nursing_scores_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ??? ???? ???? (DROP TABLE ??? ?????? ??????? ???????)? ?? ???? ??????? idempotent (IF EXISTS).
-- LOWER-3: DROP TABLE FIRST so a failed/locked DROP POLICY can never block the table teardown.
BEGIN;
```

```
DROP TABLE IF EXISTS nursing_scores;
```

```
DROP POLICY IF EXISTS rls_nursing_scores_tenant_isolation ON nursing_scores;
DROP INDEX IF EXISTS idx_nursing_scores_patient_id;
DROP INDEX IF EXISTS idx_nursing_scores_tenant_id;
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e6_03_nursing_scores_up.sql
```

```
=====
```

```
-- =====
```

```
-- e6_03_nursing_scores_up.sql
```

```
-- E6 NURSING / MAR ? Nursing scores (Morse fall risk, Braden, NEWS/MEWS, Pain).
```

```
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
```

```
--
```

```
-- ?????: ??? ???????? ?????????? ?????????? ?? ??? ?????? (nursing_scores.js): score_type ????
```

```
-- score ?????? ?????????? band ???????? (Low/Moderate/High...) ? inputs_json ?????????? ??????
```

```
-- ??? ??? ? 150 ????? FORCE RLS: tenant_id NOT NULL REFERENCES tenants(id) + ENABLE+FORCE RLS
```

```
-- + ????? ??? tenant_id + ??? tenant_id.
```

```
--
```

```
-- idempotent: CREATE TABLE IF NOT EXISTS + DROP/ADD CONSTRAINT IF EXISTS ??? score_type CHECK
```

```
-- + DROP POLICY IF EXISTS + CREATE INDEX IF NOT EXISTS. ????? ?? BEGIN/COMMIT.
```

```
-- =====
```

```
BEGIN;
```

```
CREATE TABLE IF NOT EXISTS nursing_scores (
```

```
    id SERIAL PRIMARY KEY,
```

```
    tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
```

```
    facility_id INTEGER,
```

```
    patient_id INTEGER NOT NULL REFERENCES patients(id), -- right-patient FK
```

```
    score_type TEXT NOT NULL DEFAULT 'news', -- morse | braden | news | pain
```

```
    score INTEGER NOT NULL DEFAULT 0,
```

```
    band TEXT DEFAULT '',
```

```
    inputs_json TEXT DEFAULT '', -- raw observations the server scored (audit)
```

```
    recorded_by INTEGER,
```

```
    recorded_by_name TEXT DEFAULT '',
```

```
    notes TEXT DEFAULT '',
```

```
    recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    CONSTRAINT chk_nursing_score_type CHECK (score_type IN ('morse', 'braden', 'news', 'pain'))
```

```
);
```

```
ALTER TABLE nursing_scores DROP CONSTRAINT IF EXISTS chk_nursing_score_type;
```

```
ALTER TABLE nursing_scores ADD CONSTRAINT chk_nursing_score_type CHECK (score_type IN ('morse', 'braden', 'news', 'pain'));
```

```
CREATE INDEX IF NOT EXISTS idx_nursing_scores_tenant_id ON nursing_scores (tenant_id);
```

```
CREATE INDEX IF NOT EXISTS idx_nursing_scores_patient_id ON nursing_scores (patient_id);
```

```
ALTER TABLE nursing_scores ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE nursing_scores FORCE ROW LEVEL SECURITY;
```

```
DROP POLICY IF EXISTS rls_nursing_scores_tenant_isolation ON nursing_scores;
```

```
CREATE POLICY rls_nursing_scores_tenant_isolation ON nursing_scores
FOR ALL
USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;
```

```
=====
FILE: migrations\e6_03_nursing_scores_validate.sql
=====
```

```
-- e6_03_nursing_scores_validate.sql (run AFTER e6_03_nursing_scores_up.sql; read-only)
-- PASS = nursing_scores exists with tenant_id FK + FORCE RLS + isolation policy + indexes + score_type CHECK.
```

```
SELECT
  (SELECT count(*) FROM information_schema.tables WHERE table_name='nursing_scores') AS scores_exists,
  -- expect 1
  (SELECT count(*) FROM information_schema.columns
   WHERE table_name='nursing_scores'
     AND column_name IN ('tenant_id','patient_id','score_type','score','band','inputs_json','recorded_by'))
AS scores_cols, -- expect 7
  (SELECT relforcerowsecurity FROM pg_class WHERE relname='nursing_scores') AS scores_force_qls,
  -- expect t
  (SELECT count(*) FROM pg_policies WHERE tablename='nursing_scores' AND policyname='rls_nursing_scores_tenant_isolation') AS scores_policy, -- expect 1
  (SELECT count(*) FROM pg_constraint WHERE conname='chk_nursing_score_type') AS scores_type_check,
  -- expect 1
  (SELECT count(*) FROM pg_indexes WHERE tablename='nursing_scores' AND indexname='idx_nursing_scores_tenant_id') AS scores_idx; -- expect 1

-- ----- tenant_id is NOT NULL -----
SELECT count(*)::int AS tenant_id_not_null FROM information_schema.columns
  WHERE table_name='nursing_scores' AND column_name='tenant_id' AND is_nullable='NO';
  -- expect 1

-- ----- FK to tenants(id) -----
SELECT count(*)::int AS fk_to_tenants FROM pg_constraint
  WHERE conrelid='nursing_scores'::regclass AND confrelid='tenants'::regclass AND contype='f';
  -- expect 1

-- ----- FK patient_id -> patients(id) -----
SELECT count(*)::int AS fk_to_patients FROM pg_constraint
  WHERE conrelid='nursing_scores'::regclass AND confrelid='patients'::regclass AND contype='f';
  -- expect 1
```

```
=====
FILE: migrations\e6_04_nursing_qls_down.sql
=====
```

```
BEGIN;
```

```

ALTER TABLE nursing_io DISABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_nursing_io_tenant_isolation ON nursing_io;
DROP INDEX IF EXISTS idx_nursing_io_tenant_id;
ALTER TABLE nursing_io ALTER COLUMN tenant_id DROP DEFAULT;
ALTER TABLE nursing_io ALTER COLUMN tenant_id DROP NOT NULL;
ALTER TABLE nursing_io DROP CONSTRAINT IF EXISTS fk_nursing_io_tenant;

ALTER TABLE nursing_handover DISABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_nursing_handover_tenant_isolation ON nursing_handover;
DROP INDEX IF EXISTS idx_nursing_handover_tenant_id;
ALTER TABLE nursing_handover ALTER COLUMN tenant_id DROP DEFAULT;
ALTER TABLE nursing_handover ALTER COLUMN tenant_id DROP NOT NULL;
ALTER TABLE nursing_handover DROP CONSTRAINT IF EXISTS fk_nursing_handover_tenant;

ALTER TABLE nursing_pain_assessments DISABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_nursing_pain_assessments_tenant_isolation ON nursing_pain_assessments;
DROP INDEX IF EXISTS idx_nursing_pain_assessments_tenant_id;
ALTER TABLE nursing_pain_assessments ALTER COLUMN tenant_id DROP DEFAULT;

COMMIT;

```

```

=====
FILE: migrations\e6_04_nursing_qls_up.sql
=====

```

```

BEGIN;

```

```

-- Create dependencies if they don't exist
CREATE TABLE IF NOT EXISTS nursing_pain_assessments (
    id SERIAL PRIMARY KEY,
    patient_id INTEGER NOT NULL REFERENCES patients(id),
    patient_name TEXT DEFAULT '',
    admission_id INTEGER REFERENCES admissions(id),
    pain_scale VARCHAR(20) DEFAULT 'NRS' CHECK (pain_scale IN ('NRS','VAS','FLACC','FACES','BPS')),
    pain_score INTEGER NOT NULL CHECK (pain_score BETWEEN 0 AND 10),
    pain_location TEXT DEFAULT '',
    pain_character TEXT DEFAULT '',
    pain_radiation TEXT DEFAULT '',
    pain_onset TEXT DEFAULT '',
    pain_duration TEXT DEFAULT '',
    aggravating_factors TEXT DEFAULT '',
    relieving_factors TEXT DEFAULT '',
    current_analgesia TEXT DEFAULT '',
    pain_goal INTEGER DEFAULT 3,
    reassessment_time TIMESTAMPTZ,
    notes TEXT DEFAULT '',
    assessed_by TEXT NOT NULL DEFAULT '',
    assessed_at TIMESTAMPTZ DEFAULT NOW(),
    tenant_id INTEGER NOT NULL DEFAULT 1,
    facility_id INTEGER,
    created_at TIMESTAMPTZ DEFAULT NOW()

```

```

);
CREATE INDEX IF NOT EXISTS idx_nursing_pain_patient ON nursing_pain_assessments(patient_id, tenant_id);
CREATE INDEX IF NOT EXISTS idx_nursing_pain_admission ON nursing_pain_assessments(admission_id, tenant_id);
CREATE INDEX IF NOT EXISTS idx_nursing_pain_score ON nursing_pain_assessments(pain_score, tenant_id);

-- 1. Hardening nursing_io table
ALTER TABLE nursing_io ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE nursing_io DROP CONSTRAINT IF EXISTS fk_nursing_io_tenant;
ALTER TABLE nursing_io ADD CONSTRAINT fk_nursing_io_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
ALTER TABLE nursing_io ALTER COLUMN tenant_id SET DEFAULT (NULLIF(current_setting('app.tenant_id', true), '')::integer);

ALTER TABLE nursing_io ENABLE ROW LEVEL SECURITY;
ALTER TABLE nursing_io FORCE ROW LEVEL SECURITY;

DROP POLICY IF EXISTS rls_nursing_io_tenant_isolation ON nursing_io;
CREATE POLICY rls_nursing_io_tenant_isolation ON nursing_io
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

CREATE INDEX IF NOT EXISTS idx_nursing_io_tenant_id ON nursing_io(tenant_id);

-- 2. Hardening nursing_handover table
ALTER TABLE nursing_handover ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE nursing_handover DROP CONSTRAINT IF EXISTS fk_nursing_handover_tenant;
ALTER TABLE nursing_handover ADD CONSTRAINT fk_nursing_handover_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
ALTER TABLE nursing_handover ALTER COLUMN tenant_id SET DEFAULT (NULLIF(current_setting('app.tenant_id', true), '')::integer);

ALTER TABLE nursing_handover ENABLE ROW LEVEL SECURITY;
ALTER TABLE nursing_handover FORCE ROW LEVEL SECURITY;

DROP POLICY IF EXISTS rls_nursing_handover_tenant_isolation ON nursing_handover;
CREATE POLICY rls_nursing_handover_tenant_isolation ON nursing_handover
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

CREATE INDEX IF NOT EXISTS idx_nursing_handover_tenant_id ON nursing_handover(tenant_id);

-- 3. Hardening nursing_pain_assessments table
ALTER TABLE nursing_pain_assessments ALTER COLUMN tenant_id SET DEFAULT (NULLIF(current_setting('app.tenant_id', true), '')::integer);

ALTER TABLE nursing_pain_assessments ENABLE ROW LEVEL SECURITY;
ALTER TABLE nursing_pain_assessments FORCE ROW LEVEL SECURITY;

DROP POLICY IF EXISTS rls_nursing_pain_assessments_tenant_isolation ON nursing_pain_assessments;

```

```

CREATE POLICY rls_nursing_pain_assessments_tenant_isolation ON nursing_pain_assessments
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

CREATE INDEX IF NOT EXISTS idx_nursing_pain_assessments_tenant_id ON nursing_pain_assessments(tenant_id);

COMMIT;

```

```

=====
FILE: migrations\e6_04_nursing_rls_validate.sql
=====

```

```

SELECT
    tablename,
    rowsecurity,
    (SELECT count(*) FROM pg_policies WHERE tablename = t.tablename) as policy_count
FROM pg_tables t
JOIN pg_class c ON c.relname = t.tablename
WHERE t.tablename IN ('nursing_io', 'nursing_handover', 'nursing_pain_assessments');

```

```

=====
FILE: migrations\e6_05_clinical_pharmacy_down.sql
=====

```

```

BEGIN;

TRUNCATE TABLE drug_interactions;

COMMIT;

```

```

=====
FILE: migrations\e6_05_clinical_pharmacy_up.sql
=====

```

```

BEGIN;

-- Seed default drug-drug interactions
INSERT INTO drug_interactions (drug_a, drug_b, interaction_type, severity, description, clinical_action) VALUES
('Warfarin', 'Aspirin', 'Synergistic bleeding', 'High', 'Increased risk of bleeding due to combined anticoagulant and antiplatelet effects.', 'Avoid combination or monitor INR closely.'),
('ACE Inhibitors', 'Potassium', 'Hyperkalemia', 'High', 'Combination increases risk of severe hyperkalemia.', 'Monitor serum potassium levels regularly.'),
('Metformin', 'Contrast Dye', 'Lactic Acidosis', 'Moderate', 'Temporary renal dysfunction from contrast dye may cause metformin accumulation and lactic acidosis.', 'Hold metformin for 48 hours post-procedure.'),
('SSRIs', 'MAOIs', 'Serotonin Syndrome', 'High', 'Combination may cause life-threatening serotonin syndrome.', 'Contraindicated. Allow 14-day washout period between drugs.'),

```

```
('Statins', 'Macrolides', 'Myopathy/Rhabdomyolysis', 'Moderate', 'Macrolides inhibit CYP3A4, increasing statin levels and risk of muscle toxicity.', 'Temporarily suspend statin therapy during antibiotic course. '),
('NSAIDs', 'Anticoagulants', 'GI Bleeding', 'High', 'NSAIDs increase risk of gastrointestinal bleeding when used with anticoagulants.', 'Use alternative analgesics (e.g. acetaminophen) if possible. '),
('Digoxin', 'Amiodarone', 'Digoxin Toxicity', 'High', 'Amiodarone increases digoxin serum concentration, risking arrhythmias.', 'Reduce digoxin dose by 50% and monitor levels. '),
('Ciprofloxacin', 'Theophylline', 'Theophylline Toxicity', 'Moderate', 'Ciprofloxacin inhibits metabolism, increasing theophylline levels and toxicity risk.', 'Monitor theophylline levels and adjust dose. ')
ON CONFLICT DO NOTHING;
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e6_05_clinical_pharmacy_validate.sql
```

```
=====
```

```
SELECT count(*) FROM drug_interactions;
```

```
=====
```

```
FILE: migrations\e70_cardio_thoracic_extensions_down.sql
```

```
=====
```

```
-- Migration: e70_cardio_thoracic_extensions_down.sql
```

```
-- Description: Reverses the cardiothoracic extensions migration.
```

```
BEGIN;
```

```
DROP TABLE IF EXISTS vascular_grafts;
```

```
DROP TABLE IF EXISTS cardio_surgery_sessions;
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e70_cardio_thoracic_extensions_up.sql
```

```
=====
```

```
-- Migration: e70_cardio_thoracic_extensions_up.sql
```

```
-- Description: Adds specialized tables for Cardiothoracic and Vascular Surgery.
```

```
-- Compliance: PDPL / RLS Enabled
```

```
BEGIN;
```

```
-- 1. Cardiothoracic Surgery Sessions
```

```
CREATE TABLE IF NOT EXISTS cardio_surgery_sessions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    surgeon_id UUID NOT NULL,
```

```

procedure_type VARCHAR(255), -- e.g., CABG, Valve Replacement
pump_time_minutes INTEGER,
cross_clamp_time_minutes INTEGER,
bypass_flow_rate FLOAT,
intraop_map_avg INTEGER,
outcome_status VARCHAR(100),
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_tenant_cts FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
CONSTRAINT fk_patient_cts FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

-- 2. Vascular Grafts Registry
CREATE TABLE IF NOT EXISTS vascular_grafts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    session_id UUID NOT NULL,
    graft_type VARCHAR(100), -- e.g., PTFE, Dacron, Autologous
    la_diameter_mm FLOAT,
    location_segment VARCHAR(255),
    patency_status BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tenant_vg FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
    CONSTRAINT fk_session_vg FOREIGN KEY (session_id) REFERENCES cardio_surgery_sessions(id) ON DELETE CASCADE
);

-- Enable Row Level Security (RLS)
ALTER TABLE cardio_surgery_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE vascular_grafts ENABLE ROW LEVEL SECURITY;

-- Create RLS Policies (Golden Access Rule: Tenant Isolation)
CREATE POLICY tenant_isolation_policy ON cardio_surgery_sessions
    USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY tenant_isolation_policy ON vascular_grafts
    USING (tenant_id = current_setting('app.current_tenant')::UUID);

COMMIT;

=====
FILE: migrations\e70_ophthalmology_down.sql
=====

-- Migration: e70_ophthalmology_down.sql
-- Purpose: Reverse changes from e70_ophthalmology_up.sql

BEGIN;

DROP TABLE IF EXISTS glaucoma_metrics;
DROP TABLE IF EXISTS iol_registry;
DROP TABLE IF EXISTS ophthalmic_surgical_logs;

COMMIT;

```

```
=====
FILE: migrations\e70_ophthalmology_up.sql
=====
```

```
-- Migration: e70_ophthalmology_up.sql
-- Purpose: Implement specialized tables for Ophthalmology
```

```
BEGIN;
```

```
-- 1. Ophthalmic Surgical Logs
```

```
CREATE TABLE IF NOT EXISTS ophthalmic_surgical_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    surgeon_id UUID NOT NULL,
    operation_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    procedure_type VARCHAR(100), -- e.g., Phacoemulsification, Vitrectomy, Keratoplasty
    eye_side VARCHAR(10), -- Left, Right, Bilateral
    duration_minutes INTEGER,
    complications TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 2. IOL (Intraocular Lens) Registry
```

```
CREATE TABLE IF NOT EXISTS iol_registry (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    procedure_id UUID REFERENCES ophthalmic_surgical_logs(id),
    lens_type VARCHAR(50), -- Monofocal, Multifocal, Toric
    lens_power FLOAT, -- Diopters
    lens_brand VARCHAR(100),
    lens_serial_number VARCHAR(100),
    calculated_power FLOAT,
    actual_power FLOAT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 3. Glaucoma & IOP Metrics
```

```
CREATE TABLE IF NOT EXISTS glaucoma_metrics (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    log_time TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    iop_value FLOAT, -- mmHg
    drainage_device VARCHAR(100), -- e.g., Ahmed Valve, Baervat
    cup_to_disc_ratio FLOAT,
    visual_field_loss_percent FLOAT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Enable RLS
ALTER TABLE ophthalmic_surgical_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE iol_registry ENABLE ROW LEVEL SECURITY;
ALTER TABLE glaucoma_metrics ENABLE ROW LEVEL SECURITY;

-- RLS Policies
CREATE POLICY ophthalmic_surgical_logs_tenant_policy ON ophthalmic_surgical_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY iol_registry_tenant_policy ON iol_registry
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY glaucoma_metrics_tenant_policy ON glaucoma_metrics
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

COMMIT;
```

```
=====
FILE: migrations\e71_ent_down.sql
=====
```

```
-- Migration: e71_ent_down.sql
-- Purpose: Reverse changes from e71_ent_up.sql
```

```
BEGIN;

DROP TABLE IF EXISTS cochlear_implant_registry;
DROP TABLE IF EXISTS audiometry_metrics;
DROP TABLE IF EXISTS ent_surgical_logs;

COMMIT;
```

```
=====
FILE: migrations\e71_ent_up.sql
=====
```

```
-- Migration: e71_ent_up.sql
-- Purpose: Implement specialized tables for ENT (Otolaryngology)
```

```
BEGIN;

-- 1. ENT Surgical Logs
CREATE TABLE IF NOT EXISTS ent_surgical_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    surgeon_id UUID NOT NULL,
    operation_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    procedure_type VARCHAR(100), -- e.g., FESS, Tonsillectomy, Myringotomy
```

```

side VARCHAR(10), -- Left, Right, Bilateral
duration_minutes INTEGER,
complications TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. Audiometry & Tympanometry Metrics
CREATE TABLE IF NOT EXISTS audiometry_metrics (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    log_time TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    frequency_hz INTEGER[], -- Array of frequencies
    threshold_db INTEGER[], -- Array of thresholds
    tympanometry_type VARCHAR(20), -- A, B, C, As, Ad
    air_bone_gap FLOAT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Cochlear Implant Registry
CREATE TABLE IF NOT EXISTS cochlear_implant_registry (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    procedure_id UUID REFERENCES ent_surgical_logs(id),
    implant_brand VARCHAR(100),
    implant_model VARCHAR(100),
    serial_number VARCHAR(100),
    mapping_parameters JSONB, -- Store complex mapping settings
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE ent_surgical_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE audiometry_metrics ENABLE ROW LEVEL SECURITY;
ALTER TABLE cochlear_implant_registry ENABLE ROW LEVEL SECURITY;

-- RLS Policies
CREATE POLICY ent_surgical_logs_tenant_policy ON ent_surgical_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY audiometry_metrics_tenant_policy ON audiometry_metrics
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY cochlear_implant_registry_tenant_policy ON cochlear_implant_registry
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

COMMIT;

=====
FILE: migrations\e71_neuro_extensions_down.sql

```

=====

-- Migration: e71\_neuro\_extensions\_down.sql  
-- Description: Reverses the neurosurgery extensions migration.

BEGIN;

DROP TABLE IF EXISTS spine\_fusion\_records;  
DROP TABLE IF EXISTS neuro\_surgery\_sessions;

COMMIT;

=====

FILE: migrations\e71\_neuro\_extensions\_up.sql

=====

-- Migration: e71\_neuro\_extensions\_up.sql  
-- Description: Adds specialized tables for Neurosurgery and Spine Surgery.  
-- Compliance: PDPL / RLS Enabled

BEGIN;

-- 1. Neurosurgery Sessions

```
CREATE TABLE IF NOT EXISTS neuro_surgery_sessions (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    patient_id UUID NOT NULL,  
    surgeon_id UUID NOT NULL,  
    procedure_type VARCHAR(255), -- e.g., Craniotomy, Aneurysm Clipping  
    icp_max_mmhg FLOAT,  
    navigation_system_used VARCHAR(100),  
    anesthesia_type VARCHAR(100),  
    outcome_gcs_postop INTEGER,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_tenant_neuro FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,  
    CONSTRAINT fk_patient_neuro FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE  
);
```

-- 2. Spine Fusion Records

```
CREATE TABLE IF NOT EXISTS spine_fusion_records (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    session_id UUID NOT NULL,  
    levels_fused VARCHAR(100), -- e.g., L4-L5, L5-S1  
    hardware_type VARCHAR(100),  
    stability_score INTEGER,  
    postop_alignment_angle FLOAT,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_tenant_spine FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,  
    CONSTRAINT fk_session_spine FOREIGN KEY (session_id) REFERENCES neuro_surgery_sessions(id) ON DELETE CASCADE  
DE  
);
```

```
-- Enable Row Level Security (RLS)
ALTER TABLE neuro_surgery_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE spine_fusion_records ENABLE ROW LEVEL SECURITY;

-- Create RLS Policies (Golden Access Rule: Tenant Isolation)
CREATE POLICY tenant_isolation_policy ON neuro_surgery_sessions
    USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY tenant_isolation_policy ON spine_fusion_records
    USING (tenant_id = current_setting('app.current_tenant')::UUID);

COMMIT;
```

```
=====
FILE: migrations\e72_ortho_extensions_down.sql
=====
```

```
-- Migration: e72_ortho_extensions_down.sql
-- Description: Reverses the orthopedics extensions migration.
```

```
BEGIN;
```

```
DROP TABLE IF EXISTS ortho_trauma_logs;
DROP TABLE IF EXISTS ortho_joint_replacements;
```

```
COMMIT;
```

```
=====
FILE: migrations\e72_ortho_extensions_up.sql
=====
```

```
-- Migration: e72_ortho_extensions_up.sql
-- Description: Adds specialized tables for Orthopedics and Joint Replacement.
-- Compliance: PDPL / RLS Enabled
```

```
BEGIN;
```

```
-- 1. Joint Replacement Records
CREATE TABLE IF NOT EXISTS ortho_joint_replacements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    joint_type VARCHAR(100), -- e.g., Total Hip, Partial Knee
    implant_model VARCHAR(255),
    implant_serial_number VARCHAR(100),
    alignment_angle FLOAT,
    rom_postop_degrees INTEGER,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tenant_ortho FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
```

```

        CONSTRAINT fk_patient_ortho FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
    );

-- 2. Trauma & Fracture Logs
CREATE TABLE IF NOT EXISTS ortho_trauma_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    fracture_type VARCHAR(255),
    fixation_method VARCHAR(100), -- e.g., Intramedullary Nail, Plate & Screw
    hardware_used TEXT,
    reduction_quality VARCHAR(50),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tenant_trauma FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
    CONSTRAINT fk_patient_trauma FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

-- Enable Row Level Security (RLS)
ALTER TABLE ortho_joint_replacements ENABLE ROW LEVEL SECURITY;
ALTER TABLE ortho_trauma_logs ENABLE ROW LEVEL SECURITY;

-- Create RLS Policies (Golden Access Rule: Tenant Isolation)
CREATE POLICY tenant_isolation_policy ON ortho_joint_replacements
    USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY tenant_isolation_policy ON ortho_trauma_logs
    USING (tenant_id = current_setting('app.current_tenant')::UUID);

COMMIT;

=====
FILE: migrations\e72_urology_down.sql
=====

-- Migration: e72_urology_down.sql
-- Purpose: Reverse changes from e72_urology_up.sql

BEGIN;

DROP TABLE IF EXISTS urology_oncology_metrics;
DROP TABLE IF EXISTS urology_stone_registry;
DROP TABLE IF EXISTS urology_surgical_logs;

COMMIT;

=====
FILE: migrations\e72_urology_up.sql
=====

-- Migration: e72_urology_up.sql

```

```

-- Purpose: Implement specialized tables for Urology

BEGIN;

-- 1. Urology Surgical Logs
CREATE TABLE IF NOT EXISTS urology_surgical_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    surgeon_id UUID NOT NULL,
    operation_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    procedure_type VARCHAR(100), -- e.g., Robotic Prostatectomy, TURBT, PCNL
    approach VARCHAR(100), -- e.g., Laparoscopic, Endoscopic, Open
    duration_minutes INTEGER,
    blood_loss_ml INTEGER,
    complications TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. Urology Stone Registry
CREATE TABLE IF NOT EXISTS urology_stone_registry (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    procedure_id UUID REFERENCES urology_surgical_logs(id),
    stone_location VARCHAR(100), -- e.g., Upper Pole Left Kidney
    stone_size_mm FLOAT,
    stone_composition VARCHAR(50), -- e.g., Calcium Oxalate, Uric Acid
    fragmentation_success BOOLEAN,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Urology Oncology Metrics
CREATE TABLE IF NOT EXISTS urology_oncology_metrics (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    log_time TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    psa_level FLOAT, -- ng/mL
    gleason_score VARCHAR(10), -- e.g., 3+4=7
    tumor_grade VARCHAR(20),
    treatment_response VARCHAR(50),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE urology_surgical_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE urology_stone_registry ENABLE ROW LEVEL SECURITY;
ALTER TABLE urology_oncology_metrics ENABLE ROW LEVEL SECURITY;

-- RLS Policies
CREATE POLICY urology_surgical_logs_tenant_policy ON urology_surgical_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

```

```
CREATE POLICY urology_stone_registry_tenant_policy ON urology_stone_registry
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);
```

```
CREATE POLICY urology_oncology_metrics_tenant_policy ON urology_oncology_metrics
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);
```

```
COMMIT;
```

```
=====
FILE: migrations\e73_eye_extensions_down.sql
=====
```

```
-- Migration: e73_eye_extensions_down.sql
BEGIN;
DROP TABLE IF EXISTS eye_biometry_logs;
DROP TABLE IF EXISTS eye_surgery_sessions;
COMMIT;
```

```
=====
FILE: migrations\e73_eye_extensions_up.sql
=====
```

```
-- Migration: e73_eye_extensions_up.sql
-- Description: Adds specialized tables for Ophthalmology Surgery.
-- Compliance: PDPL / RLS Enabled
```

```
BEGIN;
```

```
CREATE TABLE IF NOT EXISTS eye_surgery_sessions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    surgeon_id UUID NOT NULL,
    eye_side VARCHAR(10), -- L, R, Both
    procedure_type VARCHAR(255),
    iol_model VARCHAR(100),
    iol_power FLOAT,
    axial_length FLOAT,
    outcome_visual_acuity VARCHAR(50),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tenant_eye FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
    CONSTRAINT fk_patient_eye FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);
```

```
CREATE TABLE IF NOT EXISTS eye_biometry_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id UUID NOT NULL,
    tenant_id UUID NOT NULL,
    keratometry_reading FLOAT,
```

```

predicted_iol_power FLOAT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_tenant_bio FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
CONSTRAINT fk_session_bio FOREIGN KEY (session_id) REFERENCES eye_surgery_sessions(id) ON DELETE CASCADE
);

ALTER TABLE eye_surgery_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE eye_biometry_logs ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON eye_surgery_sessions USING (tenant_id = current_setting('app.current_tenant')::UUID);
CREATE POLICY tenant_isolation_policy ON eye_biometry_logs USING (tenant_id = current_setting('app.current_tenant')::UUID);

COMMIT;

=====
FILE: migrations\e73_plastic_burns_down.sql
=====

-- Migration: e73_plastic_burns_down.sql
-- Purpose: Reverse changes from e73_plastic_burns_up.sql

BEGIN;

DROP TABLE IF EXISTS flap_monitoring_metrics;
DROP TABLE IF EXISTS burn_resuscitation_logs;
DROP TABLE IF EXISTS plastic_burns_surgical_logs;

COMMIT;

=====
FILE: migrations\e73_plastic_burns_up.sql
=====

-- Migration: e73_plastic_burns_up.sql
-- Purpose: Implement specialized tables for Plastic & Burns Surgery

BEGIN;

-- 1. Plastic & Burns Surgical Logs
CREATE TABLE IF NOT EXISTS plastic_burns_surgical_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    surgeon_id UUID NOT NULL,
    operation_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    procedure_type VARCHAR(100), -- e.g., Free Flap Transfer, Skin Graft, Rhinoplasty
    approach VARCHAR(100),
    duration_minutes INTEGER,

```

```

        blood_loss_ml INTEGER,
        complications TEXT,
        created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
    );

-- 2. Burn Resuscitation & TBSA Logs
CREATE TABLE IF NOT EXISTS burn_resuscitation_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    log_time TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    tbsa_percent FLOAT, -- Total Body Surface Area
    fluid_volume_ml FLOAT, -- Total fluid administered
    fluid_type VARCHAR(50), -- e.g., Ringer's Lactate
    urine_output_ml_hr FLOAT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Flap & Graft Monitoring Metrics
CREATE TABLE IF NOT EXISTS flap_monitoring_metrics (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    procedure_id UUID REFERENCES plastic_burns_surgical_logs(id),
    log_time TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    perfusion_status VARCHAR(50), -- e.g., Brisk, Sluggish, Absent
    capillary_refill_sec FLOAT,
    color_status VARCHAR(50), -- e.g., Pink, Pale, Cyanotic
    temperature_c FLOAT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE plastic_burns_surgical_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE burn_resuscitation_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE flap_monitoring_metrics ENABLE ROW LEVEL SECURITY;

-- RLS Policies
CREATE POLICY plastic_burns_surgical_logs_tenant_policy ON plastic_burns_surgical_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY burn_resuscitation_logs_tenant_policy ON burn_resuscitation_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY flap_monitoring_metrics_tenant_policy ON flap_monitoring_metrics
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

COMMIT;

=====
FILE: migrations\e74_ent_extensions_down.sql

```

=====

```
-- Migration: e74_ent_extensions_down.sql
BEGIN;
DROP TABLE IF EXISTS ent_audiometry_logs;
DROP TABLE IF EXISTS ent_surgery_sessions;
COMMIT;
```

=====

FILE: migrations\e74\_ent\_extensions\_up.sql

=====

```
-- Migration: e74_ent_extensions_up.sql
-- Description: Adds specialized tables for ENT Surgery.
-- Compliance: PDPL / RLS Enabled
```

```
BEGIN;
```

```
CREATE TABLE IF NOT EXISTS ent_surgery_sessions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    surgeon_id UUID NOT NULL,
    side VARCHAR(20), -- Left, Right, Bilateral
    procedure_type VARCHAR(255),
    implant_model VARCHAR(100),
    outcome_hearing_gain_db FLOAT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tenant_ent FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
    CONSTRAINT fk_patient_ent FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);
```

```
CREATE TABLE IF NOT EXISTS ent_audiometry_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id UUID NOT NULL,
    tenant_id UUID NOT NULL,
    frequency_hz INTEGER,
    decibel_level FLOAT,
    test_type VARCHAR(50), -- Air, Bone
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tenant_aud FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
    CONSTRAINT fk_session_aud FOREIGN KEY (session_id) REFERENCES ent_surgery_sessions(id) ON DELETE CASCADE
);
```

```
ALTER TABLE ent_surgery_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE ent_audiometry_logs ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY tenant_isolation_policy ON ent_surgery_sessions USING (tenant_id = current_setting('app.current_tenant')::UUID);
CREATE POLICY tenant_isolation_policy ON ent_audiometry_logs USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e74_obgyn_ivf_mfm_down.sql
```

```
=====
```

```
-- Migration: e74_obgyn_ivf_mfm_down.sql
```

```
-- Purpose: Reverse changes from e74_obgyn_ivf_mfm_up.sql
```

```
BEGIN;
```

```
DROP TABLE IF EXISTS gyn_oncology_registry;
```

```
DROP TABLE IF EXISTS obgyn_delivery_logs;
```

```
DROP TABLE IF EXISTS maternal_fetal_metrics;
```

```
DROP TABLE IF EXISTS obgyn_ivf_lab_logs;
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e74_obgyn_ivf_mfm_up.sql
```

```
=====
```

```
-- Migration: e74_obgyn_ivf_mfm_up.sql
```

```
-- Purpose: Implement specialized tables for OBGYN (IVF, MFM, and Delivery)
```

```
BEGIN;
```

```
-- 1. IVF/ICSI Lab Logs (Embryo Tracking)
```

```
CREATE TABLE IF NOT EXISTS obgyn_ivf_lab_logs (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    patient_id UUID NOT NULL,  
    cycle_id VARCHAR(100), -- Unique identifier for the IVF cycle  
    log_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    oocyte_count INTEGER,  
    fertilization_rate FLOAT,  
    embryo_grade VARCHAR(20), -- e.g., 4AA, 3BB (Gardner Scale)  
    embryo_stage VARCHAR(50), -- e.g., Cleavage, Blastocyst  
    transfer_date TIMESTAMP WITH TIME ZONE,  
    cryopreservation_count INTEGER,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- 2. Maternal-Fetal Medicine (MFM) Metrics
```

```
CREATE TABLE IF NOT EXISTS maternal_fetal_metrics (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    patient_id UUID NOT NULL,  
    gestational_age_weeks FLOAT,  
    bpd_mm FLOAT, -- Biparietal Diameter
```

```

    hc_mm FLOAT, -- Head Circumference
    ac_mm FLOAT, -- Abdominal Circumference
    fl_mm FLOAT, -- Femur Length
    estimated_fetal_weight_g FLOAT,
    growth_percentile FLOAT,
    doppler_velocity_cm_s FLOAT, -- Umbilical Artery
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Delivery & Neonatal Transition Logs
CREATE TABLE IF NOT EXISTS obgyn_delivery_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    delivery_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    delivery_mode VARCHAR(50), -- Vaginal, C-Section, Vacuum, Forceps
    delivery_duration_min INTEGER,
    apgar_1min INTEGER,
    apgar_5min INTEGER,
    birth_weight_g FLOAT,
    maternal_blood_loss_ml INTEGER,
    pph_status BOOLEAN DEFAULT FALSE, -- Postpartum Hemorrhage
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 4. Gynecology Oncology & Endometriosis Registry
CREATE TABLE IF NOT EXISTS gyn_oncology_registry (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    diagnosis VARCHAR(100), -- e.g., Endometriosis, Ovarian Cancer
    stage VARCHAR(20), -- FIGO Stage
    grade VARCHAR(20),
    treatment_plan TEXT,
    surgery_type VARCHAR(100), -- e.g., Total Hysterectomy
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE obgyn_ivf_lab_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE maternal_fetal_metrics ENABLE ROW LEVEL SECURITY;
ALTER TABLE obgyn_delivery_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE gyn_oncology_registry ENABLE ROW LEVEL SECURITY;

-- RLS Policies
CREATE POLICY obgyn_ivf_lab_logs_tenant_policy ON obgyn_ivf_lab_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY maternal_fetal_metrics_tenant_policy ON maternal_fetal_metrics
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY obgyn_delivery_logs_tenant_policy ON obgyn_delivery_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

```

```
CREATE POLICY gyn_oncology_registry_tenant_policy ON gyn_oncology_registry
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e75_pediatrics_nicu_down.sql
```

```
=====
```

```
-- Migration: e75_pediatrics_nicu_down.sql
-- Purpose: Reverse changes from e75_pediatrics_nicu_up.sql
```

```
BEGIN;
```

```
DROP TABLE IF EXISTS neonatal_transition_logs;
DROP TABLE IF EXISTS peds_milestone_tracking;
DROP TABLE IF EXISTS nicu_ventilation_logs;
DROP TABLE IF EXISTS peds_growth_logs;
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e75_pediatrics_nicu_up.sql
```

```
=====
```

```
-- Migration: e75_pediatrics_nicu_up.sql
-- Purpose: Implement specialized tables for Pediatrics & NICU (Level III/IV)
```

```
BEGIN;
```

```
-- 1. Pediatric Growth & Biometry Logs
```

```
CREATE TABLE IF NOT EXISTS peds_growth_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    log_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    weight_kg FLOAT,
    length_cm FLOAT,
    head_circ_cm FLOAT,
    bmi FLOAT,
    weight_zscore FLOAT,
    length_zscore FLOAT,
    head_zscore FLOAT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 2. NICU Ventilation & Respiratory Logs
```

```
CREATE TABLE IF NOT EXISTS nicu_ventilation_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
```

```

patient_id UUID NOT NULL,
log_time TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
vent_mode VARCHAR(50), -- e.g., HFOV, Conventional, CPAP
fio2_percent FLOAT,
peep_cmh2o FLOAT,
mean_airway_pressure_cmh2o FLOAT,
tidal_volume_ml FLOAT,
respiratory_rate_bpm INTEGER,
spo2_percent FLOAT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Pediatric Developmental Milestone Tracking
CREATE TABLE IF NOT EXISTS peds_milestone_tracking (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    milestone_category VARCHAR(50), -- e.g., Motor, Language, Social
    milestone_name VARCHAR(100), -- e.g., Sits without support, First words
    status VARCHAR(20), -- Achieved, Delayed, Not Yet
    achievement_date DATE,
    notes TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 4. Neonatal Transition & Stabilization Logs
CREATE TABLE IF NOT EXISTS neonatal_transition_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    birth_time TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    apgar_1min INTEGER,
    apgar_5min INTEGER,
    apgar_10min INTEGER,
    initial_stabilization_notes TEXT,
    surfactant_administered BOOLEAN DEFAULT FALSE,
    surfactant_type VARCHAR(100),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE peds_growth_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE nicu_ventilation_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE peds_milestone_tracking ENABLE ROW LEVEL SECURITY;
ALTER TABLE neonatal_transition_logs ENABLE ROW LEVEL SECURITY;

-- RLS Policies
CREATE POLICY peds_growth_logs_tenant_policy ON peds_growth_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY nicu_ventilation_logs_tenant_policy ON nicu_ventilation_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY peds_milestone_tracking_tenant_policy ON peds_milestone_tracking

```

```

        USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY neonatal_transition_logs_tenant_policy ON neonatal_transition_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

COMMIT;

=====
FILE: migrations\e75_urology_extensions_down.sql
=====

-- Migration: e75_urology_extensions_down.sql
BEGIN;
DROP TABLE IF EXISTS urology_stone_logs;
DROP TABLE IF EXISTS urology_surgery_sessions;
COMMIT;

=====
FILE: migrations\e75_urology_extensions_up.sql
=====

-- Migration: e75_urology_extensions_up.sql
-- Description: Adds specialized tables for Urology Surgery.
-- Compliance: PDPL / RLS Enabled

BEGIN;

CREATE TABLE IF NOT EXISTS urology_surgery_sessions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    surgeon_id UUID NOT NULL,
    procedure_type VARCHAR(255),
    stent_used BOOLEAN,
    stent_type VARCHAR(100),
    duration_min INTEGER,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tenant_uro FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
    CONSTRAINT fk_patient_uro FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS urology_stone_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id UUID NOT NULL,
    tenant_id UUID NOT NULL,
    stone_size_mm FLOAT,
    stone_location VARCHAR(255),
    fragmentation_rate_pct FLOAT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tenant_stone FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,

```

```
CONSTRAINT fk_session_stone FOREIGN KEY (session_id) REFERENCES urology_surgery_sessions(id) ON DELETE CASCADE
);
```

```
ALTER TABLE urology_surgery_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE urology_stone_logs ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY tenant_isolation_policy ON urology_surgery_sessions USING (tenant_id = current_setting('app.current_tenant')::UUID);
CREATE POLICY tenant_isolation_policy ON urology_stone_logs USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

```
COMMIT;
```

```
=====
FILE: migrations\e76_peds_subspecialties_down.sql
=====
```

```
-- Migration: e76_peds_subspecialties_down.sql
-- Purpose: Reverse changes from e76_peds_subspecialties_up.sql
```

```
BEGIN;
```

```
DROP TABLE IF EXISTS peds_neuro_logs;
DROP TABLE IF EXISTS peds_nephro_logs;
DROP TABLE IF EXISTS peds_cardio_logs;
```

```
COMMIT;
```

```
=====
FILE: migrations\e76_peds_subspecialties_up.sql
=====
```

```
-- Migration: e76_peds_subspecialties_up.sql
-- Purpose: Implement specialized tables for Pediatric Subspecialties (Cardio, Nephro, Neuro)
```

```
BEGIN;
```

```
-- 1. Pediatric Cardiology Logs
CREATE TABLE IF NOT EXISTS peds_cardio_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    log_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    chd_diagnosis VARCHAR(100), -- Congenital Heart Disease
    aortic_zscore FLOAT,
    pulmonary_zscore FLOAT,
    ef_percent FLOAT,
    echo_findings TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
```

```

);

-- 2. Pediatric Nephrology Logs
CREATE TABLE IF NOT EXISTS peds_nephro_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    log_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    gfr_calculated FLOAT, -- Schwartz formula
    proteinuria_grade VARCHAR(20),
    dialysis_type VARCHAR(50), -- Peritoneal, Hemodialysis
    dialysis_frequency_per_week INTEGER,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Pediatric Neurology Logs
CREATE TABLE IF NOT EXISTS peds_neuro_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    log_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    seizure_type VARCHAR(100),
    seizure_frequency_per_day INTEGER,
    bayley_iii_score FLOAT, -- Developmental scale
    mri_findings TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE peds_cardio_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE peds_nephro_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE peds_neuro_logs ENABLE ROW LEVEL SECURITY;

-- RLS Policies
CREATE POLICY peds_cardio_logs_tenant_policy ON peds_cardio_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY peds_nephro_logs_tenant_policy ON peds_nephro_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY peds_neuro_logs_tenant_policy ON peds_neuro_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

COMMIT;

=====
FILE: migrations\e76_plastic_extensions_down.sql
=====

-- Migration: e76_plastic_extensions_down.sql
BEGIN;
DROP TABLE IF EXISTS aesthetic_logs;

```

```
DROP TABLE IF EXISTS plastic_surgery_sessions;
COMMIT;
```

```
=====
FILE: migrations\e76_plastic_extensions_up.sql
=====
```

```
-- Migration: e76_plastic_extensions_up.sql
-- Description: Adds specialized tables for Plastic & Reconstructive Surgery.
-- Compliance: PDPL / RLS Enabled
```

```
BEGIN;
```

```
CREATE TABLE IF NOT EXISTS plastic_surgery_sessions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    surgeon_id UUID NOT NULL,
    procedure_type VARCHAR(255),
    flap_used BOOLEAN,
    flap_type VARCHAR(100),
    perfusion_index FLOAT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tenant_plas FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
    CONSTRAINT fk_patient_plas FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);
```

```
CREATE TABLE IF NOT EXISTS aesthetic_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id UUID NOT NULL,
    tenant_id UUID NOT NULL,
    target_area VARCHAR(255),
    material_used TEXT,
    session_count INTEGER,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tenant_aes FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
    CONSTRAINT fk_session_aes FOREIGN KEY (session_id) REFERENCES plastic_surgery_sessions(id) ON DELETE CASCADE
DE
);
```

```
ALTER TABLE plastic_surgery_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE aesthetic_logs ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY tenant_isolation_policy ON plastic_surgery_sessions USING (tenant_id = current_setting('app.current_tenant')::UUID);
CREATE POLICY tenant_isolation_policy ON aesthetic_logs USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

```
COMMIT;
```

```
=====
FILE: migrations\e77_diagnostics_advanced_down.sql
=====
```

```
-- Migration: e77_diagnostics_advanced_down.sql
-- Purpose: Reverse changes from e77_diagnostics_advanced_up.sql
```

```
BEGIN;
```

```
DROP TABLE IF EXISTS pathology_digital_logs;
DROP TABLE IF EXISTS nuclear_med_logs;
DROP TABLE IF EXISTS radiology_advanced_metrics;
DROP TABLE IF EXISTS diag_molecular_logs;
```

```
COMMIT;
```

```
=====
FILE: migrations\e77_diagnostics_advanced_up.sql
=====
```

```
-- Migration: e77_diagnostics_advanced_up.sql
-- Purpose: Implement specialized tables for Advanced Diagnostics (Molecular, Radiology, Nuclear, Pathology)
```

```
BEGIN;
```

```
-- 1. Molecular Diagnostics & NGS Logs
```

```
CREATE TABLE IF NOT EXISTS diag_molecular_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    log_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    test_type VARCHAR(100), -- e.g., NGS, Liquid Biopsy, PCR
    gene_mutation VARCHAR(100), -- e.g., EGFR L858R, BRAF V600E
    variant_allele_frequency FLOAT,
    molecular_typing TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 2. Advanced Radiology Metrics (Volumetric & Kinetic)
```

```
CREATE TABLE IF NOT EXISTS radiology_advanced_metrics (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    procedure_id UUID, -- Link to base radiology table
    lesion_volume_mm3 FLOAT,
    kinetic_curve_type VARCHAR(50), -- e.g., Wash-in, Wash-out
    contrast_enhancement_rate FLOAT,
    ai_detection_confidence FLOAT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 3. Nuclear Medicine & Tracer Logs
```

```

CREATE TABLE IF NOT EXISTS nuclear_med_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    tracer_used VARCHAR(100), -- e.g., 18F-FDG, 99mTc-Sestamibi
    suv_max FLOAT, -- Standardized Uptake Value
    uptake_region VARCHAR(100),
    quantification_value FLOAT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 4. Digital Pathology & IHC Logs
CREATE TABLE IF NOT EXISTS pathology_digital_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    slide_id VARCHAR(100),
    ihc_marker VARCHAR(100), -- e.g., HER2, PD-L1
    ihc_score INTEGER, -- 0, 1+, 2+, 3+
    molecular_typing TEXT,
    digital_slide_url TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE diag_molecular_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE radiology_advanced_metrics ENABLE ROW LEVEL SECURITY;
ALTER TABLE nuclear_med_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE pathology_digital_logs ENABLE ROW LEVEL SECURITY;

-- RLS Policies
CREATE POLICY diag_molecular_logs_tenant_policy ON diag_molecular_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY radiology_advanced_metrics_tenant_policy ON radiology_advanced_metrics
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY nuclear_med_logs_tenant_policy ON nuclear_med_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY pathology_digital_logs_tenant_policy ON pathology_digital_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

COMMIT;

=====
FILE: migrations\e77_obgyn_peds_extensions_down.sql
=====

-- Migration: e77_obgyn_peds_extensions_down.sql
BEGIN;
DROP TABLE IF EXISTS nicu_monitoring;

```

```
DROP TABLE IF EXISTS ivf_cycles;
DROP TABLE IF EXISTS mfm_scans;
COMMIT;
```

```
=====
FILE: migrations\e77_obgyn_peds_extensions_up.sql
=====
```

```
-- Migration: e77_obgyn_peds_extensions_up.sql
-- Description: Adds specialized tables for Maternal-Fetal Medicine, IVF, and NICU.
-- Compliance: PDPL / RLS Enabled
```

```
BEGIN;
```

```
-- 1. Maternal-Fetal Medicine (MFM) Scans
```

```
CREATE TABLE IF NOT EXISTS mfm_scans (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    scan_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    fetal_weight_est FLOAT,
    crown_rump_length FLOAT,
    anomaly_detected BOOLEAN DEFAULT FALSE,
    anomaly_description TEXT,
    scan_type VARCHAR(100), -- e.g., 4D Ultrasound, Doppler
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tenant_mfm FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
    CONSTRAINT fk_patient_mfm FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);
```

```
-- 2. IVF Cycle Tracking
```

```
CREATE TABLE IF NOT EXISTS ivf_cycles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    cycle_start_date DATE,
    egg_count INTEGER,
    fertilization_rate_pct FLOAT,
    embryo_grade VARCHAR(50),
    transfer_date DATE,
    outcome VARCHAR(100), -- e.g., Success, Failed
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tenant_ivf FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
    CONSTRAINT fk_patient_ivf FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);
```

```
-- 3. NICU Monitoring
```

```
CREATE TABLE IF NOT EXISTS nicu_monitoring (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    record_time TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
```

```

ventilator_mode VARCHAR(100),
oxygen_saturation_pct FLOAT,
temperature_c FLOAT,
heart_rate_bpm INTEGER,
nutrition_type VARCHAR(100),
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_tenant_nicu FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
CONSTRAINT fk_patient_nicu FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

```

```

-- Enable Row Level Security (RLS)

```

```

ALTER TABLE mfm_scans ENABLE ROW LEVEL SECURITY;

```

```

ALTER TABLE ivf_cycles ENABLE ROW LEVEL SECURITY;

```

```

ALTER TABLE nicu_monitoring ENABLE ROW LEVEL SECURITY;

```

```

-- Create RLS Policies (Golden Access Rule: Tenant Isolation)

```

```

CREATE POLICY tenant_isolation_policy ON mfm_scans
    USING (tenant_id = current_setting('app.current_tenant')::UUID);

```

```

CREATE POLICY tenant_isolation_policy ON ivf_cycles
    USING (tenant_id = current_setting('app.current_tenant')::UUID);

```

```

CREATE POLICY tenant_isolation_policy ON nicu_monitoring
    USING (tenant_id = current_setting('app.current_tenant')::UUID);

```

```

COMMIT;

```

```

=====
FILE: migrations\e77_rad_extensions_up.sql
=====

```

```

-- Migration: e77_rad_extensions_up.sql
-- Description: Adds specialized tables for Advanced Radiology and Nuclear Medicine.
-- Compliance: PDPL / RLS Enabled

```

```

BEGIN;

```

```

CREATE TABLE IF NOT EXISTS radiology_scans (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    modality VARCHAR(50), -- CT, MRI, PET, XRAY
    study_uid VARCHAR(255),
    image_path TEXT, -- Path to phi_vault/
    contrast_used BOOLEAN DEFAULT FALSE,
    radiation_dose_msv FLOAT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tenant_rad FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
    CONSTRAINT fk_patient_rad FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

```

```

CREATE TABLE IF NOT la_radiology_reports (

```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
scan_id UUID NOT NULL,
tenant_id UUID NOT NULL,
findings TEXT,
impression TEXT,
status VARCHAR(50), -- Draft, Final, Critical
radiologist_id UUID NOT NULL,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_tenant_rep FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
CONSTRAINT fk_scan_rep FOREIGN KEY (scan_id) REFERENCES radiology_scans(id) ON DELETE CASCADE
);

ALTER TABLE radiology_scans ENABLE ROW LEVEL SECURITY;
ALTER TABLE la_radiology_reports ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON radiology_scans USING (tenant_id = current_setting('app.current_tenant')::UUID);
CREATE POLICY tenant_isolation_policy ON la_radiology_reports USING (tenant_id = current_setting('app.current_tenant')::UUID);

COMMIT;

=====
FILE: migrations\e78_critical_care_down.sql
=====

-- Migration: e78_critical_care_down.sql
-- Purpose: Reverse changes from e78_critical_care_up.sql

BEGIN;

DROP TABLE IF EXISTS shock_titration_logs;
DROP TABLE IF EXISTS sepsis_bundle_tracking;
DROP TABLE IF EXISTS crit_care_ventilation_logs;
DROP TABLE IF EXISTS crit_care_hemodynamics;

COMMIT;

=====
FILE: migrations\e78_critical_care_up.sql
=====

-- Migration: e78_critical_care_up.sql
-- Purpose: Implement specialized tables for Critical Care & Emergency (Wave 5)

BEGIN;

-- 1. Hemodynamic Monitoring Logs
CREATE TABLE IF NOT EXISTS crit_care_hemodynamics (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

tenant_id UUID NOT NULL,
patient_id UUID NOT NULL,
log_time TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
map_value FLOAT, -- Mean Arterial Pressure (mmHg)
cvp_value FLOAT, -- Central Venous Pressure (mmHg)
cardiac_output FLOAT, -- L/min
stroke_volume FLOAT,
heart_rate INTEGER,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. Advanced Ventilation & Weaning Logs
CREATE TABLE IF NOT EXISTS crit_care_ventilation_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    log_time TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    vent_mode VARCHAR(50), -- e.g., PRVC, APRV, SIMV
    rsbi_value FLOAT, -- Rapid Shallow Breathing Index
    weaning_status VARCHAR(50), -- e.g., Ready, Failed, In-Progress
    fio2_percent FLOAT,
    peep_cmh2o FLOAT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Sepsis Bundle Compliance Tracking
CREATE TABLE IF NOT EXISTS sepsis_bundle_tracking (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    bundle_start_time TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    lactate_initial FLOAT,
    lactate_followup FLOAT,
    antibiotics_administered BOOLEAN DEFAULT FALSE,
    fluid_resuscitation_ml INTEGER,
    bundle_compliance_status VARCHAR(50), -- e.g., Fully Compliant, Partial
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 4. Shock Titration & Vasopressor Logs
CREATE TABLE IF NOT EXISTS shock_titration_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    log_time TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    drug_name VARCHAR(100), -- e.g., Norepinephrine, Epinephrine, Dobutamine
    dose_mcg_kg_min FLOAT,
    response_map_value FLOAT,
    titration_action VARCHAR(50), -- e.g., Increased, Decreased, Maintained
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE crit_care_hemodynamics ENABLE ROW LEVEL SECURITY;

```

```

ALTER TABLE crit_care_ventilation_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE sepsis_bundle_tracking ENABLE ROW LEVEL SECURITY;
ALTER TABLE shock_titration_logs ENABLE ROW LEVEL SECURITY;

-- RLS Policies
CREATE POLICY crit_care_hemodynamics_tenant_policy ON crit_care_hemodynamics
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY crit_care_ventilation_logs_tenant_policy ON crit_care_ventilation_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY sepsis_bundle_tracking_tenant_policy ON sepsis_bundle_tracking
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY shock_titration_logs_tenant_policy ON shock_titration_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

COMMIT;

```

```

=====
FILE: migrations\e78_lab_extensions_up.sql
=====

```

```

-- Migration: e78_lab_extensions_up.sql
-- Description: Adds specialized tables for Central Laboratory and Pathology.
-- Compliance: PDPL / RLS Enabled

```

```

BEGIN;

CREATE TABLE IF NOT EXISTS lab_results_extended (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    test_code VARCHAR(100),
    value FLOAT,
    unit VARCHAR(50),
    reference_range VARCHAR(100),
    is_critical BOOLEAN DEFAULT FALSE,
    verification_status VARCHAR(50), -- Preliminary, Final
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tenant_lab FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
    CONSTRAINT fk_patient_lab FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS lab_qc_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    test_code VARCHAR(100),
    control_value FLOAT,
    expected_value FLOAT,
    variance_pct FLOAT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

```

```

        CONSTRAINT fk_tenant_qc FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE
    );

ALTER TABLE lab_results_extended ENABLE ROW LEVEL SECURITY;
ALTER TABLE lab_qc_logs ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON lab_results_extended USING (tenant_id = current_setting('app.current_tenant')::UUID);
CREATE POLICY tenant_isolation_policy ON lab_qc_logs USING (tenant_id = current_setting('app.current_tenant')::UUID);

COMMIT;

```

```

=====
FILE: migrations\e79_func_extensions_up.sql
=====

```

```

-- Migration: e79_func_extensions_up.sql
-- Description: Adds specialized tables for Functional Tests (ECG, EEG, EMG).
-- Compliance: PDPL / RLS Enabled

```

```

BEGIN;

```

```

CREATE TABLE IF NOT EXISTS functional_test_results (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    test_type VARCHAR(100), -- ECG, EEG, EMG, PFT
    waveform_path TEXT, -- Path to phi_vault/
    interpretation TEXT,
    is_abnormal BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tenant_func FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
    CONSTRAINT fk_patient_func FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

```

```

ALTER TABLE functional_test_results ENABLE ROW LEVEL SECURITY;

```

```

CREATE POLICY tenant_isolation_policy ON functional_test_results USING (tenant_id = current_setting('app.current_tenant')::UUID);

```

```

COMMIT;

```

```

=====
FILE: migrations\e79_rehab_support_down.sql
=====

```

```

-- Migration: e79_rehab_support_down.sql
-- Purpose: Reverse changes from e79_rehab_support_up.sql

```

```
BEGIN;
```

```
DROP TABLE IF EXISTS psychosocial_support_logs;  
DROP TABLE IF EXISTS rehab_speech_logs;  
DROP TABLE IF EXISTS rehab_occupational_logs;  
DROP TABLE IF EXISTS rehab_physical_logs;
```

```
COMMIT;
```

```
=====  
FILE: migrations\e79_rehab_support_up.sql  
=====
```

```
-- Migration: e79_rehab_support_up.sql  
-- Purpose: Implement specialized tables for Rehab & Support (Wave 6)
```

```
BEGIN;
```

```
-- 1. Physical Therapy Logs
```

```
CREATE TABLE IF NOT EXISTS rehab_physical_logs (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    patient_id UUID NOT NULL,  
    log_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    joint_name VARCHAR(100),  
    rom_degrees FLOAT, -- Range of Motion  
    mmt_grade INTEGER, -- Manual Muscle Testing (0-5)  
    gait_status VARCHAR(50), -- e.g., Independent, Assisted, Non-ambulatory  
    balance_score FLOAT,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- 2. Occupational Therapy Logs
```

```
CREATE TABLE IF NOT EXISTS rehab_occupational_logs (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    patient_id UUID NOT NULL,  
    log_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    adl_score INTEGER, -- Activities of Daily Living score  
    adaptive_equipment_needed TEXT,  
    cognitive_function_score FLOAT,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- 3. Speech & Language Pathology Logs
```

```
CREATE TABLE IF NOT EXISTS rehab_speech_logs (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    patient_id UUID NOT NULL,  
    log_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    dysphagia_grade INTEGER, -- 0-4  
    communication_level VARCHAR(50),
```

```

    voice_quality VARCHAR(50),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

#### -- 4. Psychosocial Support Logs

```

CREATE TABLE IF NOT EXISTS psychosocial_support_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    log_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    phq9_score INTEGER, -- Depression
    gad7_score INTEGER, -- Anxiety
    sdoh_markers TEXT, -- Social Determinants of Health
    support_system_grade VARCHAR(20),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

#### -- Enable RLS

```

ALTER TABLE rehab_physical_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE rehab_occupational_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE rehab_speech_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE psychosocial_support_logs ENABLE ROW LEVEL SECURITY;

```

#### -- RLS Policies

```

CREATE POLICY rehab_physical_logs_tenant_policy ON rehab_physical_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

```

```

CREATE POLICY rehab_occupational_logs_tenant_policy ON rehab_occupational_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

```

```

CREATE POLICY rehab_speech_logs_tenant_policy ON rehab_speech_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

```

```

CREATE POLICY psychosocial_support_logs_tenant_policy ON psychosocial_support_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

```

```

COMMIT;

```

```

=====

```

```

FILE: migrations\e7_01_emergency_ed_workflow_down.sql

```

```

=====

```

```

-- e7_01_emergency_ed_workflow_down.sql (rollback of e7_01_emergency_ed_workflow_up.sql)

```

```

-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.

```

```

--

```

```

-- ??????: emergency_visits ???? ???? ?????? (?? ????? ??? ??????) ? ???? ?? ????? ??????.

```

```

-- ?????? ??? ?? ?????? E7: ?????? ?? ?????? ?? ?????? ????????? (?? ?????? ??? tenant_id

```

```

-- ??? ???? ?????? ??? E7). idempotent (IF EXISTS).

```

```

BEGIN;

```

```

-- 1. policy first (RLS additions).

```

```

DROP POLICY IF EXISTS rls_emergency_visits_tenant_isolation ON emergency_visits;

```

```

ALTER TABLE IF EXISTS emergency_visits NO FORCE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS emergency_visits DISABLE ROW LEVEL SECURITY;

-- 2. constraints added by E7.
ALTER TABLE IF EXISTS emergency_visits DROP CONSTRAINT IF EXISTS chk_emergency_visits_phase;
ALTER TABLE IF EXISTS emergency_visits DROP CONSTRAINT IF EXISTS chk_emergency_visits_esi;
ALTER TABLE IF EXISTS emergency_visits DROP CONSTRAINT IF EXISTS fk_emergency_visits_tenant;

-- 3. indexes added by E7.
DROP INDEX IF EXISTS idx_emergency_visits_tenant_id;
DROP INDEX IF EXISTS idx_emergency_visits_board;

-- 4. workflow columns added by E7 (tenant_id is intentionally retained ? pre-dates E7).
ALTER TABLE IF EXISTS emergency_visits DROP COLUMN IF EXISTS esi_level;
ALTER TABLE IF EXISTS emergency_visits DROP COLUMN IF EXISTS esi_rationale;
ALTER TABLE IF EXISTS emergency_visits DROP COLUMN IF EXISTS er_phase;
ALTER TABLE IF EXISTS emergency_visits DROP COLUMN IF EXISTS triage_started_at;
ALTER TABLE IF EXISTS emergency_visits DROP COLUMN IF EXISTS provider_assigned_at;
ALTER TABLE IF EXISTS emergency_visits DROP COLUMN IF EXISTS time_to_provider_min;
ALTER TABLE IF EXISTS emergency_visits DROP COLUMN IF EXISTS disposition_type;

COMMIT;

=====
FILE: migrations\e7_01_emergency_ed_workflow_up.sql
=====

-- =====
-- e7_01_emergency_ed_workflow_up.sql
-- E7 EMERGENCY DEPARTMENT ? ESI triage + workflow state machine on emergency_visits.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ????? ???? ?????? ??????? ?????? (emergency_visits) ?????? ??? ??? ???????:
--      ??? ???? ??????? ??????? ??????? (esi_level / esi_rationale)? ????? ????? (er_phase)?
--      ?????? ????? (triage_started_at / provider_assigned_at / time_to_provider_min) ??? ??????
--      (disposition_type). ?? ????? ??? ????? ??? 150 ????? FORCE RLS ??? emergency_visits
--      (?????? ??????? ??? tenant_id ??? ??? ?????? ??? ? ??? ????? E7).
--
-- emergency_visits ????: ?????? ADD COLUMN IF NOT EXISTS? ??? tenant_id ??????? ?? ???
-- NOT NULL ????? FK ??? tenants(id) ??? idempotent. patient_id ??? INTEGER (??? ??????
-- ?? ??? FK ??? patient_id ?? ??????? ??????? ? ?????? ?????-????? + RLS).
--
-- idempotent: ADD COLUMN IF NOT EXISTS + DROP/ADD CONSTRAINT IF EXISTS + DROP POLICY IF EXISTS
-- + CREATE INDEX IF NOT EXISTS. wrapped BEGIN; ? COMMIT;
-- =====
BEGIN;

-- ----- 1. ED workflow / ESI columns -----
ALTER TABLE emergency_visits ADD COLUMN IF NOT EXISTS esi_level INTEGER DEFAULT 0;
ALTER TABLE emergency_visits ADD COLUMN IF NOT EXISTS esi_rationale TEXT DEFAULT '';
ALTER TABLE emergency_visits ADD COLUMN IF NOT EXISTS er_phase TEXT DEFAULT 'Arrival';
ALTER TABLE emergency_visits ADD COLUMN IF NOT EXISTS triage_started_at TEXT DEFAULT '';

```

```

ALTER TABLE emergency_visits ADD COLUMN IF NOT EXISTS provider_assigned_at TEXT DEFAULT '';
ALTER TABLE emergency_visits ADD COLUMN IF NOT EXISTS time_to_provider_min INTEGER DEFAULT 0;
ALTER TABLE emergency_visits ADD COLUMN IF NOT EXISTS disposition_type TEXT DEFAULT '';

-- er_phase domain guard (idempotent: drop then add).
ALTER TABLE emergency_visits DROP CONSTRAINT IF EXISTS chk_emergency_visits_phase;
ALTER TABLE emergency_visits ADD CONSTRAINT chk_emergency_visits_phase
    CHECK (er_phase IN ('Arrival','Triage','Waiting','InTreatment','Disposition'));

-- esi_level domain guard (0 = not yet triaged; 1..5 = ESI).
ALTER TABLE emergency_visits DROP CONSTRAINT IF EXISTS chk_emergency_visits_es;
ALTER TABLE emergency_visits ADD CONSTRAINT chk_emergency_visits_es;
    CHECK (esi_level BETWEEN 0 AND 5);

-- ----- 2. tenant_id hardening + FK (table pre-exists with nullable tenant_id) -----
ALTER TABLE emergency_visits ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE emergency_visits SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE emergency_visits ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE emergency_visits DROP CONSTRAINT IF EXISTS fk_emergency_visits_tenant;
ALTER TABLE emergency_visits ADD CONSTRAINT fk_emergency_visits_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;

CREATE INDEX IF NOT EXISTS idx_emergency_visits_tenant_id ON emergency_visits (tenant_id);
CREATE INDEX IF NOT EXISTS idx_emergency_visits_board ON emergency_visits (tenant_id, status, esi_level);

-- ----- 3. FORCE RLS + canonical tenant-isolation policy (closes the E7 RLS gap) -----
ALTER TABLE emergency_visits ENABLE ROW LEVEL SECURITY;
ALTER TABLE emergency_visits FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_emergency_visits_tenant_isolation ON emergency_visits;
CREATE POLICY rls_emergency_visits_tenant_isolation ON emergency_visits
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

=====
FILE: migrations\e7_01_emergency_ed_workflow_validate.sql
=====

-- e7_01_emergency_ed_workflow_validate.sql (run AFTER e7_01_emergency_ed_workflow_up.sql; read-only)
-- PASS = emergency_visits has ESI/workflow columns + tenant_id NOT NULL FK + FORCE RLS + isolation policy + C
HECKS + indexes.

SELECT
    (SELECT count(*) FROM information_schema.columns
     WHERE table_name='emergency_visits'
        AND column_name IN ('esi_level','esi_rationale','er_phase','triage_started_at',
                            'provider_assigned_at','time_to_provider_min','disposition_type')) AS workflow_cols
, -- expect 7
    (SELECT relforcerowsecurity FROM pg_class WHERE relname='emergency_visits') AS force_rls,
-- expect t

```

```

(SELECT relrowsecurity FROM pg_class WHERE relname='emergency_visits') AS rls_enabled,
-- expect 1
(SELECT count(*) FROM pg_policies WHERE tablename='emergency_visits'
    AND policyname='rls_emergency_visits_tenant_isolation') AS isolation_policy,
-- expect 1
(SELECT count(*) FROM pg_constraint WHERE conname='chk_emergency_visits_phase') AS phase_check,
-- expect 1
(SELECT count(*) FROM pg_constraint WHERE conname='chk_emergency_visits_esi') AS esi_check,
-- expect 1
(SELECT count(*) FROM pg_indexes WHERE tablename='emergency_visits' AND indexname='idx_emergency_visits_tenant_id') AS tenant_idx, -- expect 1
(SELECT count(*) FROM pg_indexes WHERE tablename='emergency_visits' AND indexname='idx_emergency_visits_board') AS board_idx;      -- expect 1

-- ----- tenant_id is NOT NULL -----
SELECT count(*)::int AS tenant_id_not_null FROM information_schema.columns
    WHERE table_name='emergency_visits' AND column_name='tenant_id' AND is_nullable='NO';
-- expect 1

-- ----- FK to tenants(id) -----
SELECT count(*)::int AS fk_to_tenants FROM pg_constraint
    WHERE conrelid='emergency_visits'::regclass AND confrelid='tenants'::regclass AND contype='f';
-- expect 1

=====
FILE: migrations\e7_02_emergency_rls_down.sql
=====

-- e7_02_emergency_rls_down.sql (rollback of e7_02_emergency_rls_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ??????: ??????? ?????? ?????? ? ?? ????????. ?????? ??? ?? ?????? + FK + ??? E7
-- (????? tenant_id ??? E7). ?????? ?????? ?? ??????. idempotent (IF EXISTS).
BEGIN;

-- ===== emergency_trauma_assessments =====
DROP POLICY IF EXISTS rls_emergency_trauma_tenant_isolation ON emergency_trauma_assessments;
ALTER TABLE IF EXISTS emergency_trauma_assessments NO FORCE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS emergency_trauma_assessments DISABLE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS emergency_trauma_assessments DROP CONSTRAINT IF EXISTS fk_emergency_trauma_tenant;
DROP INDEX IF EXISTS idx_emergency_trauma_tenant_id;

-- ===== emergency_beds =====
DROP POLICY IF EXISTS rls_emergency_beds_tenant_isolation ON emergency_beds;
ALTER TABLE IF EXISTS emergency_beds NO FORCE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS emergency_beds DISABLE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS emergency_beds DROP CONSTRAINT IF EXISTS fk_emergency_beds_tenant;
DROP INDEX IF EXISTS idx_emergency_beds_tenant_id;

COMMIT;

```

```

=====
FILE: migrations\e7_02_emergency_rls_up.sql
=====

-- =====
-- e7_02_emergency_rls_up.sql
-- E7 EMERGENCY DEPARTMENT ? FORCE RLS on emergency_trauma_assessments + emergency_beds.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ??? ??? E7 ??????? ? ??? ??????? ?????? tenant_id ??? ??? ?????? ??? FORCE RLS.
-- ????? ??? ?????? ??????? (USING/WITH CHECK ??? app.tenant_id) + ??? tenant_id NOT NULL + FK.
-- ??????: emergency_beds ?????? branch_id (???? facility_id) ??? ?????? ??? ??? tenant_id.
--
-- idempotent: UPDATE backfill + SET NOT NULL + DROP/ADD CONSTRAINT IF EXISTS + DROP POLICY IF EXISTS
-- + CREATE INDEX IF NOT EXISTS. wrapped BEGIN; ? COMMIT;
-- =====
BEGIN;

-- ===== emergency_trauma_assessments =====
ALTER TABLE emergency_trauma_assessments ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE emergency_trauma_assessments SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE emergency_trauma_assessments ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE emergency_trauma_assessments DROP CONSTRAINT IF EXISTS fk_emergency_trauma_tenant;
ALTER TABLE emergency_trauma_assessments ADD CONSTRAINT fk_emergency_trauma_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_emergency_trauma_tenant_id ON emergency_trauma_assessments (tenant_id);

ALTER TABLE emergency_trauma_assessments ENABLE ROW LEVEL SECURITY;
ALTER TABLE emergency_trauma_assessments FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_emergency_trauma_tenant_isolation ON emergency_trauma_assessments;
CREATE POLICY rls_emergency_trauma_tenant_isolation ON emergency_trauma_assessments
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== emergency_beds =====
ALTER TABLE emergency_beds ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE emergency_beds SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE emergency_beds ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE emergency_beds DROP CONSTRAINT IF EXISTS fk_emergency_beds_tenant;
ALTER TABLE emergency_beds ADD CONSTRAINT fk_emergency_beds_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_emergency_beds_tenant_id ON emergency_beds (tenant_id);

ALTER TABLE emergency_beds ENABLE ROW LEVEL SECURITY;
ALTER TABLE emergency_beds FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_emergency_beds_tenant_isolation ON emergency_beds;
CREATE POLICY rls_emergency_beds_tenant_isolation ON emergency_beds
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

=====

FILE: migrations\e7\_02\_emergency\_rls\_validate.sql

=====

-- e7\_02\_emergency\_rls\_validate.sql (run AFTER e7\_02\_emergency\_rls\_up.sql; read-only)  
-- PASS = both ED child tables have tenant\_id NOT NULL FK + FORCE RLS + isolation policy + index.

SELECT

```
(SELECT relforcerowsecurity FROM pg_class WHERE relname='emergency_trauma_assessments') AS trauma_force_rls,
-- expect t
(SELECT count(*) FROM pg_policies WHERE tablename='emergency_trauma_assessments'
AND policyname='rls_emergency_trauma_tenant_isolation') AS trauma_policy,
-- expect 1
(SELECT count(*) FROM pg_indexes WHERE tablename='emergency_trauma_assessments'
AND indexname='idx_emergency_trauma_tenant_id') AS trauma_idx,
-- expect 1
(SELECT count(*)::int FROM information_schema.columns WHERE table_name='emergency_trauma_assessments'
AND column_name='tenant_id' AND is_nullable='NO') AS trauma_tenant_not_null,
-- expect 1
(SELECT count(*)::int FROM pg_constraint
WHERE conrelid='emergency_trauma_assessments'::regclass AND confrelid='tenants'::regclass AND contype='
f') AS trauma_fk, -- expect 1
(SELECT relforcerowsecurity FROM pg_class WHERE relname='emergency_beds') AS beds_force_rls,
-- expect t
(SELECT count(*) FROM pg_policies WHERE tablename='emergency_beds'
AND policyname='rls_emergency_beds_tenant_isolation') AS beds_policy,
-- expect 1
(SELECT count(*) FROM pg_indexes WHERE tablename='emergency_beds'
AND indexname='idx_emergency_beds_tenant_id') AS beds_idx,
-- expect 1
(SELECT count(*)::int FROM information_schema.columns WHERE table_name='emergency_beds'
AND column_name='tenant_id' AND is_nullable='NO') AS beds_tenant_not_null,
-- expect 1
(SELECT count(*)::int FROM pg_constraint
WHERE conrelid='emergency_beds'::regclass AND confrelid='tenants'::regclass AND contype='f') AS beds_fk
; -- expect 1
```

=====

FILE: migrations\e80\_admin\_ops\_down.sql

=====

-- Migration: e80\_admin\_ops\_down.sql  
-- Purpose: Reverse changes from e80\_admin\_ops\_up.sql

BEGIN;

```
DROP TABLE IF EXISTS supply_chain_metrics;
DROP TABLE IF EXISTS hcm_credentialing_logs;
DROP TABLE IF EXISTS financial_integrity_logs;
```

```
DROP TABLE IF EXISTS admin_resource_logs;
```

```
COMMIT;
```

```
=====
FILE: migrations\e80_admin_ops_up.sql
=====
```

```
-- Migration: e80_admin_ops_up.sql
-- Purpose: Implement specialized tables for Admin & Ops (Wave 7)
```

```
BEGIN;
```

```
-- 1. Resource Optimization Logs
```

```
CREATE TABLE IF NOT EXISTS admin_resource_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    log_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    bed_occupancy_percent FLOAT,
    staff_patient_ratio FLOAT,
    or_utilization_percent FLOAT,
    resource_bottleneck VARCHAR(100),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 2. Financial Integrity & Leakage Logs
```

```
CREATE TABLE IF NOT EXISTS financial_integrity_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    log_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    revenue_leakage_amount FLOAT,
    cost_per_case_avg FLOAT,
    billing_anomaly_detected BOOLEAN,
    anomaly_description TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 3. HCM & Credentialing Logs
```

```
CREATE TABLE IF NOT EXISTS hcm_credentialing_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    physician_id UUID,
    license_expiry_date DATE,
    credentialing_status VARCHAR(50), -- e.g., Active, Pending, Expired
    last_audit_date DATE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 4. Supply Chain & Cold-Chain Metrics
```

```
CREATE TABLE IF NOT EXISTS supply_chain_metrics (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
```

```

        item_id UUID,
        inventory_level INTEGER,
        cold_chain_temp_c FLOAT,
        temp_deviation_alert BOOLEAN,
        reorder_point_reached BOOLEAN,
        created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
    );

-- Enable RLS
ALTER TABLE admin_resource_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE financial_integrity_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE hcm_credentialing_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE supply_chain_metrics ENABLE ROW LEVEL SECURITY;

-- RLS Policies
CREATE POLICY admin_resource_logs_tenant_policy ON admin_resource_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY financial_integrity_logs_tenant_policy ON financial_integrity_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY hcm_credentialing_logs_tenant_policy ON hcm_credentialing_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY supply_chain_metrics_tenant_policy ON supply_chain_metrics
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

COMMIT;

```

```

=====
FILE: migrations\e80_critical_care_down.sql
=====

```

```

-- Migration: e80_critical_care_down.sql
BEGIN;
DROP TABLE IF EXISTS anesthesia_records;
DROP TABLE IF EXISTS icu_vital_streams;
DROP TABLE IF EXISTS er_triage_logs;
COMMIT;

```

```

=====
FILE: migrations\e80_critical_care_up.sql
=====

```

```

-- Migration: e80_critical_care_up.sql
-- Description: Adds specialized tables for Emergency, ICU, and Anesthesia.
-- Compliance: PDPL / RLS Enabled

BEGIN;

```

-- 1. ER Triage Logs

```
CREATE TABLE IF NOT EXISTS er_triage_logs (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    patient_id UUID NOT NULL,  
    esi_level INTEGER, -- 1 (Resuscitation) to 5 (Non-Urgent)  
    chief_complaint TEXT,  
    triage_vitals JSONB, -- { "bp": "120/80", "hr": 90, "spo2": 95 }  
    triage_time TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_tenant_er FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,  
    CONSTRAINT fk_patient_er FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE  
);
```

-- 2. ICU Vital Streams (High Frequency)

```
CREATE TABLE IF NOT EXISTS icu_vital_streams (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    patient_id UUID NOT NULL,  
    record_time TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    map_mmHg INTEGER,  
    heart_rate_bpm INTEGER,  
    spo2_pct FLOAT,  
    respiratory_rate_bpm INTEGER,  
    temp_c FLOAT,  
    mean_arterial_pressure FLOAT,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_tenant_icu FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,  
    CONSTRAINT fk_patient_icu FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE  
);
```

-- 3. Anesthesia Records

```
CREATE TABLE IF NOT EXISTS anesthesia_records (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    session_id UUID NOT NULL,  
    patient_id UUID NOT NULL,  
    drug_used VARCHAR(255),  
    dose_mg FLOAT,  
    intubation_time TIMESTAMP WITH TIME ZONE,  
    extubation_time TIMESTAMP WITH TIME ZONE,  
    pacu_recovery_score INTEGER,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_tenant_anes FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,  
    CONSTRAINT fk_patient_anes FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE  
);
```

-- Enable Row Level Security (RLS)

```
ALTER TABLE er_triage_logs ENABLE ROW LEVEL SECURITY;  
ALTER TABLE icu_vital_streams ENABLE ROW LEVEL SECURITY;  
ALTER TABLE anesthesia_records ENABLE ROW LEVEL SECURITY;
```

-- Create RLS Policies (Golden Access Rule: Tenant Isolation)

```
CREATE POLICY tenant_isolation_policy ON er_triage_logs USING (tenant_id = current_setting('app.current_tenant
```

```
'):::UUID);
CREATE POLICY tenant_isolation_policy ON icu_vital_streams USING (tenant_id = current_setting('app.current_tenant')::UUID);
CREATE POLICY tenant_isolation_policy ON anesthesia_records USING (tenant_id = current_setting('app.current_tenant')::UUID);

COMMIT;
```

=====

FILE: migrations\e81\_rare\_specialties\_down.sql

=====

```
-- Migration: e81_rare_specialties_down.sql
-- Purpose: Reverse changes from e81_rare_specialties_up.sql
```

```
BEGIN;
```

```
DROP TABLE IF EXISTS regenerative_logs;
DROP TABLE IF EXISTS nanomedicine_metrics;
DROP TABLE IF EXISTS rare_disease_logs;
```

```
COMMIT;
```

=====

FILE: migrations\e81\_rare\_specialties\_up.sql

=====

```
-- Migration: e81_rare_specialties_up.sql
-- Purpose: Implement specialized tables for Rare Specialties & Nanomedicine (Wave 10)
```

```
BEGIN;
```

```
-- 1. Rare Disease & Orphan Registry
```

```
CREATE TABLE IF NOT EXISTS rare_disease_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    log_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    phenotype_description TEXT,
    genotype_mutation VARCHAR(255), -- e.g., CFTR DeltaF508
    orphan_drug_used VARCHAR(100),
    treatment_response TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 2. Nanomedicine & Targeted Delivery Logs
```

```
CREATE TABLE IF NOT EXISTS nanomedicine_metrics (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
```

```

log_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
nanoparticle_type VARCHAR(100), -- e.g., Liposomal, Gold NP, Carbon Nanotube
targeting_ligand VARCHAR(100),
delivery_efficiency_percent FLOAT,
toxicity_score FLOAT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Regenerative Medicine & Stem Cell Logs
CREATE TABLE IF NOT EXISTS regenerative_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    patient_id UUID NOT NULL,
    log_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    therapy_type VARCHAR(100), -- e.g., MSCs, iPSCs, Organoid Graft
    la-Surgical logic for tissue regeneration.
    regeneration_rate_mm_day FLOAT,
    viability_percent FLOAT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE rare_disease_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE nanomedicine_metrics ENABLE ROW LEVEL SECURITY;
ALTER TABLE regenerative_logs ENABLE ROW LEVEL SECURITY;

-- RLS Policies
CREATE POLICY rare_disease_logs_tenant_policy ON rare_disease_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY nanomedicine_metrics_tenant_policy ON nanomedicine_metrics
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY regenerative_logs_tenant_policy ON regenerative_logs
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

COMMIT;

=====
FILE: migrations\e8_01_inpatient_adt_qls_down.sql
=====

-- e8_01_inpatient_adt_qls_down.sql (rollback of e8_01_inpatient_adt_qls_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ??????: ?????? ?????? ?????? ?????? ? ?? ???????. ?????? ??? ?? ?????? E8 (???????? ??
-- FORCE/ENABLE ?? ?????? ?? ??????)? ?????? ????? tenant_id ????? E8. ?????? ??????.
-- idempotent (IF EXISTS).
BEGIN;

-- ===== admissions =====
DROP POLICY IF EXISTS qls_admissions_tenant_isolation ON admissions;

```

```

ALTER TABLE IF EXISTS admissions NO FORCE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS admissions DISABLE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS admissions DROP CONSTRAINT IF EXISTS fk_admissions_tenant;
DROP INDEX IF EXISTS idx_admissions_tenant_id;

-- ===== beds =====
DROP POLICY IF EXISTS rls_beds_tenant_isolation ON beds;
ALTER TABLE IF EXISTS beds NO FORCE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS beds DISABLE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS beds DROP CONSTRAINT IF EXISTS fk_beds_tenant;
ALTER TABLE IF EXISTS beds DROP CONSTRAINT IF EXISTS chk_beds_status;
DROP INDEX IF EXISTS idx_beds_tenant_id;

-- ===== wards =====
DROP POLICY IF EXISTS rls_wards_tenant_isolation ON wards;
ALTER TABLE IF EXISTS wards NO FORCE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS wards DISABLE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS wards DROP CONSTRAINT IF EXISTS fk_wards_tenant;
DROP INDEX IF EXISTS idx_wards_tenant_id;

-- ===== bed_transfers =====
DROP POLICY IF EXISTS rls_bed_transfers_tenant_isolation ON bed_transfers;
ALTER TABLE IF EXISTS bed_transfers NO FORCE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS bed_transfers DISABLE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS bed_transfers DROP CONSTRAINT IF EXISTS fk_bed_transfers_tenant;
DROP INDEX IF EXISTS idx_bed_transfers_tenant_id;

-- ===== admission_daily_rounds =====
DROP POLICY IF EXISTS rls_admission_rounds_tenant_isolation ON admission_daily_rounds;
ALTER TABLE IF EXISTS admission_daily_rounds NO FORCE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS admission_daily_rounds DISABLE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS admission_daily_rounds DROP CONSTRAINT IF EXISTS fk_admission_rounds_tenant;
DROP INDEX IF EXISTS idx_admission_rounds_tenant_id;

COMMIT;

```

```

=====
FILE: migrations\e8_01_inpatient_adt_rls_up.sql
=====

```

```

-- =====
-- e8_01_inpatient_adt_rls_up.sql
-- E8 INPATIENT / ADT ? FORCE RLS on the five existing ADT tables.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ??? ??? E8 ??????? ? ??? ?????? ?????? (admissions, beds, wards, bed_transfers,
-- admission_daily_rounds) ??? tenant_id ??? ??? ????? ??? FORCE RLS. ????? ??? ?????
-- ??????? (USING/WITH CHECK ??? app.tenant_id) + ??? tenant_id NOT NULL + FK -> tenants(id).
-- ??????: bed_transfers ?????? branch_id (???? facility_id) ??? ????? ??? ??? tenant_id.
--
-- ??? ??????? ?????? ?????? (?? ?????? ??? ??????) ? ?????? ADD COLUMN IF NOT EXISTS + backfill
-- + SET NOT NULL + DROP/ADD CONSTRAINT IF EXISTS + DROP POLICY IF EXISTS + CREATE INDEX

```

```

-- IF NOT EXISTS. idempotent. wrapped BEGIN; ? COMMIT;
-- =====
BEGIN;

-- ===== admissions =====
ALTER TABLE admissions ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE admissions SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE admissions ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE admissions DROP CONSTRAINT IF EXISTS fk_admissions_tenant;
ALTER TABLE admissions ADD CONSTRAINT fk_admissions_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_admissions_tenant_id ON admissions (tenant_id);
ALTER TABLE admissions ENABLE ROW LEVEL SECURITY;
ALTER TABLE admissions FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_admissions_tenant_isolation ON admissions;
CREATE POLICY rls_admissions_tenant_isolation ON admissions
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== beds =====
ALTER TABLE beds ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE beds SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE beds ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE beds DROP CONSTRAINT IF EXISTS fk_beds_tenant;
ALTER TABLE beds ADD CONSTRAINT fk_beds_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_beds_tenant_id ON beds (tenant_id);
-- bed status domain guard (server lifecycle: Available/Reserved/Occupied/Cleaning/Blocked).
ALTER TABLE beds DROP CONSTRAINT IF EXISTS chk_beds_status;
ALTER TABLE beds ADD CONSTRAINT chk_beds_status
    CHECK (status IN ('Available', 'Reserved', 'Occupied', 'Cleaning', 'Blocked'));
ALTER TABLE beds ENABLE ROW LEVEL SECURITY;
ALTER TABLE beds FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_beds_tenant_isolation ON beds;
CREATE POLICY rls_beds_tenant_isolation ON beds
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== wards =====
ALTER TABLE wards ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE wards SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE wards ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE wards DROP CONSTRAINT IF EXISTS fk_wards_tenant;
ALTER TABLE wards ADD CONSTRAINT fk_wards_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_wards_tenant_id ON wards (tenant_id);
ALTER TABLE wards ENABLE ROW LEVEL SECURITY;
ALTER TABLE wards FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_wards_tenant_isolation ON wards;
CREATE POLICY rls_wards_tenant_isolation ON wards
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)

```

```

WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== bed_transfers =====
ALTER TABLE bed_transfers ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE bed_transfers SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE bed_transfers ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE bed_transfers DROP CONSTRAINT IF EXISTS fk_bed_transfers_tenant;
ALTER TABLE bed_transfers ADD CONSTRAINT fk_bed_transfers_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_bed_transfers_tenant_id ON bed_transfers (tenant_id);
ALTER TABLE bed_transfers ENABLE ROW LEVEL SECURITY;
ALTER TABLE bed_transfers FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_bed_transfers_tenant_isolation ON bed_transfers;
CREATE POLICY rls_bed_transfers_tenant_isolation ON bed_transfers
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== admission_daily_rounds =====
ALTER TABLE admission_daily_rounds ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE admission_daily_rounds SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE admission_daily_rounds ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE admission_daily_rounds DROP CONSTRAINT IF EXISTS fk_admission_rounds_tenant;
ALTER TABLE admission_daily_rounds ADD CONSTRAINT fk_admission_rounds_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_admission_rounds_tenant_id ON admission_daily_rounds (tenant_id);
ALTER TABLE admission_daily_rounds ENABLE ROW LEVEL SECURITY;
ALTER TABLE admission_daily_rounds FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_admission_rounds_tenant_isolation ON admission_daily_rounds;
CREATE POLICY rls_admission_rounds_tenant_isolation ON admission_daily_rounds
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

=====
FILE: migrations\e8_01_inpatient_adt_qls_validate.sql
=====

-- e8_01_inpatient_adt_qls_validate.sql (run AFTER e8_01_inpatient_adt_qls_up.sql; read-only)
-- PASS = all five ADT tables have tenant_id NOT NULL FK + FORCE RLS + isolation policy + index.

SELECT
    (SELECT relforcerowsecurity FROM pg_class WHERE relname='admissions')                AS admissions_force_qls,
    -- expect t
    (SELECT count(*) FROM pg_policies WHERE tablename='admissions'
        AND policyname='qls_admissions_tenant_isolation')                            AS admissions_policy,
    -- expect 1
    (SELECT count(*)::int FROM information_schema.columns WHERE table_name='admissions'
        AND column_name='tenant_id' AND is_nullable='NO')                            AS admissions_tenant_not_
null, -- expect 1

```

```

(SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='admissions'::regclass AND confrelid='tenants'::regclass AND contype='f') AS admissions_
fk,    -- expect 1

(SELECT relforcerowsecurity FROM pg_class WHERE relname='beds')                AS beds_force_rls,
    -- expect t
(SELECT count(*) FROM pg_policies WHERE tablename='beds'
 AND polycname='rls_beds_tenant_isolation')                                AS beds_policy,
    -- expect 1
(SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='beds'::regclass AND conname='chk_beds_status')            AS beds_status_chk,
    -- expect 1

(SELECT relforcerowsecurity FROM pg_class WHERE relname='wards')            AS wards_force_rls,
    -- expect t
(SELECT count(*) FROM pg_policies WHERE tablename='wards'
 AND polycname='rls_wards_tenant_isolation')                                AS wards_policy,
    -- expect 1

(SELECT relforcerowsecurity FROM pg_class WHERE relname='bed_transfers')    AS bed_transfers_force_r
ls,    -- expect t
(SELECT count(*) FROM pg_policies WHERE tablename='bed_transfers'
 AND polycname='rls_bed_transfers_tenant_isolation')                        AS bed_transfers_policy,
    -- expect 1

(SELECT relforcerowsecurity FROM pg_class WHERE relname='admission_daily_rounds') AS rounds_force_rls,
    -- expect t
(SELECT count(*) FROM pg_policies WHERE tablename='admission_daily_rounds'
 AND polycname='rls_admission_rounds_tenant_isolation')                    AS rounds_policy;
    -- expect 1

```

```

=====
FILE: migrations\e8_02_bed_status_history_down.sql
=====

```

```

-- e8_02_bed_status_history_down.sql (rollback of e8_02_bed_status_history_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ??????: bed_status_history ???? ???? ?????? ??? ?????? ? ???? ?????? ?????? ?????? (??? ??
-- ?????? ?????? ???????? ?????? ??????) ?????? ???? ?????? E8. idempotent (IF EXISTS).
BEGIN;

-- 1. drop the table first (cascades its policy/indexes/constraints).
DROP TABLE IF EXISTS bed_status_history CASCADE;

-- 2. defensive: drop the policy explicitly if the table somehow lingered.
DROP POLICY IF EXISTS rls_bed_status_history_tenant_isolation ON bed_status_history;

COMMIT;

```

```

=====
FILE: migrations\e8_02_bed_status_history_up.sql
=====

-- =====
-- e8_02_bed_status_history_up.sql
-- E8 INPATIENT / ADT ? NEW table: bed_status_history (audit of bed-status transitions).
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ??? ???? ?????? ??? ???? ???? ?????? (Available/Reserved/Occupied/Cleaning/Blocked)
-- ? ? ???? ??? ?????? ?????? ??? ?????? ?? ?????? ?????????/?????? ???? ??????????. ??? ???? ?????
-- ??? tenant_id INTEGER NOT NULL REFERENCES tenants(id) ??? ??????? + FORCE RLS ??????? ??????????.
-- ??????: ??? ?????? NOT ????? ??? bootstrap ?? db_postgres.js (???? ?????? ???).
--
-- idempotent: CREATE TABLE IF NOT EXISTS + CREATE INDEX IF NOT EXISTS + DROP/ADD POLICY IF EXISTS.
-- =====
BEGIN;

CREATE TABLE IF NOT EXISTS bed_status_history (
    id                SERIAL PRIMARY KEY,
    bed_id            INTEGER NOT NULL REFERENCES beds(id) ON DELETE CASCADE,
    admission_id      INTEGER REFERENCES admissions(id) ON DELETE SET NULL,
    patient_id        INTEGER REFERENCES patients(id) ON DELETE SET NULL,
    from_status       TEXT NOT NULL,
    to_status         TEXT NOT NULL,
    reason            TEXT DEFAULT '',
    changed_by        TEXT DEFAULT '',
    changed_at        TIMESTAMP DEFAULT now(),
    tenant_id         INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    facility_id       INTEGER,
    CONSTRAINT chk_bed_status_history_to CHECK (to_status IN ('Available','Reserved','Occupied','Cleaning'
, 'Blocked')),
    CONSTRAINT chk_bed_status_history_from CHECK (from_status IN ('Available','Reserved','Occupied','Cleaning'
, 'Blocked'))
);

CREATE INDEX IF NOT EXISTS idx_bed_status_history_tenant_id ON bed_status_history (tenant_id);
CREATE INDEX IF NOT EXISTS idx_bed_status_history_bed ON bed_status_history (tenant_id, bed_id);

ALTER TABLE bed_status_history ENABLE ROW LEVEL SECURITY;
ALTER TABLE bed_status_history FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_bed_status_history_tenant_isolation ON bed_status_history;
CREATE POLICY rls_bed_status_history_tenant_isolation ON bed_status_history
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

=====
FILE: migrations\e8_02_bed_status_history_validate.sql
=====

```

```
-- e8_02_bed_status_history_validate.sql (run AFTER e8_02_bed_status_history_up.sql; read-only)
-- PASS = bed_status_history exists with tenant_id NOT NULL FK + FORCE RLS + isolation policy + index.

SELECT
  (SELECT count(*)::int FROM information_schema.tables WHERE table_name='bed_status_history')      AS table_exi
sts,      -- expect 1
  (SELECT relforcerowsecurity FROM pg_class WHERE relname='bed_status_history')                AS force_rls
,      -- expect t
  (SELECT count(*) FROM pg_policies WHERE tablename='bed_status_history'
    AND policyname='rls_bed_status_history_tenant_isolation')                                AS policy,
    -- expect 1
  (SELECT count(*)::int FROM information_schema.columns WHERE table_name='bed_status_history'
    AND column_name='tenant_id' AND is_nullable='NO')                                      AS tenant_no
t_null,   -- expect 1
  (SELECT count(*)::int FROM pg_constraint
    WHERE conrelid='bed_status_history'::regclass AND confrelid='tenants'::regclass AND contype='f') AS ten
ant_fk,   -- expect 1
  (SELECT count(*)::int FROM pg_constraint
    WHERE conrelid='bed_status_history'::regclass AND confrelid='beds'::regclass AND contype='f')   AS bed
_fk,      -- expect 1
  (SELECT count(*)::int FROM pg_constraint
    WHERE conrelid='bed_status_history'::regclass AND confrelid='patients'::regclass AND contype='f') AS pa
tient_fk, -- expect 1
  (SELECT count(*) FROM pg_indexes WHERE tablename='bed_status_history'
    AND indexname='idx_bed_status_history_tenant_id')                                      AS idx;
    -- expect 1
```

```
=====
FILE: migrations\e9_01_icu_rls_down.sql
=====
```

```
-- e9_01_icu_rls_down.sql (rollback of e9_01_icu_rls_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ??????: ?????? ?????? ?????? ?????? ? ?? ??????. ?????? ??? ?? ?????? E9 (?????? ??
-- FORCE/ENABLE ?? ?????? ?? ??????)? ?????? ?????? tenant_id ?????? E9 (????? ?? bootstrap).
-- ?????? ??????. idempotent (IF EXISTS).
BEGIN;
```

```
-- ===== icu_monitoring =====
DROP POLICY IF EXISTS rls_icu_monitoring_tenant_isolation ON icu_monitoring;
ALTER TABLE IF EXISTS icu_monitoring NO FORCE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS icu_monitoring DISABLE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS icu_monitoring DROP CONSTRAINT IF EXISTS fk_icu_monitoring_tenant;
DROP INDEX IF EXISTS idx_icu_monitoring_tenant_id;
```

```
-- ===== icu_ventilator =====
DROP POLICY IF EXISTS rls_icu_ventilator_tenant_isolation ON icu_ventilator;
ALTER TABLE IF EXISTS icu_ventilator NO FORCE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS icu_ventilator DISABLE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS icu_ventilator DROP CONSTRAINT IF EXISTS fk_icu_ventilator_tenant;
```

```
DROP INDEX IF EXISTS idx_icu_ventilator_tenant_id;
```

```
-- ===== icu_scores =====
```

```
DROP POLICY IF EXISTS rls_icu_scores_tenant_isolation ON icu_scores;  
ALTER TABLE IF EXISTS icu_scores NO FORCE ROW LEVEL SECURITY;  
ALTER TABLE IF EXISTS icu_scores DISABLE ROW LEVEL SECURITY;  
ALTER TABLE IF EXISTS icu_scores DROP CONSTRAINT IF EXISTS fk_icu_scores_tenant;  
DROP INDEX IF EXISTS idx_icu_scores_tenant_id;
```

```
-- ===== icu_fluid_balance =====
```

```
DROP POLICY IF EXISTS rls_icu_fluid_balance_tenant_isolation ON icu_fluid_balance;  
ALTER TABLE IF EXISTS icu_fluid_balance NO FORCE ROW LEVEL SECURITY;  
ALTER TABLE IF EXISTS icu_fluid_balance DISABLE ROW LEVEL SECURITY;  
ALTER TABLE IF EXISTS icu_fluid_balance DROP CONSTRAINT IF EXISTS fk_icu_fluid_balance_tenant;  
DROP INDEX IF EXISTS idx_icu_fluid_balance_tenant_id;
```

```
COMMIT;
```

```
=====
```

FILE: migrations\e9\_01\_icu\_rls\_up.sql

```
=====
```

```
-- =====
```

```
-- e9_01_icu_rls_up.sql
```

```
-- E9 ICU / CRITICAL CARE ? FORCE RLS on the four EXISTING icu_* tables.
```

```
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
```

```
--
```

```
-- ?????: ??? E9 ?????? ? ?????? ?????? ?????? (icu_monitoring, icu_ventilator, icu_scores,  
-- icu_fluid_balance) ??? tenant_id (????? ?? bootstrap) ??? ??? ????? ??? FORCE RLS ? ??? ???  
-- ??????. ????? ??? ?????? ?????? (USING/WITH CHECK ??? app.tenant_id) + ??? tenant_id NOT NULL  
-- + FK -> tenants(id) + ???. ??? ?????? ?????? ?????? (?? ?????? ??? ??????) ??? ?????? ?? down.  
-- ?????: ?? ????? ?????? ?????? ??? bootstrap ?? db_postgres.js ??? ??? ?????? (???? ?????? ???).  
--
```

```
-- idempotent: ADD COLUMN IF NOT EXISTS + backfill + SET NOT NULL + DROP/ADD CONSTRAINT IF EXISTS  
-- + CREATE INDEX IF NOT EXISTS + DROP POLICY IF EXISTS. wrapped BEGIN; ? COMMIT;  
-- =====
```

```
BEGIN;
```

```
-- ===== icu_monitoring (ICU flowsheet) =====
```

```
ALTER TABLE icu_monitoring ADD COLUMN IF NOT EXISTS tenant_id INTEGER;  
UPDATE icu_monitoring SET tenant_id = 1 WHERE tenant_id IS NULL;  
ALTER TABLE icu_monitoring ALTER COLUMN tenant_id SET NOT NULL;  
ALTER TABLE icu_monitoring DROP CONSTRAINT IF EXISTS fk_icu_monitoring_tenant;  
ALTER TABLE icu_monitoring ADD CONSTRAINT fk_icu_monitoring_tenant  
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;  
CREATE INDEX IF NOT EXISTS idx_icu_monitoring_tenant_id ON icu_monitoring (tenant_id);  
ALTER TABLE icu_monitoring ENABLE ROW LEVEL SECURITY;  
ALTER TABLE icu_monitoring FORCE ROW LEVEL SECURITY;  
DROP POLICY IF EXISTS rls_icu_monitoring_tenant_isolation ON icu_monitoring;  
CREATE POLICY rls_icu_monitoring_tenant_isolation ON icu_monitoring  
    FOR ALL  
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
```

```

        WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== icu_ventilator =====
ALTER TABLE icu_ventilator ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE icu_ventilator SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE icu_ventilator ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE icu_ventilator DROP CONSTRAINT IF EXISTS fk_icu_ventilator_tenant;
ALTER TABLE icu_ventilator ADD CONSTRAINT fk_icu_ventilator_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_icu_ventilator_tenant_id ON icu_ventilator (tenant_id);
ALTER TABLE icu_ventilator ENABLE ROW LEVEL SECURITY;
ALTER TABLE icu_ventilator FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_icu_ventilator_tenant_isolation ON icu_ventilator;
CREATE POLICY rls_icu_ventilator_tenant_isolation ON icu_ventilator
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== icu_scores =====
ALTER TABLE icu_scores ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE icu_scores SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE icu_scores ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE icu_scores DROP CONSTRAINT IF EXISTS fk_icu_scores_tenant;
ALTER TABLE icu_scores ADD CONSTRAINT fk_icu_scores_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_icu_scores_tenant_id ON icu_scores (tenant_id);
ALTER TABLE icu_scores ENABLE ROW LEVEL SECURITY;
ALTER TABLE icu_scores FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_icu_scores_tenant_isolation ON icu_scores;
CREATE POLICY rls_icu_scores_tenant_isolation ON icu_scores
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== icu_fluid_balance =====
ALTER TABLE icu_fluid_balance ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE icu_fluid_balance SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE icu_fluid_balance ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE icu_fluid_balance DROP CONSTRAINT IF EXISTS fk_icu_fluid_balance_tenant;
ALTER TABLE icu_fluid_balance ADD CONSTRAINT fk_icu_fluid_balance_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_icu_fluid_balance_tenant_id ON icu_fluid_balance (tenant_id);
ALTER TABLE icu_fluid_balance ENABLE ROW LEVEL SECURITY;
ALTER TABLE icu_fluid_balance FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_icu_fluid_balance_tenant_isolation ON icu_fluid_balance;
CREATE POLICY rls_icu_fluid_balance_tenant_isolation ON icu_fluid_balance
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

=====

FILE: migrations\e9\_01\_icu\_rls\_validate.sql

=====

-- e9\_01\_icu\_rls\_validate.sql (run AFTER e9\_01\_icu\_rls\_up.sql; read-only)

-- PASS = each of the 4 icu\_\* tables has FORCE RLS + isolation policy + tenant\_id NOT NULL FK + index.

SELECT

```
-- icu_monitoring
(SELECT relforcerowsecurity FROM pg_class WHERE relname='icu_monitoring') AS mon_force_rls,
-- t
(SELECT count(*) FROM pg_policies WHERE tablename='icu_monitoring'
AND policyname='rls_icu_monitoring_tenant_isolation') AS mon_policy,
-- 1
(SELECT count(*)::int FROM information_schema.columns WHERE table_name='icu_monitoring'
AND column_name='tenant_id' AND is_nullable='NO') AS mon_tenant_nn,
-- 1
(SELECT count(*)::int FROM pg_constraint
WHERE conrelid='icu_monitoring'::regclass AND confrelid='tenants'::regclass AND contype='f') AS mon_tenant_fk,
-- 1
-- icu_ventilator
(SELECT relforcerowsecurity FROM pg_class WHERE relname='icu_ventilator') AS vent_force_rls,
-- t
(SELECT count(*) FROM pg_policies WHERE tablename='icu_ventilator'
AND policyname='rls_icu_ventilator_tenant_isolation') AS vent_policy,
-- 1
(SELECT count(*)::int FROM information_schema.columns WHERE table_name='icu_ventilator'
AND column_name='tenant_id' AND is_nullable='NO') AS vent_tenant_nn,
-- 1
(SELECT count(*)::int FROM pg_constraint
WHERE conrelid='icu_ventilator'::regclass AND confrelid='tenants'::regclass AND contype='f') AS vent_tenant_fk,
-- 1
-- icu_scores
(SELECT relforcerowsecurity FROM pg_class WHERE relname='icu_scores') AS sc_force_rls,
-- t
(SELECT count(*) FROM pg_policies WHERE tablename='icu_scores'
AND policyname='rls_icu_scores_tenant_isolation') AS sc_policy,
-- 1
(SELECT count(*)::int FROM information_schema.columns WHERE table_name='icu_scores'
AND column_name='tenant_id' AND is_nullable='NO') AS sc_tenant_nn,
-- 1
(SELECT count(*)::int FROM pg_constraint
WHERE conrelid='icu_scores'::regclass AND confrelid='tenants'::regclass AND contype='f') AS sc_tenant_fk,
-- 1
-- icu_fluid_balance
(SELECT relforcerowsecurity FROM pg_class WHERE relname='icu_fluid_balance') AS fb_force_rls,
-- t
(SELECT count(*) FROM pg_policies WHERE tablename='icu_fluid_balance'
AND policyname='rls_icu_fluid_balance_tenant_isolation') AS fb_policy,
-- 1
(SELECT count(*)::int FROM information_schema.columns WHERE table_name='icu_fluid_balance'
AND column_name='tenant_id' AND is_nullable='NO') AS fb_tenant_nn,
-- 1
(SELECT count(*)::int FROM pg_constraint
```

```

WHERE conrelid='icu_fluid_balance'::regclass AND confrelid='tenants'::regclass AND contype='f') AS fb_tenant_fk, -- 1
(SELECT count(*) FROM pg_indexes WHERE tablename='icu_fluid_balance'
AND indexname='idx_icu_fluid_balance_tenant_id') AS fb_id
x; -- 1

```

```
=====
```

```
FILE: migrations\e9_02_icu_infusions_down.sql
```

```
=====
```

```

-- e9_02_icu_infusions_down.sql (rollback of e9_02_icu_infusions_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ??????: icu_infusions ???? ???? ?????? ??? ?????? ? ???? ?????? ?????? ?????? (??? ?? ?????
-- ?????? ??????? ?????? ??????) ?????? ???? ?????? E9. idempotent (IF EXISTS).
BEGIN;

```

```

-- 1. drop the table first (cascades its policy/indexes/constraints).
DROP TABLE IF EXISTS icu_infusions CASCADE;

-- 2. defensive: drop the policy explicitly if the table somehow lingered.
DROP POLICY IF EXISTS rls_icu_infusions_tenant_isolation ON icu_infusions;

COMMIT;

```

```
=====
```

```
FILE: migrations\e9_02_icu_infusions_up.sql
```

```
=====
```

```

-- =====
-- e9_02_icu_infusions_up.sql
-- E9 ICU / CRITICAL CARE ? NEW table: icu_infusions (continuous IV drips / infusions).
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ???? ???? ?????? ?????????? ?????????? ?????????? (????????/????????/????????) ????????? ??????
-- ????? ?????? ?????. ??? ?????? ?????????? ?? E9 (?? ???? ?????? infusions/drips ??????). ??? ????
-- ????? ???? tenant_id INTEGER NOT NULL REFERENCES tenants(id) ??? ??????? + FK ???????/??????
-- + FORCE RLS ??????? ?????????? (USING/WITH CHECK ??? app.tenant_id).
-- ??????: ??? ?????? NOT ?????? ??? bootstrap ?? db_postgres.js (???? ?????? ???).
--
-- idempotent: CREATE TABLE IF NOT EXISTS + CREATE INDEX IF NOT EXISTS + DROP/ADD POLICY IF EXISTS.
-- =====
BEGIN;

```

```

CREATE TABLE IF NOT EXISTS icu_infusions (
    id SERIAL PRIMARY KEY,
    admission_id INTEGER NOT NULL REFERENCES admissions(id) ON DELETE CASCADE,
    patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    drug TEXT NOT NULL,
    concentration TEXT DEFAULT '',

```

```

rate            REAL DEFAULT 0,
rate_unit       TEXT DEFAULT 'mL/hr',
dose            REAL DEFAULT 0,
dose_unit       TEXT DEFAULT '',
route           TEXT DEFAULT 'IV',
status          TEXT DEFAULT 'Running',
allergy_warning TEXT,
notes           TEXT DEFAULT '',
recorded_by     TEXT DEFAULT '',
created_at      TIMESTAMP DEFAULT now(),
tenant_id       INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
facility_id     INTEGER,
CONSTRAINT chk_icu_infusions_status CHECK (status IN ('Running','Paused','Stopped','Completed'))
);

```

```

CREATE INDEX IF NOT EXISTS idx_icu_infusions_tenant_id ON icu_infusions (tenant_id);
CREATE INDEX IF NOT EXISTS idx_icu_infusions_admission ON icu_infusions (tenant_id, admission_id);

```

```

ALTER TABLE icu_infusions ENABLE ROW LEVEL SECURITY;
ALTER TABLE icu_infusions FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_icu_infusions_tenant_isolation ON icu_infusions;
CREATE POLICY rls_icu_infusions_tenant_isolation ON icu_infusions
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

```

=====
FILE: migrations\e9_02_icu_infusions_validate.sql
=====

```

```

-- e9_02_icu_infusions_validate.sql (run AFTER e9_02_icu_infusions_up.sql; read-only)
-- PASS = icu_infusions exists with tenant_id NOT NULL FK + admission/patient FK + FORCE RLS + policy + index.

```

```

SELECT
    (SELECT count(*)::int FROM information_schema.tables WHERE table_name='icu_infusions')           AS table_ex
ists,      -- expect 1
    (SELECT relforcerowsecurity FROM pg_class WHERE relname='icu_infusions')                   AS force_r
ls,      -- expect t
    (SELECT count(*) FROM pg_policies WHERE tablename='icu_infusions'
        AND policyname='rls_icu_infusions_tenant_isolation')                               AS policy,
        -- expect 1
    (SELECT count(*)::int FROM information_schema.columns WHERE table_name='icu_infusions'
        AND column_name='tenant_id' AND is_nullable='NO')                               AS tenant_
not_null, -- expect 1
    (SELECT count(*)::int FROM pg_constraint
        WHERE conrelid='icu_infusions'::regclass AND confrelid='tenants'::regclass AND contype='f') AS tenant_
fk,      -- expect 1
    (SELECT count(*)::int FROM pg_constraint
        WHERE conrelid='icu_infusions'::regclass AND confrelid='admissions'::regclass AND contype='f') AS admis
sion_fk,  -- expect 1

```

```

(SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='icu_infusions'::regclass AND confrelid='patients'::regclass AND contype='f') AS patient
_fk,      -- expect 1
(SELECT count(*) FROM pg_indexes WHERE tablename='icu_infusions'
 AND indexname='idx_icu_infusions_tenant_id')      AS idx;
      -- expect 1

```

```

=====
FILE: migrations\ex_01_orders_down.sql
=====

```

```

-- ex_01_orders_down.sql (rollback of ex_01_orders_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ???? order_items ?? orders ?? order_sets (????? ?????? ?????? FK) ?? ??????????. idempotent (IF EXISTS).
BEGIN;

```

```

DROP POLICY IF EXISTS rls_order_items_tenant_isolation ON order_items;
DROP TABLE IF EXISTS order_items;

```

```

DROP POLICY IF EXISTS rls_orders_tenant_isolation ON orders;
ALTER TABLE IF EXISTS orders DROP CONSTRAINT IF EXISTS chk_orders_status;
DROP TABLE IF EXISTS orders;

```

```

DROP POLICY IF EXISTS rls_order_sets_tenant_isolation ON order_sets;
DROP TABLE IF EXISTS order_sets;

```

```

COMMIT;

```

```

=====
FILE: migrations\ex_01_orders_up.sql
=====

```

```

-- =====
-- ex_01_orders_up.sql
-- E-X1 UNIFIED ORDERS ? foundational migration (1 of group EX).
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ????? ???? ????? ?????? (orders/order_items/order_sets) ???? ??? ??????? ?????????
-- ??????? ??????? (lab_radiology_orders / prescriptions + pharmacy_prescriptions_queue).
-- type ? (lab, rad, med, consult). encounter_id ???? ??? NULL ???? ?? ???? ???? encounters ???
-- (?????? visit_lifecycle)? ??? ??????? ?????? ??? patient_id ??? ???? ??????.
-- ?? ?????: tenant_id NOT NULL REFERENCES tenants(id) + ENABLE+FORCE RLS + ????? ??? tenant_id
-- (??? ???? ??? 150 ?????) + ???? tenant_id.
--

```

```

-- idempotent: CREATE TABLE IF NOT EXISTS + DROP POLICY IF EXISTS + CREATE INDEX IF NOT EXISTS.
-- =====

```

```

BEGIN;

```

```

-- ----- order_sets (reusable order templates; created BEFORE orders for FK target) -----
CREATE TABLE IF NOT EXISTS order_sets (

```

```

    id SERIAL PRIMARY KEY,
    tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    items_json TEXT NOT NULL DEFAULT '[]',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_order_sets_tenant_id ON order_sets (tenant_id);

ALTER TABLE order_sets ENABLE ROW LEVEL SECURITY;
ALTER TABLE order_sets FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_order_sets_tenant_isolation ON order_sets;
CREATE POLICY rls_order_sets_tenant_isolation ON order_sets
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ----- orders (unified order header) -----
CREATE TABLE IF NOT EXISTS orders (
    id SERIAL PRIMARY KEY,
    tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    facility_id INTEGER,
    encounter_id INTEGER,                -- nullable: no encounters table yet (E1/E2)
    patient_id INTEGER NOT NULL,
    type TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'pending',
    ordered_by INTEGER,
    order_set_id INTEGER REFERENCES order_sets(id) ON DELETE SET NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_orders_type CHECK (type IN ('lab', 'rad', 'med', 'consult')),
    CONSTRAINT chk_orders_status CHECK (status IN ('pending', 'active', 'completed', 'cancelled'))
);

-- idempotent: ensure the status CHECK exists even when the table pre-dates this migration
-- (CREATE TABLE IF NOT EXISTS skips the inline constraint on an existing table). DROP then ADD.
ALTER TABLE orders DROP CONSTRAINT IF EXISTS chk_orders_status;
ALTER TABLE orders ADD CONSTRAINT chk_orders_status CHECK (status IN ('pending', 'active', 'completed', 'cancelled'));

CREATE INDEX IF NOT EXISTS idx_orders_tenant_id ON orders (tenant_id);
CREATE INDEX IF NOT EXISTS idx_orders_patient_id ON orders (patient_id);
CREATE INDEX IF NOT EXISTS idx_orders_encounter_id ON orders (encounter_id);

ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE orders FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_orders_tenant_isolation ON orders;
CREATE POLICY rls_orders_tenant_isolation ON orders
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ----- order_items (line items of an order) -----
CREATE TABLE IF NOT EXISTS order_items (
    id SERIAL PRIMARY KEY,

```

```

tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
order_id INTEGER NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
catalog_ref TEXT,
qty INTEGER NOT NULL DEFAULT 1,
instructions TEXT,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_order_items_tenant_id ON order_items (tenant_id);
CREATE INDEX IF NOT EXISTS idx_order_items_order_id ON order_items (order_id);

ALTER TABLE order_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE order_items FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_order_items_tenant_isolation ON order_items;
CREATE POLICY rls_order_items_tenant_isolation ON order_items
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

```

=====
FILE: migrations\ex_01_orders_validate.sql
=====

```

```

-- ex_01_orders_validate.sql (run AFTER ex_01_orders_up.sql; read-only)
-- PASS = orders/order_items/order_sets exist with tenant_id FK + FORCE RLS + isolation policy + indexes + type CHECK.

-- ----- orders header -----
SELECT
    (SELECT count(*) FROM information_schema.tables WHERE table_name='orders') AS orders_exists,
    -- expect 1
    (SELECT count(*) FROM information_schema.columns
        WHERE table_name='orders'
        AND column_name IN ('tenant_id','encounter_id','patient_id','type','status','ordered_by','order_set_id'
    )) AS orders_cols, -- expect 7
    (SELECT relforcerowsecurity FROM pg_class WHERE relname='orders') AS orders_force_rls,
    -- expect t
    (SELECT count(*) FROM pg_policies WHERE tablename='orders' AND policyname='rls_orders_tenant_isolation') AS
orders_policy, -- expect 1
    (SELECT count(*) FROM pg_constraint WHERE conname='chk_orders_type') AS orders_type_check,
    -- expect 1
    (SELECT count(*) FROM pg_constraint WHERE conname='chk_orders_status') AS orders_status_check,
    -- expect 1
    (SELECT count(*) FROM pg_indexes WHERE tablename='orders' AND indexname='idx_orders_tenant_id') AS orders_id
x; -- expect 1

-- ----- order_items -----
SELECT
    (SELECT count(*) FROM information_schema.tables WHERE table_name='order_items') AS items_exists,
    -- expect 1

```

```

(SELECT count(*) FROM information_schema.columns
 WHERE table_name='order_items'
    AND column_name IN ('tenant_id','order_id','catalog_ref','qty','instructions')) AS items_cols,
-- expect 5
(SELECT relforcerowsecurity FROM pg_class WHERE relname='order_items') AS items_force_rls,
-- expect t
(SELECT count(*) FROM pg_policies WHERE tablename='order_items' AND policyname='rls_order_items_tenant_isola
tion') AS items_policy, -- expect 1
(SELECT count(*) FROM pg_indexes WHERE tablename='order_items' AND indexname='idx_order_items_tenant_id') AS
items_idx; -- expect 1

-- ----- order_sets -----
SELECT
  (SELECT count(*) FROM information_schema.tables WHERE table_name='order_sets') AS sets_exists,
  -- expect 1
  (SELECT count(*) FROM information_schema.columns
    WHERE table_name='order_sets'
    AND column_name IN ('tenant_id','name','items_json')) AS sets_cols,
  -- expect 3
  (SELECT relforcerowsecurity FROM pg_class WHERE relname='order_sets') AS sets_force_rls,
  -- expect t
  (SELECT count(*) FROM pg_policies WHERE tablename='order_sets' AND policyname='rls_order_sets_tenant_isolati
on') AS sets_policy, -- expect 1
  (SELECT count(*) FROM pg_indexes WHERE tablename='order_sets' AND indexname='idx_order_sets_tenant_id') AS s
ets_idx; -- expect 1

-- ----- FKs to tenants(id) on all three -----
SELECT count(*)::int AS fk_to_tenants FROM pg_constraint
 WHERE conrelid IN ('orders'::regclass,'order_items'::regclass,'order_sets'::regclass)
    AND confrelid='tenants'::regclass AND contype='f';
-- expect 3

-- ----- order_items -> orders FK + orders -> order_sets FK -----
SELECT
  (SELECT count(*) FROM pg_constraint
    WHERE conrelid='order_items'::regclass AND confrelid='orders'::regclass AND contype='f') AS items_to_orde
rs_fk, -- expect 1
  (SELECT count(*) FROM pg_constraint
    WHERE conrelid='orders'::regclass AND confrelid='order_sets'::regclass AND contype='f') AS orders_to_sets
_fk; -- expect 1

=====
FILE: migrations\ex_02_rbac_down.sql
=====

-- ex_02_rbac_down.sql (rollback of ex_02_rbac_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ??? role_permissions ?? permissions ?? ???????. idempotent (IF EXISTS).
BEGIN;

DROP POLICY IF EXISTS rls_role_permissions_tenant_isolation ON role_permissions;
DROP TABLE IF EXISTS role_permissions;

```

```
DROP TABLE IF EXISTS permissions;
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\ex_02_rbac_up.sql
```

```
=====
```

```
-- =====
```

```
-- ex_02_rbac_up.sql
```

```
-- E-X3 RBAC MATRIX ? foundational migration (2 of group EX).
```

```
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
```

```
--
```

```
-- ?????: ??? ?????? ??? ?????? ?????? ?????? L6 (??? system_users.permissions ????? + ???
```

```
-- user_permissions ??? ?????????). ?????:
```

```
-- permissions(id, key UNIQUE, description) -- ?????? ?????? ?????????? (???? ??? ?????? ?????????)
```

```
-- role_permissions(tenant_id, role, permission_key) -- ?????? ???->?????? ??? ?????? (FORCE RLS)
```

```
-- permissions ?????? ?????? (?? tenant_id) ? ?????? ??? ????????? ??? lab_tests_catalog ?????????.
```

```
-- role_permissions ?????? ??????????: tenant_id NOT NULL + ENABLE+FORCE RLS + ?????? ?????? ?????????.
```

```
-- middleware requirePermission(key) ?? rbac.js ??? role_permissions ?????? (fallback) ??? requireRole
```

```
-- ??? ????? ?? ?? ?????? (??? ?????). ?????? ?? seeds/ex_02_rbac_seed.sql (?? ?????? ???).
```

```
--
```

```
-- idempotent: CREATE TABLE IF NOT EXISTS + DROP POLICY IF EXISTS + CREATE INDEX IF NOT EXISTS.
```

```
-- =====
```

```
BEGIN;
```

```
-- ----- permissions (global catalog of permission keys) -----
```

```
CREATE TABLE IF NOT EXISTS permissions (
```

```
    id SERIAL PRIMARY KEY,
```

```
    key TEXT NOT NULL,
```

```
    description TEXT,
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    CONSTRAINT uq_permissions_key UNIQUE (key)
```

```
);
```

```
-- ----- role_permissions (per-tenant role -> permission_key matrix) -----
```

```
CREATE TABLE IF NOT EXISTS role_permissions (
```

```
    id SERIAL PRIMARY KEY,
```

```
    tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
```

```
    role TEXT NOT NULL,
```

```
    permission_key TEXT NOT NULL,
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    CONSTRAINT uq_role_permission UNIQUE (tenant_id, role, permission_key)
```

```
);
```

```
CREATE INDEX IF NOT EXISTS idx_role_permissions_tenant_id ON role_permissions (tenant_id);
```

```
CREATE INDEX IF NOT EXISTS idx_role_permissions_lookup ON role_permissions (tenant_id, role);
```

```
ALTER TABLE role_permissions ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE role_permissions FORCE ROW LEVEL SECURITY;
```

```
DROP POLICY IF EXISTS rls_role_permissions_tenant_isolation ON role_permissions;
```

```

CREATE POLICY rls_role_permissions_tenant_isolation ON role_permissions
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

```

=====
FILE: migrations\ex_02_rbac_validate.sql
=====

```

```

-- ex_02_rbac_validate.sql (run AFTER ex_02_rbac_up.sql; read-only)
-- PASS = permissions (global, UNIQUE key) + role_permissions (tenant_id FK + FORCE RLS + isolation policy + u
nique + index).

```

```

-- ----- permissions (global catalog) -----

```

```

SELECT
    (SELECT count(*) FROM information_schema.tables WHERE table_name='permissions') AS permissions_exists,
    -- expect 1
    (SELECT count(*) FROM information_schema.columns
        WHERE table_name='permissions' AND column_name IN ('key','description')) AS permissions_cols,
    -- expect 2
    (SELECT count(*) FROM pg_constraint WHERE conname='uq_permissions_key') AS permissions_key_unique;
    -- expect 1

```

```

-- ----- role_permissions (tenant-scoped matrix) -----

```

```

SELECT
    (SELECT count(*) FROM information_schema.tables WHERE table_name='role_permissions') AS rp_exists,
    -- expect 1
    (SELECT count(*) FROM information_schema.columns
        WHERE table_name='role_permissions' AND column_name IN ('tenant_id','role','permission_key')) AS rp_cols,
    -- expect 3
    (SELECT reforcerowsecurity FROM pg_class WHERE relname='role_permissions') AS rp_force_qls,
    -- expect t
    (SELECT count(*) FROM pg_policies
        WHERE tablename='role_permissions' AND policyname='rls_role_permissions_tenant_isolation') AS rp_policy,
    -- expect 1
    (SELECT count(*) FROM pg_constraint WHERE conname='uq_role_permission') AS rp_unique,
    -- expect 1
    (SELECT count(*) FROM pg_indexes
        WHERE tablename='role_permissions' AND indexname='idx_role_permissions_tenant_id') AS rp_idx;
    -- expect 1

```

```

-- ----- FK role_permissions -> tenants(id) -----

```

```

SELECT count(*)::int AS rp_fk_to_tenants FROM pg_constraint
    WHERE conrelid='role_permissions'::regclass AND confrelid='tenants'::regclass AND contype='f';
    -- expect 1

```

```

=====
FILE: migrations\ex_03_tenant_id_indexes_down.sql

```

=====

```
-- ex_03_tenant_id_indexes_down.sql (rollback of ex_03_tenant_id_indexes_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ???? ??????? ??? (?? ???? ??????/?????). DROP INDEX CONCURRENTLY ????? ???? ????????.
-- idempotent ??? IF EXISTS.
```

```
-- E-X net-new tables
```

```
DROP INDEX CONCURRENTLY IF EXISTS idx_orders_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_order_items_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_order_sets_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_role_permissions_tenant_id;
```

```
-- 88 prepared tables
```

```
DROP INDEX CONCURRENTLY IF EXISTS idx_admission_daily_rounds_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_branches_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_cme_activities_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_cme_events_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_cme_registrations_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_company_settings_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_cosmetic_cases_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_cosmetic_consent_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_cosmetic_followups_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_cosmetic_photos_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_cssd_batches_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_cssd_instrument_sets_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_cssd_load_items_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_cssd_sterilization_cycles_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_departments_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_discount_rules_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_doctor_inventory_request_items_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_doctor_inventory_requests_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_emergency_beds_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_emergency_trauma_assessments_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_employee_exposures_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_employees_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_facilities_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_finance_cost_centers_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_finance_doctor_commissions_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_finance_fiscal_years_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_finance_tax_declarations_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_finance_vouchers_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_form_templates_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_hand_hygiene_audits_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_hr_advances_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_hr_attendance_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_hr_employee_custody_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_hr_employee_documents_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_hr_leaves_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_hr_salaries_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_infection_control_reports_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_infection_outbreaks_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_infection_surveillance_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_insurance_companies_tenant_id;
```

```

DROP INDEX CONCURRENTLY IF EXISTS idx_insurance_contracts_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_insurance_policies_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_integration_settings_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_inventory_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_inventory_dept_request_items_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_inventory_dept_requests_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_inventory_issue_items_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_inventory_issue_to_dept_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_inventory_items_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_inventory_opening_balances_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_inventory_purchase_items_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_inventory_purchases_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_inventory_stock_count_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_invoices_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_maintenance_equipment_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_maintenance_orders_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_maintenance_pm_schedules_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_maintenance_work_orders_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_medical_reports_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_mortuary_cases_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_obgyn_deliveries_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_obgyn_pregnancies_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_online_bookings_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_operating_rooms_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_pathology_cases_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_pathology_specimens_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_patients_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_pharmacy_drug_catalog_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_pharmacy_opening_balances_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_pharmacy_prescriptions_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_pharmacy_purchase_items_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_pharmacy_purchase_orders_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_pharmacy_sale_items_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_pharmacy_sales_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_pharmacy_suppliers_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_portal_appointments_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_quality_incidents_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_quality_kpis_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_quality_patient_satisfaction_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_queue_advertisements_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_referrals_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_social_work_cases_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_surgery_anesthesia_records_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_surgery_preop_assessments_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_surgery_preop_tests_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_telemedicine_sessions_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_transport_requests_tenant_id;
DROP INDEX CONCURRENTLY IF EXISTS idx_visit_lifecycle_tenant_id;

```

```

=====
FILE: migrations\ex_03_tenant_id_indexes_up.sql
=====

```

```

-- =====
-- ex_03_tenant_id_indexes_up.sql
-- E-X2 TENANT_ID INDEXES ? foundational migration (3 of group EX).
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ????? ??? tenant_id ??? ??? FORCE-RLS ????? ?????? ?? ??? ?????? (?????? ~59/147).
--     ??? ??? ?????:
--     (?) 88 ?????? ??????/?????? ?????? ?????? ?? docs/sql/tenant_id_index_candidate_up.sql (??????).
--     (?) ????? E-X ??????: orders, order_items, order_sets, role_permissions.
--     (?????? ?????? ~59 ?????? ?????? ?????? ?????? (tenant_id, facility_id, ...) ?????? inline ?? db_postgres.j
s.)
--
-- ?? CONCURRENTLY ?? ??? ?????? ??? ??? ?????? (BEGIN/COMMIT) ? ?? ?????? ??? ?????? ????????.
-- ?? ????? idempotent ??? IF NOT EXISTS. ?? ??? CONCURRENTLY (????? ??? ?????) ??? INVALID ?? ?????
-- ?????? ?????? ??????? ?????? (???????????) ?? ?? ?????? ??? ?????? ?????? ????????.
-- =====

-- ----- E-X1 / E-X3 net-new tables (created by ex_01 / ex_02; idempotent here) -----
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_orders_tenant_id ON "orders" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_order_items_tenant_id ON "order_items" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_order_sets_tenant_id ON "order_sets" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_role_permissions_tenant_id ON "role_permissions" (tenant_id);

-- ----- 64 prepared FORCE-RLS tables still missing a tenant_id index -----
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_admission_daily_rounds_tenant_id ON "admission_daily_rounds" (tena
nt_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_cosmetic_cases_tenant_id ON "cosmetic_cases" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_cosmetic_consent_tenant_id ON "cosmetic_consent" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_cosmetic_followups_tenant_id ON "cosmetic_followups" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_cosmetic_photos_tenant_id ON "cosmetic_photos" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_doctor_inventory_request_items_tenant_id ON "doctor_inventory_requ
est_items" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_doctor_inventory_requests_tenant_id ON "doctor_inventory_requests"
(tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_emergency_beds_tenant_id ON "emergency_beds" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_emergency_trauma_assessments_tenant_id ON "emergency_trauma_assess
ments" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_employee_exposures_tenant_id ON "employee_exposures" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_facilities_tenant_id ON "facilities" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_finance_doctor_commissions_tenant_id ON "finance_doctor_commission
s" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_finance_tax_declarations_tenant_id ON "finance_tax_declarations" (
tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_finance_vouchers_tenant_id ON "finance_vouchers" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_hand_hygiene_audits_tenant_id ON "hand_hygiene_audits" (tenant_id)
;
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_hr_advances_tenant_id ON "hr_advances" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_hr_attendance_tenant_id ON "hr_attendance" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_hr_employee_custody_tenant_id ON "hr_employee_custody" (tenant_id)
;
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_hr_employee_documents_tenant_id ON "hr_employee_documents" (tenant
_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_hr_leaves_tenant_id ON "hr_leaves" (tenant_id);

```

```

CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_hr_salaries_tenant_id ON "hr_salaries" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_infection_outbreaks_tenant_id ON "infection_outbreaks" (tenant_id);
;
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_infection_surveillance_tenant_id ON "infection_surveillance" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_integration_settings_tenant_id ON "integration_settings" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_inventory_tenant_id ON "inventory" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_inventory_dept_request_items_tenant_id ON "inventory_dept_request_items" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_inventory_dept_requests_tenant_id ON "inventory_dept_requests" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_inventory_issue_items_tenant_id ON "inventory_issue_items" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_inventory_issue_to_dept_tenant_id ON "inventory_issue_to_dept" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_inventory_items_tenant_id ON "inventory_items" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_inventory_opening_balances_tenant_id ON "inventory_opening_balances" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_inventory_purchase_items_tenant_id ON "inventory_purchase_items" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_inventory_purchases_tenant_id ON "inventory_purchases" (tenant_id);
;
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_inventory_stock_count_tenant_id ON "inventory_stock_count" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_invoices_tenant_id ON "invoices" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_maintenance_equipment_tenant_id ON "maintenance_equipment" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_maintenance_pm_schedules_tenant_id ON "maintenance_pm_schedules" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_maintenance_work_orders_tenant_id ON "maintenance_work_orders" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_mortuary_cases_tenant_id ON "mortuary_cases" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_obgyn_deliveries_tenant_id ON "obgyn_deliveries" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_obgyn_pregnancies_tenant_id ON "obgyn_pregnancies" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_online_bookings_tenant_id ON "online_bookings" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_operating_rooms_tenant_id ON "operating_rooms" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_pathology_cases_tenant_id ON "pathology_cases" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_patients_tenant_id ON "patients" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_pharmacy_drug_catalog_tenant_id ON "pharmacy_drug_catalog" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_pharmacy_opening_balances_tenant_id ON "pharmacy_opening_balances" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_pharmacy_purchase_items_tenant_id ON "pharmacy_purchase_items" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_pharmacy_purchase_orders_tenant_id ON "pharmacy_purchase_orders" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_pharmacy_sale_items_tenant_id ON "pharmacy_sale_items" (tenant_id);
;
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_pharmacy_sales_tenant_id ON "pharmacy_sales" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_pharmacy_suppliers_tenant_id ON "pharmacy_suppliers" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_portal_appointments_tenant_id ON "portal_appointments" (tenant_id);
;
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_quality_incidents_tenant_id ON "quality_incidents" (tenant_id);

```

```

CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_quality_kpis_tenant_id ON "quality_kpis" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_quality_patient_satisfaction_tenant_id ON "quality_patient_satisfaction" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_queue_advertisements_tenant_id ON "queue_advertisements" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_social_work_cases_tenant_id ON "social_work_cases" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_surgery_anesthesia_records_tenant_id ON "surgery_anesthesia_records" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_surgery_preop_assessments_tenant_id ON "surgery_preop_assessments" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_surgery_preop_tests_tenant_id ON "surgery_preop_tests" (tenant_id);
;
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_telemedicine_sessions_tenant_id ON "telemedicine_sessions" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_transport_requests_tenant_id ON "transport_requests" (tenant_id);

-- ----- 25 missing prepared FORCE-RLS tables -----
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_branches_tenant_id ON "branches" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_cme_activities_tenant_id ON "cme_activities" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_cme_events_tenant_id ON "cme_events" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_cme_registrations_tenant_id ON "cme_registrations" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_company_settings_tenant_id ON "company_settings" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_cssd_batches_tenant_id ON "cssd_batches" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_cssd_instrument_sets_tenant_id ON "cssd_instrument_sets" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_cssd_load_items_tenant_id ON "cssd_load_items" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_cssd_sterilization_cycles_tenant_id ON "cssd_sterilization_cycles" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_departments_tenant_id ON "departments" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_discount_rules_tenant_id ON "discount_rules" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_employees_tenant_id ON "employees" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_finance_cost_centers_tenant_id ON "finance_cost_centers" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_finance_fiscal_years_tenant_id ON "finance_fiscal_years" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_form_templates_tenant_id ON "form_templates" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_infection_control_reports_tenant_id ON "infection_control_reports" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_insurance_companies_tenant_id ON "insurance_companies" (tenant_id);
;
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_insurance_contracts_tenant_id ON "insurance_contracts" (tenant_id);
;
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_insurance_policies_tenant_id ON "insurance_policies" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_maintenance_orders_tenant_id ON "maintenance_orders" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_medical_reports_tenant_id ON "medical_reports" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_pathology_specimens_tenant_id ON "pathology_specimens" (tenant_id);
;
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_pharmacy_prescriptions_tenant_id ON "pharmacy_prescriptions" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_referrals_tenant_id ON "referrals" (tenant_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_visit_lifecycle_tenant_id ON "visit_lifecycle" (tenant_id);

```

=====

FILE: migrations\ex\_03\_tenant\_id\_indexes\_validate.sql

=====

-- ex\_03\_tenant\_id\_indexes\_validate.sql (run AFTER ex\_03\_tenant\_id\_indexes\_up.sql; read-only)

-- PASS = every targeted tenant\_id index exists AND is valid (not a failed CONCURRENTLY remnant).

-- 1) total count of idx\_\*\_tenant\_id indexes created by this migration set (expect 68 = 64 prepared + 4 E-X)

SELECT count(\*)::int AS tenant\_id\_indexes\_present

FROM pg\_indexes

WHERE indexname LIKE 'idx\\_%\\_tenant\_id' ESCAPE '\'

AND indexname IN (

'idx\_orders\_tenant\_id','idx\_order\_items\_tenant\_id','idx\_order\_sets\_tenant\_id','idx\_role\_permissions\_tenant\_id',

'idx\_admission\_daily\_rounds\_tenant\_id','idx\_cosmetic\_cases\_tenant\_id','idx\_cosmetic\_consent\_tenant\_id',

'idx\_cosmetic\_followups\_tenant\_id','idx\_cosmetic\_photos\_tenant\_id','idx\_doctor\_inventory\_request\_items\_tenant\_id',

'idx\_doctor\_inventory\_requests\_tenant\_id','idx\_emergency\_beds\_tenant\_id','idx\_emergency\_trauma\_assessments\_tenant\_id',

'idx\_employee\_exposures\_tenant\_id','idx\_facilities\_tenant\_id','idx\_finance\_doctor\_commissions\_tenant\_id',

'idx\_finance\_tax\_declarations\_tenant\_id','idx\_finance\_vouchers\_tenant\_id','idx\_hand\_hygiene\_audits\_tenant\_id',

'idx\_hr\_advances\_tenant\_id','idx\_hr\_attendance\_tenant\_id','idx\_hr\_employee\_custody\_tenant\_id',

'idx\_hr\_employee\_documents\_tenant\_id','idx\_hr\_leaves\_tenant\_id','idx\_hr\_salaries\_tenant\_id',

'idx\_infection\_outbreaks\_tenant\_id','idx\_infection\_surveillance\_tenant\_id','idx\_integration\_settings\_tenant\_id',

'idx\_inventory\_tenant\_id','idx\_inventory\_dept\_request\_items\_tenant\_id','idx\_inventory\_dept\_requests\_tenant\_id',

'idx\_inventory\_issue\_items\_tenant\_id','idx\_inventory\_issue\_to\_dept\_tenant\_id','idx\_inventory\_items\_tenant\_id',

'idx\_inventory\_opening\_balances\_tenant\_id','idx\_inventory\_purchase\_items\_tenant\_id','idx\_inventory\_purchases\_tenant\_id',

'idx\_inventory\_stock\_count\_tenant\_id','idx\_invoices\_tenant\_id','idx\_maintenance\_equipment\_tenant\_id',

'idx\_maintenance\_pm\_schedules\_tenant\_id','idx\_maintenance\_work\_orders\_tenant\_id','idx\_mortuary\_cases\_tenant\_id',

'idx\_obgyn\_deliveries\_tenant\_id','idx\_obgyn\_pregnancies\_tenant\_id','idx\_online\_bookings\_tenant\_id',

'idx\_operating\_rooms\_tenant\_id','idx\_pathology\_cases\_tenant\_id','idx\_patients\_tenant\_id',

'idx\_pharmacy\_drug\_catalog\_tenant\_id','idx\_pharmacy\_opening\_balances\_tenant\_id','idx\_pharmacy\_purchase\_items\_tenant\_id',

'idx\_pharmacy\_purchase\_orders\_tenant\_id','idx\_pharmacy\_sale\_items\_tenant\_id','idx\_pharmacy\_sales\_tenant\_id',

'idx\_pharmacy\_suppliers\_tenant\_id','idx\_portal\_appointments\_tenant\_id','idx\_quality\_incidents\_tenant\_id',

'idx\_quality\_kpis\_tenant\_id','idx\_quality\_patient\_satisfaction\_tenant\_id','idx\_queue\_advertisements\_tenant\_id',

'idx\_social\_work\_cases\_tenant\_id','idx\_surgery\_anesthesia\_records\_tenant\_id','idx\_surgery\_preop\_assessment\_tenant\_id',

'idx\_surgery\_preop\_tests\_tenant\_id','idx\_telemedicine\_sessions\_tenant\_id','idx\_transport\_requests\_tenant\_id'

);

-- 2) any INVALID indexes left behind by a failed CONCURRENTLY build (expect 0)

SELECT count(\*)::int AS invalid\_tenant\_id\_indexes

FROM pg\_index i

JOIN pg\_class c ON c.oid = i.indexrelid

```
WHERE NOT i.indisvalid
AND c.relname LIKE 'idx\_%\_tenant_id' ESCAPE '\';
```

```
=====
FILE: migrations\p0_01_deferred_modules_qls_down.sql
=====
```

```
-- =====
-- p0_01_deferred_modules_qls_down.sql
-- Rollback for deferred modules RLS enablement.
-- =====

BEGIN;

-- 1. medical_records_files
DROP POLICY IF EXISTS rls_medical_records_files_tenant_isolation ON medical_records_files;
ALTER TABLE medical_records_files NO FORCE ROW LEVEL SECURITY;
ALTER TABLE medical_records_files DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_medical_records_files_tenant_id;
ALTER TABLE medical_records_files DROP CONSTRAINT IF EXISTS fk_medical_records_files_tenant;

-- 2. medical_records_requests
DROP POLICY IF EXISTS rls_medical_records_requests_tenant_isolation ON medical_records_requests;
ALTER TABLE medical_records_requests NO FORCE ROW LEVEL SECURITY;
ALTER TABLE medical_records_requests DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_medical_records_requests_tenant_id;
ALTER TABLE medical_records_requests DROP CONSTRAINT IF EXISTS fk_medical_records_requests_tenant;

-- 3. medical_records_coding
DROP POLICY IF EXISTS rls_medical_records_coding_tenant_isolation ON medical_records_coding;
ALTER TABLE medical_records_coding NO FORCE ROW LEVEL SECURITY;
ALTER TABLE medical_records_coding DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_medical_records_coding_tenant_id;
ALTER TABLE medical_records_coding DROP CONSTRAINT IF EXISTS fk_medical_records_coding_tenant;

-- 4. clinical_pharmacy_reviews
DROP POLICY IF EXISTS rls_clinical_pharmacy_reviews_tenant_isolation ON clinical_pharmacy_reviews;
ALTER TABLE clinical_pharmacy_reviews NO FORCE ROW LEVEL SECURITY;
ALTER TABLE clinical_pharmacy_reviews DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_clinical_pharmacy_reviews_tenant_id;
ALTER TABLE clinical_pharmacy_reviews DROP CONSTRAINT IF EXISTS fk_clinical_pharmacy_reviews_tenant;

-- 5. patient_drug_education
DROP POLICY IF EXISTS rls_patient_drug_education_tenant_isolation ON patient_drug_education;
ALTER TABLE patient_drug_education NO FORCE ROW LEVEL SECURITY;
ALTER TABLE patient_drug_education DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_patient_drug_education_tenant_id;
ALTER TABLE patient_drug_education DROP CONSTRAINT IF EXISTS fk_patient_drug_education_tenant;

-- 6. rehab_patients
DROP POLICY IF EXISTS rls_rehab_patients_tenant_isolation ON rehab_patients;
ALTER TABLE rehab_patients NO FORCE ROW LEVEL SECURITY;
ALTER TABLE rehab_patients DISABLE ROW LEVEL SECURITY;
```

```
DROP INDEX IF EXISTS idx_rehab_patients_tenant_id;
ALTER TABLE rehab_patients DROP CONSTRAINT IF EXISTS fk_rehab_patients_tenant;

-- 7. rehab_sessions
DROP POLICY IF EXISTS rls_rehab_sessions_tenant_isolation ON rehab_sessions;
ALTER TABLE rehab_sessions NO FORCE ROW LEVEL SECURITY;
ALTER TABLE rehab_sessions DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_rehab_sessions_tenant_id;
ALTER TABLE rehab_sessions DROP CONSTRAINT IF EXISTS fk_rehab_sessions_tenant;

-- 8. rehab_goals
DROP POLICY IF EXISTS rls_rehab_goals_tenant_isolation ON rehab_goals;
ALTER TABLE rehab_goals NO FORCE ROW LEVEL SECURITY;
ALTER TABLE rehab_goals DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_rehab_goals_tenant_id;
ALTER TABLE rehab_goals DROP CONSTRAINT IF EXISTS fk_rehab_goals_tenant;

-- 9. rehab_assessments
DROP POLICY IF EXISTS rls_rehab_assessments_tenant_isolation ON rehab_assessments;
ALTER TABLE rehab_assessments NO FORCE ROW LEVEL SECURITY;
ALTER TABLE rehab_assessments DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_rehab_assessments_tenant_id;
ALTER TABLE rehab_assessments DROP CONSTRAINT IF EXISTS fk_rehab_assessments_tenant;

-- 10. portal_users
DROP POLICY IF EXISTS rls_portal_users_tenant_isolation ON portal_users;
ALTER TABLE portal_users NO FORCE ROW LEVEL SECURITY;
ALTER TABLE portal_users DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_portal_users_tenant_id;
ALTER TABLE portal_users DROP CONSTRAINT IF EXISTS fk_portal_users_tenant;

-- 11. portal_appointments
DROP POLICY IF EXISTS rls_portal_appointments_tenant_isolation ON portal_appointments;
ALTER TABLE portal_appointments NO FORCE ROW LEVEL SECURITY;
ALTER TABLE portal_appointments DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_portal_appointments_tenant_id;
ALTER TABLE portal_appointments DROP CONSTRAINT IF EXISTS fk_portal_appointments_tenant;

-- 12. diet_orders
DROP POLICY IF EXISTS rls_diet_orders_tenant_isolation ON diet_orders;
ALTER TABLE diet_orders NO FORCE ROW LEVEL SECURITY;
ALTER TABLE diet_orders DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_diet_orders_tenant_id;
ALTER TABLE diet_orders DROP CONSTRAINT IF EXISTS fk_diet_orders_tenant;

-- 13. diet_meals
DROP POLICY IF EXISTS rls_diet_meals_tenant_isolation ON diet_meals;
ALTER TABLE diet_meals NO FORCE ROW LEVEL SECURITY;
ALTER TABLE diet_meals DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_diet_meals_tenant_id;
ALTER TABLE diet_meals DROP CONSTRAINT IF EXISTS fk_diet_meals_tenant;

-- 14. nutrition_assessments
DROP POLICY IF EXISTS rls_nutrition_assessments_tenant_isolation ON nutrition_assessments;
```

```

ALTER TABLE nutrition_assessments NO FORCE ROW LEVEL SECURITY;
ALTER TABLE nutrition_assessments DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_nutrition_assessments_tenant_id;
ALTER TABLE nutrition_assessments DROP CONSTRAINT IF EXISTS fk_nutrition_assessments_tenant;

-- 15. approvals
DROP POLICY IF EXISTS rls_approvals_tenant_isolation ON approvals;
ALTER TABLE approvals NO FORCE ROW LEVEL SECURITY;
ALTER TABLE approvals DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_approvals_tenant_id;
ALTER TABLE approvals DROP CONSTRAINT IF EXISTS fk_approvals_tenant;

-- 16. package_sessions
DROP POLICY IF EXISTS rls_package_sessions_tenant_isolation ON package_sessions;
ALTER TABLE package_sessions NO FORCE ROW LEVEL SECURITY;
ALTER TABLE package_sessions DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_package_sessions_tenant_id;
ALTER TABLE package_sessions DROP CONSTRAINT IF EXISTS fk_package_sessions_tenant;

-- 17. internal_messages
DROP POLICY IF EXISTS rls_internal_messages_tenant_isolation ON internal_messages;
ALTER TABLE internal_messages NO FORCE ROW LEVEL SECURITY;
ALTER TABLE internal_messages DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_internal_messages_tenant_id;
ALTER TABLE internal_messages DROP CONSTRAINT IF EXISTS fk_internal_messages_tenant;

COMMIT;

=====
FILE: migrations\p0_01_deferred_modules_rls_up.sql
=====

-- =====
-- p0_01_deferred_modules_rls_up.sql
-- PHASE-0 REMEDIATION ? FORCE RLS on deferred/modern modules:
--   medical_records_files, medical_records_requests, medical_records_coding,
--   clinical_pharmacy_reviews, patient_drug_education,
--   rehab_patients, rehab_sessions, rehab_goals, rehab_assessments,
--   portal_users, portal_appointments,
--   diet_orders, diet_meals, nutrition_assessments,
--   approvals, package_sessions, internal_messages.
-- =====
BEGIN;

-- Helper macro-like structure for each table

-- 1. medical_records_files
ALTER TABLE medical_records_files ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE medical_records_files SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE medical_records_files ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE medical_records_files DROP CONSTRAINT IF EXISTS fk_medical_records_files_tenant;
ALTER TABLE medical_records_files ADD CONSTRAINT fk_medical_records_files_tenant

```

```

FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_medical_records_files_tenant_id ON medical_records_files (tenant_id);
ALTER TABLE medical_records_files ENABLE ROW LEVEL SECURITY;
ALTER TABLE medical_records_files FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_medical_records_files_tenant_isolation ON medical_records_files;
CREATE POLICY rls_medical_records_files_tenant_isolation ON medical_records_files
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 2. medical_records_requests
ALTER TABLE medical_records_requests ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE medical_records_requests SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE medical_records_requests ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE medical_records_requests DROP CONSTRAINT IF EXISTS fk_medical_records_requests_tenant;
ALTER TABLE medical_records_requests ADD CONSTRAINT fk_medical_records_requests_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_medical_records_requests_tenant_id ON medical_records_requests (tenant_id);
ALTER TABLE medical_records_requests ENABLE ROW LEVEL SECURITY;
ALTER TABLE medical_records_requests FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_medical_records_requests_tenant_isolation ON medical_records_requests;
CREATE POLICY rls_medical_records_requests_tenant_isolation ON medical_records_requests
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 3. medical_records_coding
ALTER TABLE medical_records_coding ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE medical_records_coding SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE medical_records_coding ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE medical_records_coding DROP CONSTRAINT IF EXISTS fk_medical_records_coding_tenant;
ALTER TABLE medical_records_coding ADD CONSTRAINT fk_medical_records_coding_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_medical_records_coding_tenant_id ON medical_records_coding (tenant_id);
ALTER TABLE medical_records_coding ENABLE ROW LEVEL SECURITY;
ALTER TABLE medical_records_coding FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_medical_records_coding_tenant_isolation ON medical_records_coding;
CREATE POLICY rls_medical_records_coding_tenant_isolation ON medical_records_coding
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 4. clinical_pharmacy_reviews
ALTER TABLE clinical_pharmacy_reviews ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE clinical_pharmacy_reviews SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE clinical_pharmacy_reviews ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE clinical_pharmacy_reviews DROP CONSTRAINT IF EXISTS fk_clinical_pharmacy_reviews_tenant;
ALTER TABLE clinical_pharmacy_reviews ADD CONSTRAINT fk_clinical_pharmacy_reviews_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_clinical_pharmacy_reviews_tenant_id ON clinical_pharmacy_reviews (tenant_id);
ALTER TABLE clinical_pharmacy_reviews ENABLE ROW LEVEL SECURITY;
ALTER TABLE clinical_pharmacy_reviews FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_clinical_pharmacy_reviews_tenant_isolation ON clinical_pharmacy_reviews;
CREATE POLICY rls_clinical_pharmacy_reviews_tenant_isolation ON clinical_pharmacy_reviews

```

```

FOR ALL
USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 5. patient_drug_education
ALTER TABLE patient_drug_education ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE patient_drug_education SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE patient_drug_education ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE patient_drug_education DROP CONSTRAINT IF EXISTS fk_patient_drug_education_tenant;
ALTER TABLE patient_drug_education ADD CONSTRAINT fk_patient_drug_education_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_patient_drug_education_tenant_id ON patient_drug_education (tenant_id);
ALTER TABLE patient_drug_education ENABLE ROW LEVEL SECURITY;
ALTER TABLE patient_drug_education FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_patient_drug_education_tenant_isolation ON patient_drug_education;
CREATE POLICY rls_patient_drug_education_tenant_isolation ON patient_drug_education
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 6. rehab_patients
ALTER TABLE rehab_patients ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE rehab_patients SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE rehab_patients ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE rehab_patients DROP CONSTRAINT IF EXISTS fk_rehab_patients_tenant;
ALTER TABLE rehab_patients ADD CONSTRAINT fk_rehab_patients_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_rehab_patients_tenant_id ON rehab_patients (tenant_id);
ALTER TABLE rehab_patients ENABLE ROW LEVEL SECURITY;
ALTER TABLE rehab_patients FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_rehab_patients_tenant_isolation ON rehab_patients;
CREATE POLICY rls_rehab_patients_tenant_isolation ON rehab_patients
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 7. rehab_sessions
ALTER TABLE rehab_sessions ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE rehab_sessions SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE rehab_sessions ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE rehab_sessions DROP CONSTRAINT IF EXISTS fk_rehab_sessions_tenant;
ALTER TABLE rehab_sessions ADD CONSTRAINT fk_rehab_sessions_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_rehab_sessions_tenant_id ON rehab_sessions (tenant_id);
ALTER TABLE rehab_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE rehab_sessions FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_rehab_sessions_tenant_isolation ON rehab_sessions;
CREATE POLICY rls_rehab_sessions_tenant_isolation ON rehab_sessions
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 8. rehab_goals
ALTER TABLE rehab_goals ADD COLUMN IF NOT EXISTS tenant_id INTEGER;

```

```

UPDATE rehab_goals SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE rehab_goals ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE rehab_goals DROP CONSTRAINT IF EXISTS fk_rehab_goals_tenant;
ALTER TABLE rehab_goals ADD CONSTRAINT fk_rehab_goals_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_rehab_goals_tenant_id ON rehab_goals (tenant_id);
ALTER TABLE rehab_goals ENABLE ROW LEVEL SECURITY;
ALTER TABLE rehab_goals FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_rehab_goals_tenant_isolation ON rehab_goals;
CREATE POLICY rls_rehab_goals_tenant_isolation ON rehab_goals
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 9. rehab_assessments
ALTER TABLE rehab_assessments ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE rehab_assessments SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE rehab_assessments ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE rehab_assessments DROP CONSTRAINT IF EXISTS fk_rehab_assessments_tenant;
ALTER TABLE rehab_assessments ADD CONSTRAINT fk_rehab_assessments_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_rehab_assessments_tenant_id ON rehab_assessments (tenant_id);
ALTER TABLE rehab_assessments ENABLE ROW LEVEL SECURITY;
ALTER TABLE rehab_assessments FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_rehab_assessments_tenant_isolation ON rehab_assessments;
CREATE POLICY rls_rehab_assessments_tenant_isolation ON rehab_assessments
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 10. portal_users
ALTER TABLE portal_users ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE portal_users SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE portal_users ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE portal_users DROP CONSTRAINT IF EXISTS fk_portal_users_tenant;
ALTER TABLE portal_users ADD CONSTRAINT fk_portal_users_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_portal_users_tenant_id ON portal_users (tenant_id);
ALTER TABLE portal_users ENABLE ROW LEVEL SECURITY;
ALTER TABLE portal_users FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_portal_users_tenant_isolation ON portal_users;
CREATE POLICY rls_portal_users_tenant_isolation ON portal_users
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 11. portal_appointments
ALTER TABLE portal_appointments ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE portal_appointments SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE portal_appointments ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE portal_appointments DROP CONSTRAINT IF EXISTS fk_portal_appointments_tenant;
ALTER TABLE portal_appointments ADD CONSTRAINT fk_portal_appointments_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_portal_appointments_tenant_id ON portal_appointments (tenant_id);

```

```

ALTER TABLE portal_appointments ENABLE ROW LEVEL SECURITY;
ALTER TABLE portal_appointments FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_portal_appointments_tenant_isolation ON portal_appointments;
CREATE POLICY rls_portal_appointments_tenant_isolation ON portal_appointments
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 12. diet_orders
ALTER TABLE diet_orders ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE diet_orders SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE diet_orders ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE diet_orders DROP CONSTRAINT IF EXISTS fk_diet_orders_tenant;
ALTER TABLE diet_orders ADD CONSTRAINT fk_diet_orders_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_diet_orders_tenant_id ON diet_orders (tenant_id);
ALTER TABLE diet_orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE diet_orders FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_diet_orders_tenant_isolation ON diet_orders;
CREATE POLICY rls_diet_orders_tenant_isolation ON diet_orders
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 13. diet_meals
ALTER TABLE diet_meals ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE diet_meals SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE diet_meals ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE diet_meals DROP CONSTRAINT IF EXISTS fk_diet_meals_tenant;
ALTER TABLE diet_meals ADD CONSTRAINT fk_diet_meals_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_diet_meals_tenant_id ON diet_meals (tenant_id);
ALTER TABLE diet_meals ENABLE ROW LEVEL SECURITY;
ALTER TABLE diet_meals FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_diet_meals_tenant_isolation ON diet_meals;
CREATE POLICY rls_diet_meals_tenant_isolation ON diet_meals
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 14. nutrition_assessments
ALTER TABLE nutrition_assessments ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE nutrition_assessments SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE nutrition_assessments ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE nutrition_assessments DROP CONSTRAINT IF EXISTS fk_nutrition_assessments_tenant;
ALTER TABLE nutrition_assessments ADD CONSTRAINT fk_nutrition_assessments_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_nutrition_assessments_tenant_id ON nutrition_assessments (tenant_id);
ALTER TABLE nutrition_assessments ENABLE ROW LEVEL SECURITY;
ALTER TABLE nutrition_assessments FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_nutrition_assessments_tenant_isolation ON nutrition_assessments;
CREATE POLICY rls_nutrition_assessments_tenant_isolation ON nutrition_assessments
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)

```

```

        WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 15. approvals
ALTER TABLE approvals ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE approvals SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE approvals ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE approvals DROP CONSTRAINT IF EXISTS fk_approvals_tenant;
ALTER TABLE approvals ADD CONSTRAINT fk_approvals_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_approvals_tenant_id ON approvals (tenant_id);
ALTER TABLE approvals ENABLE ROW LEVEL SECURITY;
ALTER TABLE approvals FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_approvals_tenant_isolation ON approvals;
CREATE POLICY rls_approvals_tenant_isolation ON approvals
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 16. package_sessions
ALTER TABLE package_sessions ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE package_sessions SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE package_sessions ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE package_sessions DROP CONSTRAINT IF EXISTS fk_package_sessions_tenant;
ALTER TABLE package_sessions ADD CONSTRAINT fk_package_sessions_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_package_sessions_tenant_id ON package_sessions (tenant_id);
ALTER TABLE package_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE package_sessions FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_package_sessions_tenant_isolation ON package_sessions;
CREATE POLICY rls_package_sessions_tenant_isolation ON package_sessions
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 17. internal_messages
ALTER TABLE internal_messages ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE internal_messages SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE internal_messages ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE internal_messages DROP CONSTRAINT IF EXISTS fk_internal_messages_tenant;
ALTER TABLE internal_messages ADD CONSTRAINT fk_internal_messages_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_internal_messages_tenant_id ON internal_messages (tenant_id);
ALTER TABLE internal_messages ENABLE ROW LEVEL SECURITY;
ALTER TABLE internal_messages FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_internal_messages_tenant_isolation ON internal_messages;
CREATE POLICY rls_internal_messages_tenant_isolation ON internal_messages
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

=====

FILE: migrations\p0\_01\_deferred\_modules\_rls\_validate.sql

=====

-- p0\_01\_deferred\_modules\_rls\_validate.sql (run AFTER p0\_01\_deferred\_modules\_rls\_up.sql; read-only)

-- PASS = all 17 tables have tenant\_id NOT NULL FK + FORCE RLS + isolation policy.

SELECT

(SELECT relforcerowsecurity FROM pg\_class WHERE relname='medical\_records\_files') AS med\_files\_force\_rls,  
(SELECT count(\*) FROM pg\_policies WHERE tablename='medical\_records\_files' AND policyname='rls\_medical\_record  
s\_files\_tenant\_isolation') AS med\_files\_policy,

(SELECT relforcerowsecurity FROM pg\_class WHERE relname='medical\_records\_requests') AS med\_req\_force\_rls,  
(SELECT count(\*) FROM pg\_policies WHERE tablename='medical\_records\_requests' AND policyname='rls\_medical\_rec  
ords\_requests\_tenant\_isolation') AS med\_req\_policy,

(SELECT relforcerowsecurity FROM pg\_class WHERE relname='medical\_records\_coding') AS med\_coding\_force\_rls,  
(SELECT count(\*) FROM pg\_policies WHERE tablename='medical\_records\_coding' AND policyname='rls\_medical\_recor  
ds\_coding\_tenant\_isolation') AS med\_coding\_policy,

(SELECT relforcerowsecurity FROM pg\_class WHERE relname='clinical\_pharmacy\_reviews') AS rx\_reviews\_force\_rls  
,  
(SELECT count(\*) FROM pg\_policies WHERE tablename='clinical\_pharmacy\_reviews' AND policyname='rls\_clinical\_p  
harmacy\_reviews\_tenant\_isolation') AS rx\_reviews\_policy,

(SELECT relforcerowsecurity FROM pg\_class WHERE relname='patient\_drug\_education') AS rx\_edu\_force\_rls,  
(SELECT count(\*) FROM pg\_policies WHERE tablename='patient\_drug\_education' AND policyname='rls\_patient\_drug\_  
education\_tenant\_isolation') AS rx\_edu\_policy,

(SELECT relforcerowsecurity FROM pg\_class WHERE relname='rehab\_patients') AS rehab\_pat\_force\_rls,  
(SELECT count(\*) FROM pg\_policies WHERE tablename='rehab\_patients' AND policyname='rls\_rehab\_patients\_tenant\_  
isolation') AS rehab\_pat\_policy,

(SELECT relforcerowsecurity FROM pg\_class WHERE relname='rehab\_sessions') AS rehab\_sess\_force\_rls,  
(SELECT count(\*) FROM pg\_policies WHERE tablename='rehab\_sessions' AND policyname='rls\_rehab\_sessions\_tenant\_  
isolation') AS rehab\_sess\_policy,

(SELECT relforcerowsecurity FROM pg\_class WHERE relname='rehab\_goals') AS rehab\_goals\_force\_rls,  
(SELECT count(\*) FROM pg\_policies WHERE tablename='rehab\_goals' AND policyname='rls\_rehab\_goals\_tenant\_isola  
tion') AS rehab\_goals\_policy,

(SELECT relforcerowsecurity FROM pg\_class WHERE relname='rehab\_assessments') AS rehab\_ass\_force\_rls,  
(SELECT count(\*) FROM pg\_policies WHERE tablename='rehab\_assessments' AND policyname='rls\_rehab\_assessments\_  
tenant\_isolation') AS rehab\_ass\_policy,

(SELECT relforcerowsecurity FROM pg\_class WHERE relname='portal\_users') AS portal\_users\_force\_rls,  
(SELECT count(\*) FROM pg\_policies WHERE tablename='portal\_users' AND policyname='rls\_portal\_users\_tenant\_iso  
lation') AS portal\_users\_policy,

(SELECT relforcerowsecurity FROM pg\_class WHERE relname='portal\_appointments') AS portal\_app\_force\_rls,  
(SELECT count(\*) FROM pg\_policies WHERE tablename='portal\_appointments' AND policyname='rls\_portal\_appointme  
nts\_tenant\_isolation') AS portal\_app\_policy,

(SELECT relforcerowsecurity FROM pg\_class WHERE relname='diet\_orders') AS diet\_orders\_force\_rls,

```
(SELECT count(*) FROM pg_policies WHERE tablename='diet_orders' AND policyname='rls_diet_orders_tenant_isolation') AS diet_orders_policy,
```

```
(SELECT relforcerowsecurity FROM pg_class WHERE relname='diet_meals') AS diet_meals_force_rls,  
(SELECT count(*) FROM pg_policies WHERE tablename='diet_meals' AND policyname='rls_diet_meals_tenant_isolation') AS diet_meals_policy,
```

```
(SELECT relforcerowsecurity FROM pg_class WHERE relname='nutrition_assessments') AS nutrition_ass_force_rls,  
(SELECT count(*) FROM pg_policies WHERE tablename='nutrition_assessments' AND policyname='rls_nutrition_assessments_tenant_isolation') AS nutrition_ass_policy,
```

```
(SELECT relforcerowsecurity FROM pg_class WHERE relname='approvals') AS approvals_force_rls,  
(SELECT count(*) FROM pg_policies WHERE tablename='approvals' AND policyname='rls_approvals_tenant_isolation') AS approvals_policy,
```

```
(SELECT relforcerowsecurity FROM pg_class WHERE relname='package_sessions') AS pkg_sess_force_rls,  
(SELECT count(*) FROM pg_policies WHERE tablename='package_sessions' AND policyname='rls_package_sessions_tenant_isolation') AS pkg_sess_policy,
```

```
(SELECT relforcerowsecurity FROM pg_class WHERE relname='internal_messages') AS msg_force_rls,  
(SELECT count(*) FROM pg_policies WHERE tablename='internal_messages' AND policyname='rls_internal_messages_tenant_isolation') AS msg_policy;
```

```
=====  
FILE: migrations\p1_01_account_lockout_down.sql  
=====
```

```
BEGIN;  
  
ALTER TABLE system_users DROP COLUMN IF EXISTS failed_login_attempts;  
ALTER TABLE system_users DROP COLUMN IF EXISTS lockout_until;  
  
COMMIT;
```

```
=====  
FILE: migrations\p1_01_account_lockout_up.sql  
=====
```

```
BEGIN;  
  
-- Add failed_login_attempts column  
ALTER TABLE system_users ADD COLUMN IF NOT EXISTS failed_login_attempts INTEGER DEFAULT 0;  
  
-- Add lockout_until column  
ALTER TABLE system_users ADD COLUMN IF NOT EXISTS lockout_until TIMESTAMP WITH TIME ZONE DEFAULT NULL;  
  
COMMIT;
```

=====

FILE: migrations\p1\_01\_account\_lockout\_validate.sql

=====

```
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'system_users'
      AND column_name IN ('failed_login_attempts', 'lockout_until');
```

=====

FILE: migrations\p1\_01\_legacy\_core\_rls\_down.sql

=====

```
-- =====
-- p1_01_legacy_core_rls_down.sql (rollback of p1_01_legacy_core_rls_up.sql)
-- Disables FORCE RLS and drops the isolation policies on the four legacy core tables.
-- Keeps the tenant_id column + data + FK + index (non-destructive rollback of the POLICY only).
-- CANDIDATE ONLY ? DO NOT EXECUTE WITHOUT EXPLICIT DDL APPROVAL.
-- =====
```

BEGIN;

```
DROP POLICY IF EXISTS rls_patients_tenant_isolation ON patients;
ALTER TABLE patients NO FORCE ROW LEVEL SECURITY;
ALTER TABLE patients DISABLE ROW LEVEL SECURITY;
```

```
DROP POLICY IF EXISTS rls_invoices_tenant_isolation ON invoices;
ALTER TABLE invoices NO FORCE ROW LEVEL SECURITY;
ALTER TABLE invoices DISABLE ROW LEVEL SECURITY;
```

```
DROP POLICY IF EXISTS rls_appointments_tenant_isolation ON appointments;
ALTER TABLE appointments NO FORCE ROW LEVEL SECURITY;
ALTER TABLE appointments DISABLE ROW LEVEL SECURITY;
```

```
DROP POLICY IF EXISTS rls_medical_records_tenant_isolation ON medical_records;
ALTER TABLE medical_records NO FORCE ROW LEVEL SECURITY;
ALTER TABLE medical_records DISABLE ROW LEVEL SECURITY;
```

COMMIT;

=====

FILE: migrations\p1\_01\_legacy\_core\_rls\_up.sql

=====

```
-- =====
-- p1_01_legacy_core_rls_up.sql
-- PHASE-1 REMEDIATION ? FORCE RLS on the four legacy core PHI/financial tables that carry
--   tenant_id but had NO isolation policy: patients, invoices, appointments, medical_records.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ??? ??? ???? ???? ????? ? ??? ?????? ???????? ?????? ?????? ??? app-layer WHERE tenant_id=$n
```

```

-- ??? (idiom fail-open). ??? FORCE RLS (USING/WITH CHECK ??? app.tenant_id)
-- ??? ??? ??? ????. ??? tenant_id ??? app.tenant_id ???
-- ??? ?? ??? ???/??? ??? ?? ??? ??? ??? ??? 0 ??? (fail-closed).
--
-- idempotent: ADD COLUMN IF NOT EXISTS + backfill(tenant_id=1) + SET NOT NULL + FK + index +
-- DROP/CREATE POLICY. wrapped BEGIN; ? COMMIT;
-- ????: audit_trail ??? ??? ? ??? ??? ??? logAudit (??? tenant_id +
-- ??? ??? ??? ??? ??? ???) ??? RLS ???. ??? ?? ????.
-- =====
BEGIN;

-- ===== patients =====
ALTER TABLE patients ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE patients SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE patients ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE patients DROP CONSTRAINT IF EXISTS fk_patients_tenant;
ALTER TABLE patients ADD CONSTRAINT fk_patients_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_patients_tenant_id ON patients (tenant_id);
ALTER TABLE patients ENABLE ROW LEVEL SECURITY;
ALTER TABLE patients FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_patients_tenant_isolation ON patients;
CREATE POLICY rls_patients_tenant_isolation ON patients
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== invoices =====
ALTER TABLE invoices ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE invoices SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE invoices ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE invoices DROP CONSTRAINT IF EXISTS fk_invoices_tenant;
ALTER TABLE invoices ADD CONSTRAINT fk_invoices_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_invoices_tenant_id ON invoices (tenant_id);
ALTER TABLE invoices ENABLE ROW LEVEL SECURITY;
ALTER TABLE invoices FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_invoices_tenant_isolation ON invoices;
CREATE POLICY rls_invoices_tenant_isolation ON invoices
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== appointments =====
ALTER TABLE appointments ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE appointments SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE appointments ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE appointments DROP CONSTRAINT IF EXISTS fk_appointments_tenant;
ALTER TABLE appointments ADD CONSTRAINT fk_appointments_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_appointments_tenant_id ON appointments (tenant_id);
ALTER TABLE appointments ENABLE ROW LEVEL SECURITY;
ALTER TABLE appointments FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_appointments_tenant_isolation ON appointments;

```

```

CREATE POLICY rls_appointments_tenant_isolation ON appointments
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== medical_records =====
ALTER TABLE medical_records ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE medical_records SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE medical_records ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE medical_records DROP CONSTRAINT IF EXISTS fk_medical_records_tenant;
ALTER TABLE medical_records ADD CONSTRAINT fk_medical_records_tenant
    FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_medical_records_tenant_id ON medical_records (tenant_id);
ALTER TABLE medical_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE medical_records FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_medical_records_tenant_isolation ON medical_records;
CREATE POLICY rls_medical_records_tenant_isolation ON medical_records
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

=====
FILE: migrations\p1_01_legacy_core_qls_validate.sql
=====

-- p1_01_legacy_core_qls_validate.sql (run AFTER p1_01_legacy_core_qls_up.sql; read-only)
-- PASS = each of the four legacy core tables has tenant_id NOT NULL + FK -> tenants + FORCE RLS + policy.
SELECT
    (SELECT relforcerowsecurity FROM pg_class WHERE relname='patients') AS patients_force_qls,
    -- expect t
    (SELECT count(*) FROM pg_policies WHERE tablename='patients'
    AND policyname='qls_patients_tenant_isolation') AS patients_policy,
    -- expect 1
    (SELECT count(*)::int FROM information_schema.columns WHERE table_name='patients'
    AND column_name='tenant_id' AND is_nullable='NO') AS patients_tenant_not_n
ull, -- expect 1
    (SELECT count(*)::int FROM pg_constraint
    WHERE conrelid='patients'::regclass AND confrelid='tenants'::regclass AND contype='f') AS patients_fk,
    -- expect 1

    (SELECT relforcerowsecurity FROM pg_class WHERE relname='invoices') AS invoices_force_qls,
    -- expect t
    (SELECT count(*) FROM pg_policies WHERE tablename='invoices'
    AND policyname='qls_invoices_tenant_isolation') AS invoices_policy,
    -- expect 1
    (SELECT count(*)::int FROM information_schema.columns WHERE table_name='invoices'
    AND column_name='tenant_id' AND is_nullable='NO') AS invoices_tenant_not_n
ull, -- expect 1

    (SELECT relforcerowsecurity FROM pg_class WHERE relname='appointments') AS appointments_force_ql

```

```

s,      -- expect t
(SELECT count(*) FROM pg_policies WHERE tablename='appointments'
      AND policyname='rls_appointments_tenant_isolation')
      AS appointments_policy,
      -- expect 1

(SELECT relforcerowsecurity FROM pg_class WHERE relname='medical_records')
      AS medical_records_force
_rls,  -- expect t
(SELECT count(*) FROM pg_policies WHERE tablename='medical_records'
      AND policyname='rls_medical_records_tenant_isolation')
      AS medical_records_policy;
-- expect 1

```

=====

FILE: migrations\p1\_02\_gl\_posting\_idempotency\_down.sql

=====

```

-- =====
-- p1_02_gl_posting_idempotency_down.sql (rollback of p1_02_gl_posting_idempotency_up.sql)
-- Drops the partial unique index and the idempotency_key column.
-- CANDIDATE ONLY ? DO NOT EXECUTE WITHOUT EXPLICIT DDL APPROVAL.
-- =====
BEGIN;

```

```

DROP INDEX IF EXISTS uq_fje_tenant_idempotency;
ALTER TABLE finance_journal_entries DROP COLUMN IF EXISTS idempotency_key;

COMMIT;

```

=====

FILE: migrations\p1\_02\_gl\_posting\_idempotency\_up.sql

=====

```

-- =====
-- p1_02_gl_posting_idempotency_up.sql
-- PHASE-1 REMEDIATION ? GL posting idempotency backstop for finance_journal_entries.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ??? ???? (double-posting) ??? ????? ??????? ??????? ????????.
-- finance_engine.idempotencyKey() ????? ??? ?????? ??? ????? ??? ????? ??? ?????
-- ????? ??????. ??? ????? idempotency_key + ????? ????? ??? (tenant_id, idempotency_key)
-- ????? ????? ?????? ?????? ?????? ?????? ?????? ?????? (idempotency_key IS NULL).
-- ?????? ?????? (????? hash ?? source_type|source_id|event_type|tenant) ?????? ??? ??????.
-- idempotent: ADD COLUMN IF NOT EXISTS + CREATE UNIQUE INDEX IF NOT EXISTS (partial).
-- =====
BEGIN;

```

```

ALTER TABLE finance_journal_entries ADD COLUMN IF NOT EXISTS idempotency_key TEXT;

```

```

-- Partial UNIQUE: blocks a duplicate (tenant, idempotency_key) post; pre-existing NULL keys are exempt.
CREATE UNIQUE INDEX IF NOT EXISTS uq_fje_tenant_idempotency
  ON finance_journal_entries (tenant_id, idempotency_key)

```

```

WHERE idempotency_key IS NOT NULL;

COMMIT;

=====
FILE: migrations\p1_02_gl_posting_idempotency_validate.sql
=====

-- p1_02_gl_posting_idempotency_validate.sql (run AFTER p1_02_gl_posting_idempotency_up.sql; read-only)
-- PASS = idempotency_key column exists AND the partial unique index exists.
SELECT
    (SELECT count(*)::int FROM information_schema.columns
     WHERE table_name='finance_journal_entries' AND column_name='idempotency_key') AS idempotency_col, -
- expect 1
    (SELECT count(*)::int FROM pg_indexes
     WHERE tablename='finance_journal_entries' AND indexname='uq_fje_tenant_idempotency') AS unique_index; -
- expect 1

=====
FILE: migrations\p1_03_department_owners_down.sql
=====

-- p1_03_department_owners_down.sql
-- Rollback: Remove owner_role column from clinical_departments table
ALTER TABLE clinical_departments DROP COLUMN IF EXISTS owner_role;

=====
FILE: migrations\p1_03_department_owners_up.sql
=====

-- p1_03_department_owners_up.sql
-- Step 1: Add owner_role column to clinical_departments table with a default of 'CMO'
ALTER TABLE clinical_departments ADD COLUMN IF NOT EXISTS owner_role VARCHAR(50) NOT NULL DEFAULT 'CMO';

-- Step 2: Update existing clinical departments to their default executive owners
-- CMO: Clinical & Medical (e.g. Cardiology, Pediatrics, Internal Medicine, OB/GYN, Pathology, ICU)
-- CNO: Nursing & Inpatient (e.g. Wards, Nursing triage, ADT)
-- COO: Operations & Auxiliary (e.g. Patient Transport, CSSD, Emergency beds)
-- CFO: Finance & Billing (e.g. Invoices, Accounts, Reception/Cashier)
-- CIO: IT & Systems (e.g. Settings, Admin, Clinical templates)

UPDATE clinical_departments
SET owner_role = 'CNO'
WHERE code IN ('WARDS', 'NURSING', 'INPATIENT', 'ADT', 'MATERNITY', 'OB_GYN');

UPDATE clinical_departments
SET owner_role = 'COO'
WHERE code IN ('TRANSPORT', 'CSSD', 'EMERGENCY', 'OR', 'SURGERY', 'ANESTHESIA');

```

```

UPDATE clinical_departments
SET owner_role = 'CFO'
WHERE code IN ('FINANCE', 'BILLING', 'RECEPTION', 'ACCOUNTS', 'PHARMACY');

```

```

UPDATE clinical_departments
SET owner_role = 'CIO'
WHERE code IN ('IT', 'SETTINGS', 'ADMIN');

```

```

=====
FILE: migrations\p1_03_department_owners_validate.sql
=====

```

```

-- p1_03_department_owners_validate.sql
-- Validation query for department owner matrix.
-- PASS = owner_role column exists in clinical_departments and has non-empty values.

```

```

SELECT
  (SELECT count(*) FROM information_schema.columns
   WHERE table_name='clinical_departments' AND column_name='owner_role') AS owner_role_col_exists, -- expect
1
  (SELECT count(*) FROM clinical_departments WHERE owner_role IS NULL OR owner_role = '') AS unmapped_owners_c
ount; -- expect 0

```

```

=====
FILE: migrations\p1_04_pricing_overrides_down.sql
=====

```

```

-- =====
-- p1_04_pricing_overrides_down.sql
-- Drops the pricing overrides tables.
-- =====

```

```

DROP TABLE IF EXISTS tenant_service_overrides CASCADE;
DROP TABLE IF EXISTS tenant_lab_test_overrides CASCADE;
DROP TABLE IF EXISTS tenant_radiology_overrides CASCADE;

```

```

=====
FILE: migrations\p1_04_pricing_overrides_up.sql
=====

```

```

-- =====
-- p1_04_pricing_overrides_up.sql
-- Creates tenant-specific pricing overrides tables with RLS enabled.
-- =====

```

```

-- Medical Services Overrides
CREATE TABLE IF NOT EXISTS tenant_service_overrides (

```

```

    id SERIAL PRIMARY KEY,
    tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    service_id INTEGER NOT NULL REFERENCES medical_services(id) ON DELETE CASCADE,
    custom_price REAL NOT NULL,
    is_active INTEGER DEFAULT 1,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_tenant_service UNIQUE (tenant_id, service_id)
);

CREATE INDEX IF NOT EXISTS idx_tenant_svc_overrides ON tenant_service_overrides (tenant_id, service_id);
ALTER TABLE tenant_service_overrides ENABLE ROW LEVEL SECURITY;
ALTER TABLE tenant_service_overrides FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_tenant_service_overrides_tenant_isolation ON tenant_service_overrides;
CREATE POLICY rls_tenant_service_overrides_tenant_isolation ON tenant_service_overrides
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Lab Tests Overrides
CREATE TABLE IF NOT EXISTS tenant_lab_test_overrides (
    id SERIAL PRIMARY KEY,
    tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    test_id INTEGER NOT NULL REFERENCES lab_tests_catalog(id) ON DELETE CASCADE,
    custom_price REAL NOT NULL,
    is_active INTEGER DEFAULT 1,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_tenant_lab_test UNIQUE (tenant_id, test_id)
);

CREATE INDEX IF NOT EXISTS idx_tenant_lab_overrides ON tenant_lab_test_overrides (tenant_id, test_id);
ALTER TABLE tenant_lab_test_overrides ENABLE ROW LEVEL SECURITY;
ALTER TABLE tenant_lab_test_overrides FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_tenant_lab_test_overrides_tenant_isolation ON tenant_lab_test_overrides;
CREATE POLICY rls_tenant_lab_test_overrides_tenant_isolation ON tenant_lab_test_overrides
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Radiology Overrides
CREATE TABLE IF NOT EXISTS tenant_radiology_overrides (
    id SERIAL PRIMARY KEY,
    tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    radiology_id INTEGER NOT NULL REFERENCES radiology_catalog(id) ON DELETE CASCADE,
    custom_price REAL NOT NULL,
    custom_template TEXT,
    is_active INTEGER DEFAULT 1,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_tenant_radiology UNIQUE (tenant_id, radiology_id)
);

CREATE INDEX IF NOT EXISTS idx_tenant_rad_overrides ON tenant_radiology_overrides (tenant_id, radiology_id);
ALTER TABLE tenant_radiology_overrides ENABLE ROW LEVEL SECURITY;
ALTER TABLE tenant_radiology_overrides FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_tenant_radiology_overrides_tenant_isolation ON tenant_radiology_overrides;

```

```

CREATE POLICY rls_tenant_radiology_overrides_tenant_isolation ON tenant_radiology_overrides
    FOR ALL
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Grant privileges
GRANT ALL PRIVILEGES ON TABLE tenant_service_overrides TO nama_medical_app;
GRANT ALL PRIVILEGES ON TABLE tenant_lab_test_overrides TO nama_medical_app;
GRANT ALL PRIVILEGES ON TABLE tenant_radiology_overrides TO nama_medical_app;
GRANT USAGE, SELECT ON SEQUENCE tenant_service_overrides_id_seq TO nama_medical_app;
GRANT USAGE, SELECT ON SEQUENCE tenant_lab_test_overrides_id_seq TO nama_medical_app;
GRANT USAGE, SELECT ON SEQUENCE tenant_radiology_overrides_id_seq TO nama_medical_app;

=====
FILE: migrations\p1_04_pricing_overrides_validate.sql
=====

-- =====
-- p1_04_pricing_overrides_validate.sql
-- Validates existence of pricing overrides tables.
-- =====

SELECT (
    SELECT EXISTS (
        SELECT FROM information_schema.tables
        WHERE table_name = 'tenant_service_overrides'
    ) AND EXISTS (
        SELECT FROM information_schema.tables
        WHERE table_name = 'tenant_lab_test_overrides'
    ) AND EXISTS (
        SELECT FROM information_schema.tables
        WHERE table_name = 'tenant_radiology_overrides'
    )
) AS all_ok;

=====
FILE: migrations\p1_05_waiting_queue_acuity_down.sql
=====

-- =====
-- P1 PHASE waiting_queue ACUITY ? DOWN
-- Reverses modifications to waiting_queue and drops visit_lifecycle.
-- =====

BEGIN;

-- 1. Drop visit_lifecycle policy and table
DROP POLICY IF EXISTS rls_visit_lifecycle_tenant_isolation ON visit_lifecycle;
DROP TABLE IF EXISTS visit_lifecycle CASCADE;

-- 2. Drop policy and constraints on waiting_queue

```

```
DROP POLICY IF EXISTS rls_waiting_queue_tenant_isolation ON waiting_queue;
ALTER TABLE waiting_queue DROP CONSTRAINT IF EXISTS chk_waiting_queue_status;
ALTER TABLE waiting_queue DROP CONSTRAINT IF EXISTS fk_waiting_queue_tenant;
```

```
-- 3. Drop newly added columns
```

```
ALTER TABLE waiting_queue DROP COLUMN IF EXISTS triage_level;
ALTER TABLE waiting_queue DROP COLUMN IF EXISTS acuity_notes;
ALTER TABLE waiting_queue DROP COLUMN IF EXISTS exam_room_id;
ALTER TABLE waiting_queue DROP COLUMN IF EXISTS updated_at;
```

```
-- 4. Reset status column default
```

```
ALTER TABLE waiting_queue ALTER COLUMN status SET DEFAULT 'Waiting';
UPDATE waiting_queue SET status = 'Waiting' WHERE status = 'CheckedIn';
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\p1_05_waiting_queue_acuity_up.sql
```

```
=====
```

```
-- =====
```

```
-- P1 PHASE waiting_queue ACUITY ? UP
```

```
-- Updates waiting_queue with triage levels and provisions visit_lifecycle table.
```

```
-- =====
```

```
BEGIN;
```

```
-- 1. Create visit_lifecycle table if not exists (out-of-band schema provision)
```

```
CREATE TABLE IF NOT EXISTS visit_lifecycle (
    id SERIAL PRIMARY KEY,
    tenant_id INTEGER NOT NULL DEFAULT 1,
    patient_id INTEGER,
    patient_name TEXT DEFAULT '',
    appointment_id INTEGER,
    doctor TEXT DEFAULT '',
    department TEXT DEFAULT '',
    status TEXT DEFAULT 'arrived',
    stage TEXT DEFAULT 'arrived',
    arrived_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    triage_at TIMESTAMP,
    triage_level INTEGER,
    pain_score INTEGER,
    consult_start TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 2. Enable RLS and isolating policies for visit_lifecycle
```

```
ALTER TABLE visit_lifecycle ENABLE ROW LEVEL SECURITY;
ALTER TABLE visit_lifecycle FORCE ROW LEVEL SECURITY;
```

```
DROP POLICY IF EXISTS rls_visit_lifecycle_tenant_isolation ON visit_lifecycle;
CREATE POLICY rls_visit_lifecycle_tenant_isolation ON visit_lifecycle
    FOR ALL
```

```

TO PUBLIC
USING (tenant_id = current_setting('app.tenant_id', true)::integer);

-- 3. Add new columns to waiting_queue if they do not exist
ALTER TABLE waiting_queue ADD COLUMN IF NOT EXISTS triage_level INTEGER DEFAULT 5;
ALTER TABLE waiting_queue ADD COLUMN IF NOT EXISTS acuity_notes TEXT DEFAULT '';
ALTER TABLE waiting_queue ADD COLUMN IF NOT EXISTS exam_room_id VARCHAR(50) DEFAULT '';
ALTER TABLE waiting_queue ADD COLUMN IF NOT EXISTS updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;

-- 4. Update default status and convert legacy statuses
ALTER TABLE waiting_queue ALTER COLUMN status SET DEFAULT 'CheckedIn';
UPDATE waiting_queue SET status = 'CheckedIn' WHERE status = 'Waiting';

-- 5. Add CHECK constraint on status (drop if exists first)
ALTER TABLE waiting_queue DROP CONSTRAINT IF EXISTS chk_waiting_queue_status;
ALTER TABLE waiting_queue ADD CONSTRAINT chk_waiting_queue_status
    CHECK (status IN ('CheckedIn', 'Triage', 'WaitingForProvider', 'InConsultation', 'WaitingForResults', 'ReadyForDischarge', 'NoShow'));

-- 6. Ensure tenant_id exists and is NOT NULL
ALTER TABLE waiting_queue ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE waiting_queue SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE waiting_queue ALTER COLUMN tenant_id SET NOT NULL;

-- 7. Add tenant_id FK to tenants if not exists
DO $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM information_schema.table_constraints tc
        JOIN information_schema.key_column_usage kcu ON tc.constraint_name = kcu.constraint_name
        WHERE tc.constraint_type = 'FOREIGN KEY' AND tc.table_name = 'waiting_queue' AND kcu.column_name = 'tenant_id'
    ) THEN
        ALTER TABLE waiting_queue ADD CONSTRAINT fk_waiting_queue_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
    END IF;
END $$;

-- 8. Enable Row Level Security (RLS) and Force it
ALTER TABLE waiting_queue ENABLE ROW LEVEL SECURITY;
ALTER TABLE waiting_queue FORCE ROW LEVEL SECURITY;

-- 9. Create or replace canonical tenant isolation policy
DROP POLICY IF EXISTS rls_waiting_queue_tenant_isolation ON waiting_queue;
CREATE POLICY rls_waiting_queue_tenant_isolation ON waiting_queue
    FOR ALL
    TO PUBLIC
    USING (tenant_id = current_setting('app.tenant_id', true)::integer);

COMMIT;

```

```
=====
```

FILE: migrations\p1\_05\_waiting\_queue\_acuity\_validate.sql

=====

-- =====

-- P1 PHASE waiting\_queue ACUITY ? VALIDATE

-- Asserts that waiting\_queue and visit\_lifecycle have all new features, policies, constraints, and RLS.

-- =====

DO \$\$

BEGIN

-- 1. Assert visit\_lifecycle table and RLS exist

IF NOT EXISTS (SELECT 1 FROM information\_schema.tables WHERE table\_name = 'visit\_lifecycle') THEN

RAISE EXCEPTION 'P1 Validate Error: visit\_lifecycle table is missing';

END IF;

IF NOT EXISTS (

SELECT 1 FROM pg\_class c JOIN pg\_namespace n ON n.oid = c.relnamespace

WHERE c.relname = 'visit\_lifecycle' AND n.nspname = 'public'

AND c.relrowsecurity = TRUE AND c.relforcerowsecurity = TRUE

) THEN

RAISE EXCEPTION 'P1 Validate Error: RLS or FORCE RLS not enabled on visit\_lifecycle';

END IF;

IF NOT EXISTS (

SELECT 1 FROM pg\_policies

WHERE tablename = 'visit\_lifecycle' AND policyname = 'rls\_visit\_lifecycle\_tenant\_isolation'

) THEN

RAISE EXCEPTION 'P1 Validate Error: rls\_visit\_lifecycle\_tenant\_isolation policy missing on visit\_lifecycle';

END IF;

-- 2. Assert waiting\_queue columns exist

IF NOT EXISTS (SELECT 1 FROM information\_schema.columns WHERE table\_name = 'waiting\_queue' AND column\_name = 'triage\_level') THEN

RAISE EXCEPTION 'P1 Validate Error: waiting\_queue.triage\_level is missing';

END IF;

IF NOT EXISTS (SELECT 1 FROM information\_schema.columns WHERE table\_name = 'waiting\_queue' AND column\_name = 'acuity\_notes') THEN

RAISE EXCEPTION 'P1 Validate Error: waiting\_queue.acuity\_notes is missing';

END IF;

IF NOT EXISTS (SELECT 1 FROM information\_schema.columns WHERE table\_name = 'waiting\_queue' AND column\_name = 'exam\_room\_id') THEN

RAISE EXCEPTION 'P1 Validate Error: waiting\_queue.exam\_room\_id is missing';

END IF;

-- 3. Assert RLS is enabled and forced on waiting\_queue

IF NOT EXISTS (

SELECT 1 FROM pg\_class c JOIN pg\_namespace n ON n.oid = c.relnamespace

WHERE c.relname = 'waiting\_queue' AND n.nspname = 'public'

AND c.relrowsecurity = TRUE AND c.relforcerowsecurity = TRUE

) THEN

RAISE EXCEPTION 'P1 Validate Error: RLS or FORCE RLS not enabled on waiting\_queue';

END IF;

```

-- 4. Assert canonical tenant isolation policy exists
IF NOT EXISTS (
    SELECT 1 FROM pg_policies
    WHERE tablename = 'waiting_queue' AND policyname = 'rls_waiting_queue_tenant_isolation'
) THEN
    RAISE EXCEPTION 'P1 Validate Error: rls_waiting_queue_tenant_isolation policy missing on waiting_queue
';
END IF;

-- 5. Assert check constraint exists
IF NOT EXISTS (
    SELECT 1 FROM information_schema.table_constraints
    WHERE table_name = 'waiting_queue' AND constraint_name = 'chk_waiting_queue_status' AND constraint_type = 'CHECK'
) THEN
    RAISE EXCEPTION 'P1 Validate Error: chk_waiting_queue_status constraint missing on waiting_queue';
END IF;

-- 6. Assert tenant_id FK to tenants exists
IF NOT EXISTS (
    SELECT 1 FROM information_schema.table_constraints tc
    JOIN information_schema.constraint_column_usage ccu ON tc.constraint_name = ccu.constraint_name
    WHERE tc.constraint_type = 'FOREIGN KEY' AND tc.table_name = 'waiting_queue' AND ccu.table_name = 'tenants'
) THEN
    RAISE EXCEPTION 'P1 Validate Error: tenant_id FK -> tenants missing on waiting_queue';
END IF;

RAISE NOTICE 'P1 waiting_queue ACUITY validate: OK ? All tables, columns, constraints, and RLS verified successfully.';
END $$;

```

```

=====
FILE: migrations\p1_06_exam_rooms_down.sql
=====

```

```

-- =====
-- P1 PHASE exam_rooms PROVISIONING ? DOWN
-- Reverses modifications and drops exam_rooms table.
-- =====
BEGIN;

DROP POLICY IF EXISTS rls_exam_rooms_tenant_isolation ON exam_rooms;
DROP TABLE IF EXISTS exam_rooms CASCADE;

COMMIT;

```

```

=====
FILE: migrations\p1_06_exam_rooms_up.sql

```

=====

```
-- =====
-- P1 PHASE exam_rooms PROVISIONING ? UP
-- Creates the exam_rooms table, configures RLS, and inserts default clinics.
-- =====
```

BEGIN;

-- 1. Create exam\_rooms table if not exists

```
CREATE TABLE IF NOT EXISTS exam_rooms (
    id SERIAL PRIMARY KEY,
    tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    room_number VARCHAR(50) NOT NULL,
    name_ar TEXT DEFAULT '',
    name_en TEXT DEFAULT '',
    department_id INTEGER REFERENCES clinical_departments(id) ON DELETE SET NULL,
    status VARCHAR(30) DEFAULT 'Available'
    CHECK (status IN ('Available', 'Occupied', 'UnderMaintenance', 'Inactive')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

-- 2. Create index for performance

```
CREATE INDEX IF NOT EXISTS idx_exam_rooms_tenant_status ON exam_rooms (tenant_id, status);
```

-- 3. Enable RLS and Force it

```
ALTER TABLE exam_rooms ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE exam_rooms FORCE ROW LEVEL SECURITY;
```

-- 4. Create or replace tenant isolation policy

```
DROP POLICY IF EXISTS rls_exam_rooms_tenant_isolation ON exam_rooms;
CREATE POLICY rls_exam_rooms_tenant_isolation ON exam_rooms
    FOR ALL
    TO PUBLIC
    USING (tenant_id = current_setting('app.tenant_id', true)::integer);
```

-- 5. Seed default clinics/rooms for Tenant 1

```
INSERT INTO exam_rooms (tenant_id, room_number, name_ar, name_en, status)
VALUES
    (1, 'Room 101', '??????? ?????? 1', 'General Clinic 1', 'Available'),
    (1, 'Room 102', '????? ??????? 1', 'Pediatric Clinic 1', 'Available'),
    (1, 'Room 103', '????? ?????? ??????? 1', 'OB-GYN Clinic 1', 'Available'),
    (1, 'Room 104', '????? ??????? 1', 'Dental Clinic 1', 'Available'),
    (1, 'Triage-01', '???? ?????? 1', 'Triage Room 1', 'Available'),
    (1, 'Bed-A', '???? ??????? A', 'Emergency Bed A', 'Available')
ON CONFLICT DO NOTHING;
```

COMMIT;

=====

FILE: migrations\p1\_06\_exam\_rooms\_validate.sql

=====

```

-- =====
-- P1 PHASE exam_rooms PROVISIONING ? VALIDATE
-- Asserts that exam_rooms has been created and configured successfully.
-- =====

DO $$
BEGIN
    -- 1. Assert table exists
    IF NOT EXISTS (SELECT 1 FROM information_schema.tables WHERE table_name = 'exam_rooms') THEN
        RAISE EXCEPTION 'P1 Validate Error: exam_rooms table is missing';
    END IF;

    -- 2. Assert columns exist
    IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'exam_rooms' AND column_name =
'room_number') THEN
        RAISE EXCEPTION 'P1 Validate Error: exam_rooms.room_number is missing';
    END IF;

    -- 3. Assert RLS is enabled and forced
    IF NOT EXISTS (
        SELECT 1 FROM pg_class c JOIN pg_namespace n ON n.oid = c.relnamespace
        WHERE c.relname = 'exam_rooms' AND n.nspname = 'public'
        AND c.relrowsecurity = TRUE AND c.relforcerowsecurity = TRUE
    ) THEN
        RAISE EXCEPTION 'P1 Validate Error: RLS or FORCE RLS not enabled on exam_rooms';
    END IF;

    -- 4. Assert policy exists
    IF NOT EXISTS (
        SELECT 1 FROM pg_policies
        WHERE tablename = 'exam_rooms' AND policyname = 'rls_exam_rooms_tenant_isolation'
    ) THEN
        RAISE EXCEPTION 'P1 Validate Error: rls_exam_rooms_tenant_isolation policy missing on exam_rooms';
    END IF;

    -- 5. Assert seed rows exist
    IF (SELECT count(*) FROM exam_rooms WHERE tenant_id = 1) = 0 THEN
        RAISE EXCEPTION 'P1 Validate Error: Default seed exam_rooms are missing for Tenant 1';
    END IF;

    RAISE NOTICE 'P1 exam_rooms PROVISIONING validate: OK ? Table, columns, RLS, and seed data verified succes
sfully.';
END $$;

```

```

=====
FILE: migrations\README_E4.md
=====

```

# E4 ? Radiology (RIS + PACS metadata) candidate migrations

These migrations are **\*\*candidates\*\*** for the E4 epic. They are **\*\*NOT** executed automatically\*\* and must NOT be run against production without an explicit owner gate + backup.

Every migration file exports `{ up, validate, down }` and runs against a `pg` client/pool passed in by the caller. They are **idempotent** (safe to re-run) and follow the canonical RLS posture already enforced on the 150 FORCE-RLS tables in this codebase:

- new tenant-scoped tables carry `tenant\_id INTEGER NOT NULL`
- `ALTER TABLE ... ENABLE ROW LEVEL SECURITY; ... FORCE ROW LEVEL SECURITY;`
- a single canonical FOR ALL policy:  
`USING (tenant\_id = NULLIF(current\_setting('app.tenant\_id', true), '')::integer)`  
`WITH CHECK (...)` ? identical predicate (matches `rls\_local\_dry\_run\_3\_tables.js`)
- a `(tenant\_id, ...)` composite index

`down` drops the policy, disables RLS, drops the table's own indexes, then the table.

## ## Files

| Order | File                                      | Adds                                                   |
|-------|-------------------------------------------|--------------------------------------------------------|
| 1     | `e4_01_rad_worklist_up_validate_down.js`  | `rad_exams` (RIS worklist state machine)               |
| 2     | `e4_02_dicom_studies_up_validate_down.js` | `dicom_studies` (DICOM study METADATA only ? no bytes) |
| 3     | `e4_03_rad_reports_up_validate_down.js`   | `rad_reports` (structured report + critical + signing) |

## ## Manual run convention (LOCAL ONLY, gated)

```
...  
NODE_ENV=development node -e "const {Pool}=require('pg');const p=new Pool();(async()=>{ \  
  const m=require('./migrations/e4_01_rad_worklist_up_validate_down.js'); \  
  await m.up(p); console.log(await m.validate(p)); await p.end(); }())"  
...
```

`up` runtime needs `app.tenant\_id` set per-session by the app's monkey-patched `pool.query` (see `db\_postgres.js`); when running raw migrations as a superuser this is irrelevant for DDL.

## ## DEPLOY STEPS ? NOT DEPLOYED

See the final report / commit message. Nothing here has been executed or pushed.

```
=====
FILE: migrations\saudi_compliance_up.sql
=====
```

```
-- saudi_compliance_up.sql
-- Migrations for ZATCA Phase 2, CBAHI OVR, and PDPL Consent

-- 1. PDPL Patient Consent Columns
ALTER TABLE patients ADD COLUMN IF NOT EXISTS privacy_consent_signed BOOLEAN DEFAULT FALSE;
ALTER TABLE patients ADD COLUMN IF NOT EXISTS privacy_consent_date TIMESTAMP;

-- 2. ZATCA Phase 2 Hashing & Chaining Columns
ALTER TABLE invoices ADD COLUMN IF NOT EXISTS invoice_hash VARCHAR(64);
ALTER TABLE invoices ADD COLUMN IF NOT EXISTS previous_invoice_hash VARCHAR(64);

-- 3. quality_incidents facility_id Column
ALTER TABLE quality_incidents ADD COLUMN IF NOT EXISTS facility_id INTEGER;
```

```

-- 4. CBAHI OVR Incident Table (fallback/extra)
CREATE TABLE IF NOT EXISTS clinical_incidents (
    id SERIAL PRIMARY KEY,
    tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    reporter_id INTEGER REFERENCES system_users(id),
    reporter_name VARCHAR(150),
    incident_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    department VARCHAR(100),
    severity VARCHAR(50), -- Low, Medium, High, Extreme
    description TEXT,
    action_taken TEXT,
    status VARCHAR(50) DEFAULT 'Pending', -- Pending, Under Review, Resolved
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

=====
FILE: migrations\seeds\ex_02_rbac_seed.sql
=====

-- =====
-- ex_02_rbac_seed.sql (baseline RBAC matrix seed; run AFTER ex_02_rbac_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ???? ?????? permissions (?????) + ???? role_permissions ??? ?????? ???? ???? ?????
-- ROLE_PERMISSIONS ??????? ?? server.js (???? ??????? ??????) ??? ?????? ??????? ??????.
-- ??? ?????? ????????: ON CONFLICT DO NOTHING ??? ??? ????????.
-- ??????: role_permissions ??? FORCE RLS? ??? ?????? ?? ???? ?? table owner / superuser
-- (???? ?????? RLS) ?? ??? ???? set_config('app.tenant_id',...) ????????. ?????? ?????? ??????
-- ???? per-tenant ???? tenant_id ?????? (???? ??? ???? ?????? RLS).
-- =====
BEGIN;

-- ----- 1) Global permission catalog -----
INSERT INTO permissions (key, description) VALUES
    ('orders:create', 'Create unified clinical orders (lab/rad/med/consult)'),
    ('orders:view', 'View unified clinical orders'),
    ('orders:update', 'Update order status / cancel'),
    ('lab:order', 'Create lab orders'),
    ('lab:result', 'Enter/view lab results'),
    ('rad:order', 'Create radiology orders'),
    ('rad:result', 'Enter/view radiology results'),
    ('rx:create', 'Create prescriptions'),
    ('rx:dispense', 'Dispense medications (pharmacy)'),
    ('patients:view', 'View patient records'),
    ('patients:manage', 'Create/update patient records'),
    ('appointments:manage', 'Manage appointments / scheduling'),
    ('invoices:manage', 'Create/manage invoices and billing'),
    ('reports:view', 'View clinical/operational reports'),
    ('hr:manage', 'Manage employees / HR records'),
    ('settings:manage', 'Manage system settings'),
    ('rbac:manage', 'Manage roles and permissions matrix')

```

```
ON CONFLICT (key) DO NOTHING;
```

```
-- ----- 2) Per-tenant role -> permission_key matrix -----
-- baseline role grants mirror server.js ROLE_PERMISSIONS module lists, expressed as fine-grained keys.
-- Admin is intentionally NOT seeded as rows: requirePermission() short-circuits Admin to next()
-- (mirrors ROLE_PERMISSIONS '*'), and absent rows fall back to requireRole ? both non-breaking.
DO $$
DECLARE
    t RECORD;
    grant_row RECORD;
BEGIN
    FOR t IN SELECT id FROM tenants LOOP
        -- I1 fix: role_permissions is under FORCE RLS. Without binding app.tenant_id, the policy
        -- WITH CHECK rejects every INSERT when this seed runs as a NON-superuser app role (silent 0 rows).
        -- Set the tenant context per loop so each tenant's rows pass the policy. A superuser/owner run
        -- BYPASSES RLS entirely (set_config is harmless there) ? so this is correct for EITHER role.
        PERFORM set_config('app.tenant_id', t.id::text, false);
        FOR grant_row IN
            SELECT * FROM (VALUES
                ('Doctor',      'orders:create'),
                ('Doctor',      'orders:view'),
                ('Doctor',      'orders:update'),
                ('Doctor',      'lab:order'),
                ('Doctor',      'rad:order'),
                ('Doctor',      'rx:create'),
                ('Doctor',      'patients:view'),
                ('Doctor',      'reports:view'),
                ('Nurse',       'orders:view'),
                ('Nurse',       'patients:view'),
                ('Pharmacist',  'orders:view'),
                ('Pharmacist',  'rx:dispense'),
                ('Lab Technician', 'orders:view'),
                ('Lab Technician', 'lab:result'),
                ('Radiologist', 'orders:view'),
                ('Radiologist', 'rad:result'),
                ('Reception',   'patients:view'),
                ('Reception',   'patients:manage'),
                ('Reception',   'appointments:manage'),
                ('Finance',     'invoices:manage'),
                ('Finance',     'reports:view'),
                ('HR',          'hr:manage'),
                ('HR',          'reports:view'),
                ('IT',          'settings:manage')
            ) AS v(role, permission_key)
        LOOP
            INSERT INTO role_permissions (tenant_id, role, permission_key)
            VALUES (t.id, grant_row.role, grant_row.permission_key)
            ON CONFLICT (tenant_id, role, permission_key) DO NOTHING;
        END LOOP;
    END LOOP;
END $$;

COMMIT;
```

```
=====
FILE: public\js\admin.js
=====
```

```
// --- SECURITY UTILS ---
```

```
window.escapeHTML = function escapeHTML(str) {
  if (str === null || str === undefined) return '';
  return String(str)
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#39;');
};
```

```
// ===== jumanaMedical ERP - Admin Panel =====
```

```
let currentUser = null;
```

```
let allUsers = [];
```

```
let selectedFacilityType = 'hospital';
```

```
const NAV_ITEMS = [
```

```
  { idx:0, icon: '?', en: 'Dashboard', ar: '???? ?????' },
  { idx:1, icon: '?', en: 'Reception', ar: '?????????' },
  { idx:2, icon: '?', en: 'Appointments', ar: '?????????' },
  { idx:3, icon: '????', en: 'Doctor Station', ar: '???? ??????' },
  { idx:4, icon: '?', en: 'Laboratory', ar: '?????????' },
  { idx:5, icon: '?', en: 'Radiology', ar: '????????' },
  { idx:6, icon: '?', en: 'Pharmacy', ar: '?????????' },
  { idx:7, icon: '?', en: 'HR', ar: '???????? ??????' },
  { idx:8, icon: '?', en: 'Finance', ar: '?????????' },
  { idx:9, icon: '??', en: 'Insurance', ar: '?????????' },
  { idx:10, icon: '?', en: 'Inventory', ar: '?????????' },
  { idx:11, icon: '????', en: 'Nursing', ar: '?????????' },
  { idx:12, icon: '?', en: 'Waiting Queue', ar: '????? ??????????' },
  { idx:13, icon: '?', en: 'Patient Accounts', ar: '??????? ??????' },
  { idx:14, icon: '?', en: 'Reports', ar: '?????????' },
  { idx:15, icon: '??', en: 'Messaging', ar: '?????????' },
  { idx:16, icon: '?', en: 'Catalog', ar: '?????????' },
  { idx:17, icon: '?', en: 'Dept Requests', ar: '????? ??????????' },
  { idx:18, icon: '?', en: 'Surgery', ar: '?????????' },
  { idx:19, icon: '?', en: 'Blood Bank', ar: '??? ?????' },
  { idx:20, icon: '?', en: 'Consent Forms', ar: '?????????????' },
  { idx:21, icon: '?', en: 'Emergency', ar: '?????????' },
  { idx:22, icon: '??', en: 'Inpatient ADT', ar: '?????????' },
  { idx:23, icon: '?', en: 'ICU', ar: '????????? ??????????' },
  { idx:24, icon: '?', en: 'CSSD', ar: '????????? ??????????' },
  { idx:25, icon: '??', en: 'Dietary', ar: '?????????' },
  { idx:26, icon: '?', en: 'Infection Control', ar: '????????? ??????????' },
  { idx:27, icon: '?', en: 'Quality', ar: '?????????' },
  { idx:28, icon: '?', en: 'Maintenance', ar: '?????????' },
  { idx:29, icon: '?', en: 'Transport', ar: '??? ??????????' },
```

```

    { idx:30, icon:'?', en:'Medical Records', ar:'??????? ??????' },
    { idx:31, icon:'?', en:'Clinical Pharmacy', ar:'????????? ?????????' },
    { idx:32, icon:'?', en:'Rehabilitation', ar:'????? ?????????' },
    { idx:33, icon:'?', en:'Patient Portal', ar:'????? ?????????' },
    { idx:34, icon:'?', en:'ZATCA E-Invoice', ar:'????? ??????????' },
    { idx:35, icon:'?', en:'Telemedicine', ar:'????? ?? ???' },
    { idx:36, icon:'?', en:'Pathology', ar:'??? ?????????' },
    { idx:37, icon:'?', en:'Social Work', ar:'??????? ????????????' },
    { idx:38, icon:'?', en:'Mortuary', ar:'????? ?????????' },
    { idx:39, icon:'?', en:'CME', ar:'????????? ??????' },
    { idx:40, icon:'?', en:'Cosmetic Surgery', ar:'?????? ?????????' },
    { idx:41, icon:'?', en:'OB/GYN', ar:'????????? ??????????' },
    { idx:42, icon:'?', en:'Settings', ar:'????????????' }
  ];

  // Facility type department mappings
  const FACILITY_DEPTS = {
    hospital: NAV_ITEMS.map(n => n.idx), // All departments
    health_center: [0,1,2,3,4,5,6,7,8,9,11,12,13,14,15,20,21,30,33,34,35,41,42],
    clinic: [0,1,2,3,4,6,7,8,9,11,12,13,14,15,20,30,34,42]
  };

  const FACILITY_INFO = {
    hospital: { icon:'?', title:'??????', titleEn:'Hospital', subtitle:'???? ????????? ?????? ??????????', count: FACILITY_DEPTS.hospital.length },
    health_center: { icon:'?', title:'???? ???', titleEn:'Health Center', subtitle:'????? ????????? ????????? ??????', count: FACILITY_DEPTS.health_center.length },
    clinic: { icon:'?', title:'??????', titleEn:'Clinic / Dispensary', subtitle:'????? ?????????? ??????????', count: FACILITY_DEPTS.clinic.length }
  };

  const ROLES = ['Admin','Doctor','Nurse','Pharmacist','Lab Technician','Radiologist','Reception','Finance','HR','IT','Staff'];

  // ===== INIT =====
  (async function init() {
    try {
      const data = await API.get('/api/auth/me');
      currentUser = data.user;
      if (currentUser.role !== 'Admin') {
        document.body.innerHTML = '<div style="display:flex;align-items:center;justify-content:center;height:100vh;color:#ef4444;font-size:24px;font-family:Tajawal">? ?????? ???? ? ??? ?????? ?????? ??????</div>';
        return;
      }
    } catch {
      window.location.href = '/login.html';
      return;
    }
    document.getElementById('userName').textContent = currentUser.name;
    document.getElementById('userAvatar').textContent = currentUser.name.charAt(0);
    await loadUsers();
    await loadFacilityType();
    await loadSecuritySettings();
    renderFacilityCards();
  })();

```

```

})();

// ===== TAB SWITCHING =====
function switchTab(tab) {
  document.querySelectorAll('.tab').forEach(t => t.classList.remove('active'));
  document.querySelectorAll('.tab-content').forEach(c => c.classList.remove('active'));
  const tabBtn = document.getElementById('tab' + tab.charAt(0).toUpperCase() + tab.slice(1));
  const tabContent = document.getElementById('content' + tab.charAt(0).toUpperCase() + tab.slice(1));
  if (tabBtn) tabBtn.classList.add('active');
  if (tabContent) tabContent.classList.add('active');
}

// ===== TOAST =====
function showToast(msg, type = 'success') {
  let t = document.querySelector('.toast');
  if (!t) { t = document.createElement('div'); t.className = 'toast'; document.body.appendChild(t); }
  t.className = `toast toast-${type} show`;
  t.textContent = msg;
  setTimeout(() => t.classList.remove('show'), 3000);
}

// ===== USERS =====
async function loadUsers() {
  try {
    allUsers = await API.get('/api/settings/users');
    renderUsersTable();
    renderUserStats();
    populatePermUserSelect();
  } catch (e) { showToast('??? ?? ????? ?????????', 'error'); }
}

function renderUserStats() {
  const total = allUsers.length;
  const active = allUsers.filter(u => u.is_active === 1 || u.is_active === true).length;
  const admins = allUsers.filter(u => u.role === 'Admin').length;
  const doctors = allUsers.filter(u => u.role === 'Doctor').length;
  document.getElementById('userStats').innerHTML = `
    <div class="stat-card"><div class="icon" style="background:var(--grad1)">?</div><div class="label">?????
    ?????????</div><div class="value">${total}</div></div>
    <div class="stat-card"><div class="icon" style="background:var(--grad2)">?</div><div class="label">???</di
    v><div class="value" style="color:var(--success)">${active}</div></div>
    <div class="stat-card"><div class="icon" style="background:var(--grad3)">??</div><div class="label">?????<
    /div><div class="value" style="color:var(--warning)">${admins}</div></div>
    <div class="stat-card"><div class="icon" style="background:var(--grad4)">????</div><div class="label">????
    ?</div><div class="value" style="color:var(--info)">${doctors}</div></div>
  `;
  document.getElementById('userCount').textContent = total;
}

function renderUsersTable() {
  const tbody = document.getElementById('usersBody');
  if (!allUsers.length) {
    tbody.innerHTML = '<tr><td colspan="9"><div class="empty-state"><div class="icon">?</div><p>?? ??? ????
    ??</p></div></td></tr>';
  }
}

```

```

    return;
}
tbody.innerHTML = allUsers.map((u, i) => `<tr>
  <td>${i + 1}</td>
  <td><strong>${escapeHTML(u.display_name || '-')}</strong></td>
  <td><code style="color:var(--primary)">${escapeHTML(u.username)}</code></td>
  <td>${roleBadge(u.role)}</td>
  <td>${escapeHTML(u.speciality || '-')}</td>
  <td class="ip-cell">${escapeHTML(u.last_ip || '?')}</td>
  <td>${u.is_active ? '<span class="badge badge-success">???'</span>' : '<span class="badge badge-danger">???'
??</span>'}</td>
  <td style="font-size:12px;color:var(--text3)">${u.created_at ? new Date(u.created_at).toLocaleDateString('
ar-SA') : '-'}</td>
  <td>
    <button class="btn btn-sm btn-outline" onclick="editUser(${u.id})" title="?????">??</button>
    <button class="btn btn-sm btn-outline" onclick="toggleUser(${u.id},${u.is_active ? 0 : 1})" title="${u.is
s_active ? '?????' : '?????'}">${u.is_active ? '?' : ''}</button>
    <button class="btn btn-sm btn-danger btn-icon" onclick="deleteUser(${u.id})" title="???">??</button>
  </td>
</tr>`).join('');
}

function roleBadge(role) {
  const map = { Admin:'primary', Doctor:'info', Nurse:'success', Pharmacist:'warning', 'Lab Technician':'info'
, Radiologist:'info', Reception:'success', Finance:'warning', HR:'warning', IT:'primary', Staff:'info' };
  return `<span class="badge badge-${map[role] || 'info'}">${escapeHTML(role)}</span>`;
}

// ===== ADD/EDIT USER MODAL =====
function showAddUserModal(user) {
  const isEdit = !!user;
  const overlay = document.createElement('div');
  overlay.className = 'modal-overlay';
  overlay.innerHTML = `<div class="modal">
  <h3>${isEdit ? '?? ????? ??????' : '? ????? ?????? ?????'}</h3>
  <form id="userForm" onsubmit="event.preventDefault(); saveUser(${isEdit ? user.id : 'null'})">
    <div class="form-grid">
      <div class="form-group"><label>????? ??????</label><input class="form-input" id="muName" value="${isEd
it ? escapeHTML(user.display_name || '') : ''}"></div>
      <div class="form-group"><label>??? ??????</label><input class="form-input" id="muUsername" value="${
isEdit ? escapeHTML(user.username || '') : ''}"></div>
      <div class="form-group"><label>????? ?????? ${isEdit ? '(????? ????? ??????)' : ''}</label><input typ
e="password" class="form-input" id="muPassword" placeholder="${isEdit ? '?????' : '???? ??????'}"></div>
      <div class="form-group"><label>?????</label><select class="form-input" id="muRole" onchange="toggleDoc
torFields()">
        ${ROLES.map(r => `<option value="${r}" ${isEdit && user.role === r ? 'selected' : ''}>${r}</option>`
).join('')}
      </select></div>
      <div class="form-group" id="muSpecDiv" style="display:${isEdit && user.role === 'Doctor' ? 'flex' : 'n
one'}"><label>?????</label><input class="form-input" id="muSpec" value="${isEdit ? escapeHTML(user.speciality
|| '') : ''}"></div>
      <div class="form-group" id="muCommDiv" style="display:${isEdit && user.role === 'Doctor' ? 'flex' : 'n
one'}">
        <label>???????</label>

```

```

        <div style="display:flex;gap:8px">
            <select class="form-input" id="muCommType" style="width:120px">
                <option value="percentage" ${isEdit && user.commission_type === 'percentage' ? 'selected' : ''}>
???? %</option>
                <option value="fixed" ${isEdit && user.commission_type === 'fixed' ? 'selected' : ''}>????</opti
on>

            </select>
            <input type="number" class="form-input" id="muCommValue" value="${isEdit ? user.commission_value |
| 0 : 0}" step="0.1" min="0">
            </div>
        </div>
    </div>
    <div style="display:flex;gap:12px;margin-top:20px">
        <button type="submit" class="btn btn-primary" style="flex:1">? ${isEdit ? '?????' : '?????'}</button>
        <button type="button" class="btn btn-outline" onclick="this.closest('.modal-overlay').remove()" style=
"flex:1">????</button>
    </div>
</form>
</div>`;
overlay.onclick = e => { if (e.target === overlay) overlay.remove(); };
document.body.appendChild(overlay);
}

window.toggleDoctorFields = () => {
    const role = document.getElementById('muRole')?.value;
    const show = role === 'Doctor';
    const sd = document.getElementById('muSpecDiv');
    const cd = document.getElementById('muCommDiv');
    if (sd) sd.style.display = show ? 'flex' : 'none';
    if (cd) cd.style.display = show ? 'flex' : 'none';
};

window.saveUser = async (id) => {
    const username = document.getElementById('muUsername').value.trim();
    const password = document.getElementById('muPassword').value.trim();
    const display_name = document.getElementById('muName').value.trim();
    const role = document.getElementById('muRole').value;
    const speciality = document.getElementById('muSpec')?.value || '';
    const commission_type = document.getElementById('muCommType')?.value || 'percentage';
    const commission_value = parseFloat(document.getElementById('muCommValue')?.value) || 0;

    if (!username) return showToast('???? ??? ???????', 'error');
    if (!id && !password) return showToast('???? ??? ???????', 'error');

    try {
        const data = { username, display_name, role, speciality, commission_type, commission_value, permissions: '
' };
        if (password) data.password = password;
        if (id) {
            data.is_active = 1;
            await API.put(`/api/settings/users/${id}`, data);
            showToast('?? ????? ??????? ?');
        } else {
            await API.post('/api/settings/users', data);

```

```

    showToast('?? ????? ???????? ?');
  }
  document.querySelector('.modal-overlay')?.remove();
  await loadUsers();
} catch (e) { showToast(e.message || '??? ?? ?????', 'error'); }
};

window.editUser = (id) => {
  const user = allUsers.find(u => u.id === id);
  if (user) showAddUserModal(user);
};

window.toggleUser = async (id, active) => {
  try {
    const user = allUsers.find(u => u.id === id);
    if (!user) return;
    await API.put(`/api/settings/users/${id}`, { ...user, is_active: active, password: '' });
    showToast(active ? '?? ????? ???????? ?' : '?? ????? ???????? ?');
    await loadUsers();
  } catch (e) { showToast('??', 'error'); }
};

window.deleteUser = async (id) => {
  if (!confirm('?? ??? ????? ?? ??? ??? ?????????? ??? ???????? ?? ??? ???????? ???.')) return;
  try {
    await API.delete(`/api/settings/users/${id}`);
    showToast('?? ????? ?');
    await loadUsers();
  } catch (e) { showToast(e.message || '??? ?? ?????', 'error'); }
};

// ===== PERMISSIONS =====
function populatePermUserSelect() {
  const sel = document.getElementById('permUser');
  sel.innerHTML = '<option value="">-- ????? ?????? --</option>' +
    allUsers.map(u => `<option value="${escapeHTML(u.id)}">${escapeHTML(u.display_name || u.username)} (${escapeHTML(u.role)}</option>`).join('');
}

window.loadUserPermissions = () => {
  const userId = document.getElementById('permUser').value;
  if (!userId) { document.getElementById('permsContainer').style.display = 'none'; return; }

  const user = allUsers.find(u => u.id === parseInt(userId));
  if (!user) return;

  document.getElementById('permsContainer').style.display = 'block';
  document.getElementById('savePermBtn').style.display = 'inline-flex';
  document.getElementById('selectAllBtn').style.display = 'inline-flex';
  document.getElementById('deselectAllBtn').style.display = 'inline-flex';

  const currentPerms = (user.permissions || '').split(',').filter(p => p);
  const isAdmin = user.role === 'Admin';

```

```

// Get departments based on current facility type
const allowedDepts = FACILITY_DEPTS[selectedFacilityType] || FACILITY_DEPTS.hospital;
const depts = NAV_ITEMS.filter(n => allowedDepts.includes(n.idx));

document.getElementById('permsGrid').innerHTML = depts.map(n => {
  const checked = isAdmin || currentPerms.includes(n.idx.toString());
  return `<label class="perm-item ${checked ? 'checked' : ''}">
    <input type="checkbox" value="${n.idx}" ${checked ? 'checked' : ''} ${isAdmin ? 'disabled' : ''} onChange
e="this.parentElement.classList.toggle('checked',this.checked)">
    <span>${n.icon}</span>
    <span>${n.ar}</span>
  </label>`;
}).join('');
};

window.savePermissions = async () => {
  const userId = document.getElementById('permUser').value;
  if (!userId) return;
  const user = allUsers.find(u => u.id === parseInt(userId));
  if (!user) return;

  const perms = Array.from(document.querySelectorAll('#permsGrid input:checked')).map(cb => cb.value).join(',');
};

try {
  await API.put(`/api/settings/users/${userId}`, { ...user, permissions: perms, password: '' });
  showToast('?? ?? ??????? ?');
  await loadUsers();
} catch (e) { showToast('?? ? ????', 'error'); }
};

window.selectAllPerms = () => {
  document.querySelectorAll('#permsGrid input').forEach(cb => { cb.checked = true; cb.parentElement.classList.add('checked'); });
};

window.deselectAllPerms = () => {
  document.querySelectorAll('#permsGrid input').forEach(cb => { cb.checked = false; cb.parentElement.classList.remove('checked'); });
};

// ===== FACILITY TYPE =====
async function loadFacilityType() {
  try {
    const settings = await API.get('/api/settings');
    selectedFacilityType = settings.facility_type || 'hospital';
  } catch { selectedFacilityType = 'hospital'; }
}

function renderFacilityCards() {
  const grid = document.getElementById('facilityGrid');
  grid.innerHTML = Object.entries(FACILITY_INFO).map(([key, info]) => {
    const depts = FACILITY_DEPTS[key];
    const deptNames = depts.map(idx => {
      const nav = NAV_ITEMS.find(n => n.idx === idx);
      return nav ? `${nav.icon} ${nav.ar}` : '';
    });
  });
}

```

```

    }).filter(Boolean);

    return `<div class="facility-card ${selectedFacilityType === key ? 'selected' : ''}" onclick="selectFacility('${key}')">
      <span class="f-icon">${info.icon}</span>
      <div class="f-title">${info.title}</div>
      <div class="f-subtitle">${info.subtitle}</div>
      <div class="f-count">${info.count} ???</div>
      <div class="dept-list">${deptNames.join('<br>')}</div>
    </div>`;
  }).join('');
}

window.selectFacility = (type) => {
  selectedFacilityType = type;
  document.querySelectorAll('.facility-card').forEach(c => c.classList.remove('selected'));
  event.currentTarget.classList.add('selected');
};

window.saveFacilityType = async () => {
  try {
    await API.put('/api/settings', { facility_type: selectedFacilityType });
    showToast('?? ??? ??? ?????? ? ? ??? ???? ???? ? ? ????');
    renderFacilityCards();
  } catch (e) { showToast('?? ? ????', 'error'); }
};

// ===== SECURITY SETTINGS =====
window.loadSecuritySettings = async () => {
  try {
    const settings = await API.get('/api/settings');
    document.getElementById('rlsToggle').checked = settings.rls_enabled === 'true' || settings.rls_enabled === true || settings.rls_enabled === 1 || settings.rls_enabled === '1';
    document.getElementById('mfaToggle').checked = settings.mfa_enabled === 'true' || settings.mfa_enabled === true || settings.mfa_enabled === 1 || settings.mfa_enabled === '1';
    document.getElementById('encryptionToggle').checked = settings.encryption_enabled === 'true' || settings.encryption_enabled === true || settings.encryption_enabled === 1 || settings.encryption_enabled === '1';
  } catch (e) { console.error('Error loading security settings:', e); }
};

window.saveSecuritySettings = async () => {
  try {
    const rls_enabled = document.getElementById('rlsToggle').checked;
    const mfa_enabled = document.getElementById('mfaToggle').checked;
    const encryption_enabled = document.getElementById('encryptionToggle').checked;

    await API.put('/api/settings', { rls_enabled, mfa_enabled, encryption_enabled });
    showToast('?? ??? ?????? ?????? ?????? ?????? ?');
  } catch (e) { showToast('?? ? ? ???? ????', 'error'); }
};

// ===== LOGOUT =====
window.logout = async () => {
  try { await API.post('/api/auth/logout'); } catch {}
}

```

```
    window.location.href = '/login.html';
  };
```

```
=====
FILE: public\js\api.js
=====
```

```
// ===== API Client =====
const API = {
  async request(url, options = {}) {
    const res = await fetch(url, {
      ...options,
      headers: { 'Content-Type': 'application/json', ...options.headers },
      credentials: 'same-origin'
    });
    if (res.status === 401) {
      window.location.href = '/login.html';
      throw new Error('Unauthorized');
    }
    if (!res.ok) {
      const errData = await res.json().catch(() => ({}));
      throw new Error(errData.error || `HTTP error ${res.status}`);
    }
    return res.json();
  },
  get: (url) => API.request(url),
  post: (url, data) => API.request(url, { method: 'POST', body: JSON.stringify(data) }),
  put: (url, data) => API.request(url, { method: 'PUT', body: JSON.stringify(data) }),
  patch: (url, data) => API.request(url, { method: 'PATCH', body: JSON.stringify(data) }),
  del: (url) => API.request(url, { method: 'DELETE' }),
  delete: (url) => API.request(url, { method: 'DELETE' })
};
```